Chapter 08: Pre-trained CNN Architecture and Data Argumentation

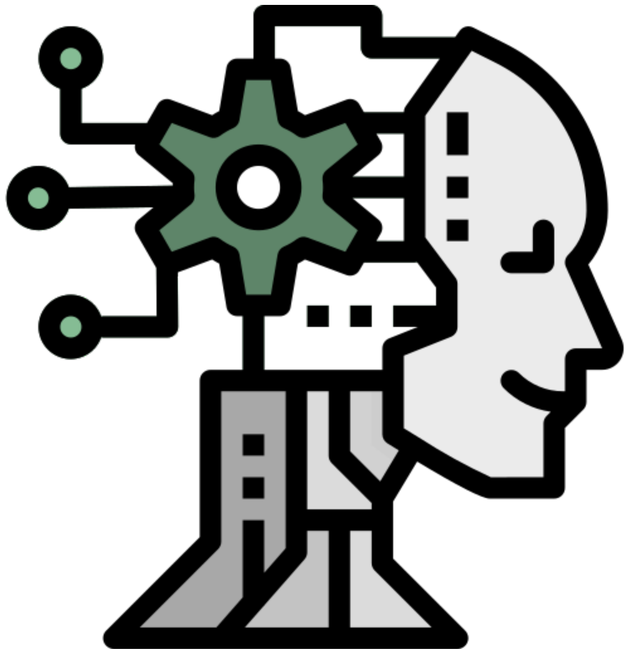
บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรูญโรจน์
คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

หัวข้อ

วิธีการสนับสนุนการฝึกสอนแบบจำลองโครงข่ายประสาทแบบคอนโวลูชัน

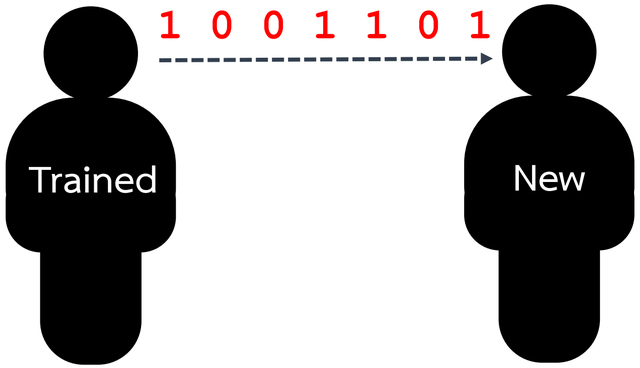
- การโอนถ่ายองค์ความรู้ (Transfer Learning)
- เทคนิคการเพิ่มจำนวนข้อมูล (Data Augmentation)

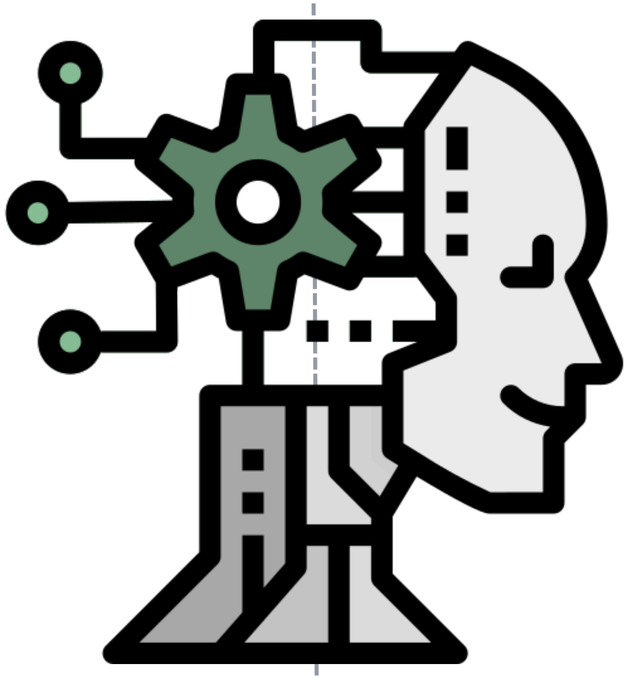
ปัญหาของการเรียนรู้เชิงลึก

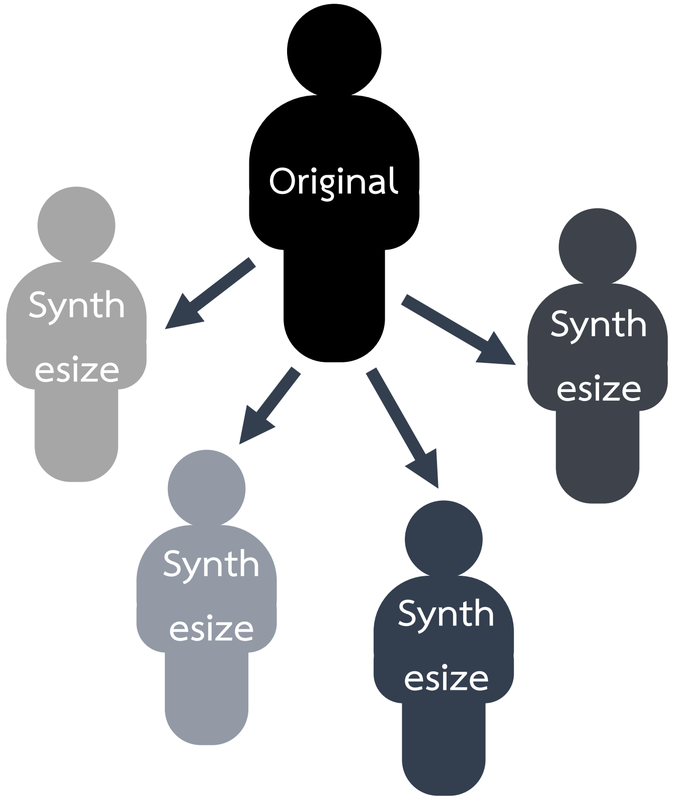


การเรียนรู้เชิงลึก (หรือที่เรียกกันว่า Deep Learning) เป็นวิธีการที่มีประสิทธิภาพในงานทางด้านต่าง ๆ อาทิเช่น การรู้จำรูปภาพ (image recognition) การตรวจจับวัตถุ (Object Detection) การแบ่งส่วนของภาพ (Image Segmentation) หรือการขจัดสัญญาณรบกวนในภาพ (image Denoising) เป็นต้น และได้รับความนิยมเป็นอย่างมากในปัจจุบัน แต่ถึงอย่างไรก็ตาม หนึ่งใน***ข้อจำกัดหลัก***ของการเรียนรู้เชิงลึก คือ ***ต้องการข้อมูลฝึกสอน*** (Training Image) ***จำนวนมหาศาล*** ซึ่งแบบจำลองที่มีประสิทธิภาพต่าง ๆ ที่ใช้งานจริง ๆ จะอาศัยภาพฝึกสอนสูงถึง 1,000,000 ภาพ เพื่อให้ได้รับแบบจำลองที่มีประสิทธิภาพและทนทานต่อสภาพแวดล้อมที่มีความไม่แน่นอนสูง

ปัญหาของการเรียนรู้เชิงลึก



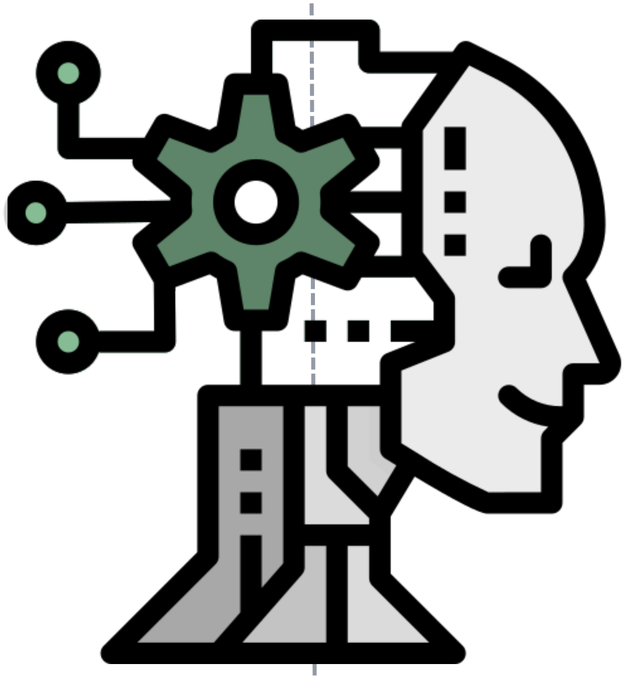


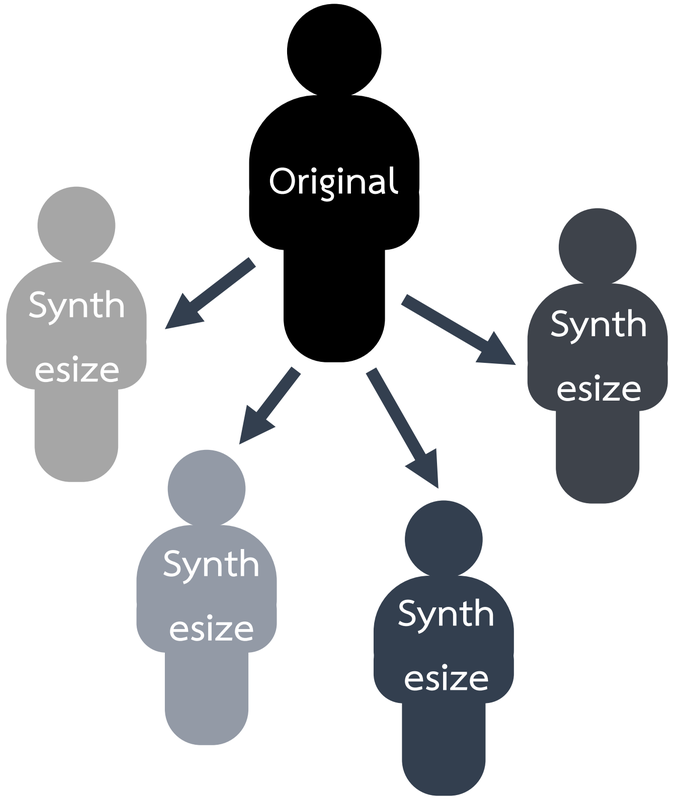


Transfer Learning

Data Augmentation

ปัญหาของการเรียนรู้เชิงลึก

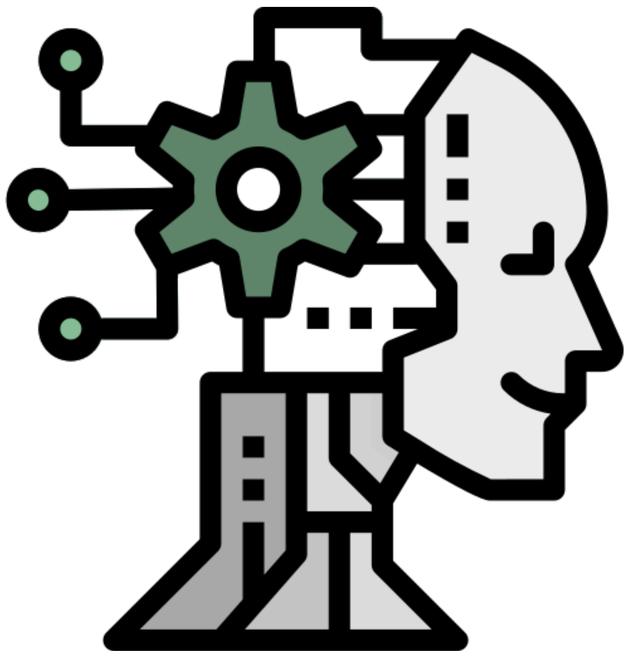




Transfer Learning

Data Augmentation

Data Augmentation



ถึงแม้ว่าการใช้งาน Transfer Learning จากแบบจำลองทั่วไปของ โครงข่ายประสาทแบบคอนโวลูชัน อาทิเช่น LeNet, AlexNet, ResNet, VGGNet, และ Inception จะประสบความสำเร็จก็ตาม แต่กรณีที่เราต้องนำมาใช้งานกับฐานข้อมูลที่ค่อนข้างมีปริมาณภาพฝึกสอนน้อย (น้อยกว่า 1,000 รูป) ก็ยังไม่เพียงพอสำหรับการฝึกสอนแบบจำลองให้มีประสิทธิภาพ

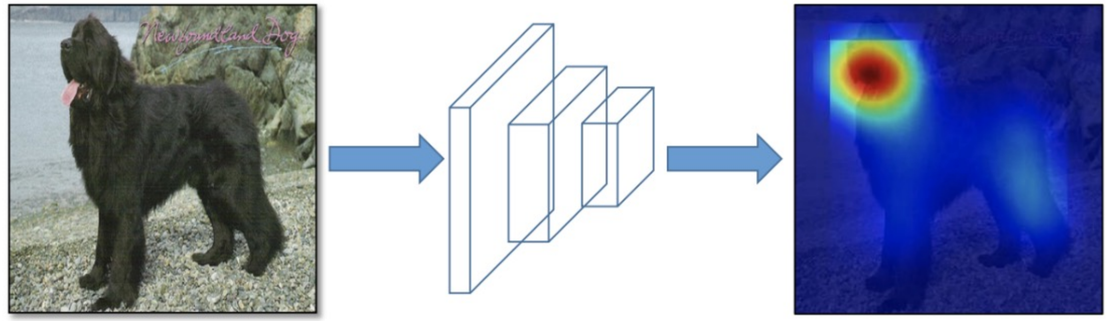
ดังนั้น เทคนิคการเพิ่มจำนวนข้อมูลจากข้อมูลที่มีอยู่จะมีความจำเป็นอย่างยิ่ง ตัวอย่างเช่น ถ้าฐานข้อมูลมีรูปภาพจำนวน 500 รูป และต้องการเพิ่มภาพฝึกสอนจาก 500 รูปไปเป็น 1,000 รูป อาจจะอาศัยการแปลงภาพเชิงเรขาคณิตบนระนาบสองมิติมาใช้งาน อาทิเช่น การเลื่อนภาพ การหมุนภาพ การย่อขยายขนาด การกลับรูปซ้ายขวาหรือบนล่าง การสุ่มตัดบางส่วนของภาพ เป็นต้น

https://www.bualabs.com/archives/1170/what-is-data-augmentation-benefits-data-augmentaion-deep-learning-training-model-deep-neural-network-regularization-ep-1/

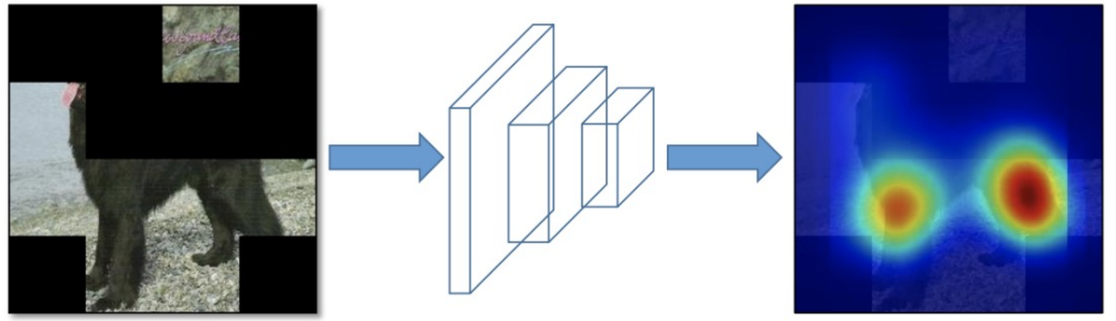
Data Augmentation

การเพิ่มข้อมูลเป็นวิธีที่ช่วยให้แบบจำลองมีประสิทธิภาพที่ดีขึ้น โดยการเพิ่มข้อมูลฝึกสอนที่มีอยู่อย่างจำกัดให้มีจำนวนมากขึ้นและหลากหลายมากยิ่งขึ้น สิ่งนี้ส่งผลทำให้แบบจำลองจะต้องค้นหาคุณสมบัติใหม่ในข้อมูลเพื่อจดจำหรือระบุวัตถุภายในรูปภาพที่น่าสนใจแทนที่จะใช้คุณสมบัติเพียงบางอย่าง ซึ่งสามารถแบ่งออกได้เป็น 3 ประเภท ได้แก่

- **Distortion**
  - Photometric Distortion
  - Geometric Distortion
- **Image Occlusion**
  - Random Erase
  - Cutout
  - Hide and Seek
  - Grid Mask
  - MixUp
- **Data Augmentation Techniques in YOLOv4**
  - CutMix
  - Mosaic



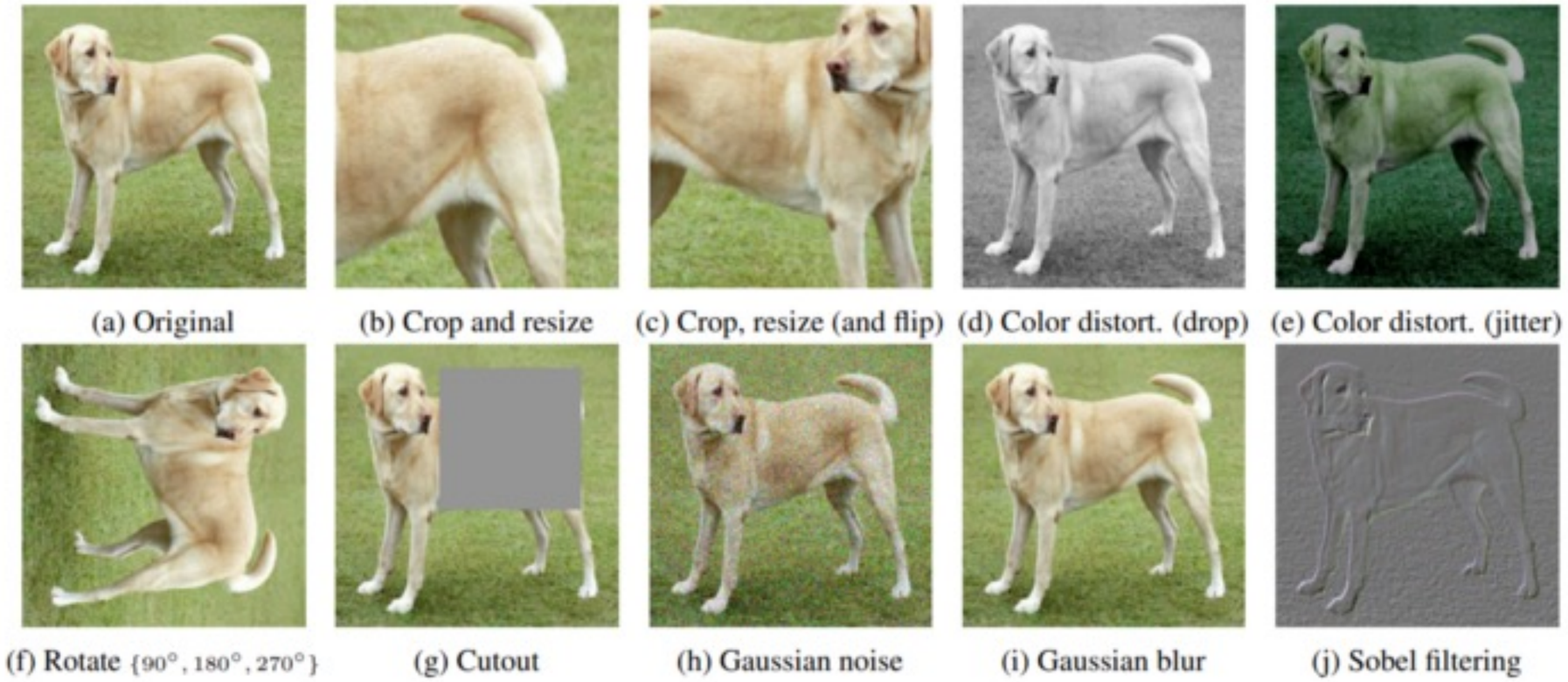
Full image



Randomly hidden patches

https://blog.roboflow.com/yolov4-data-augmentation/

Distortion Data Augmentation

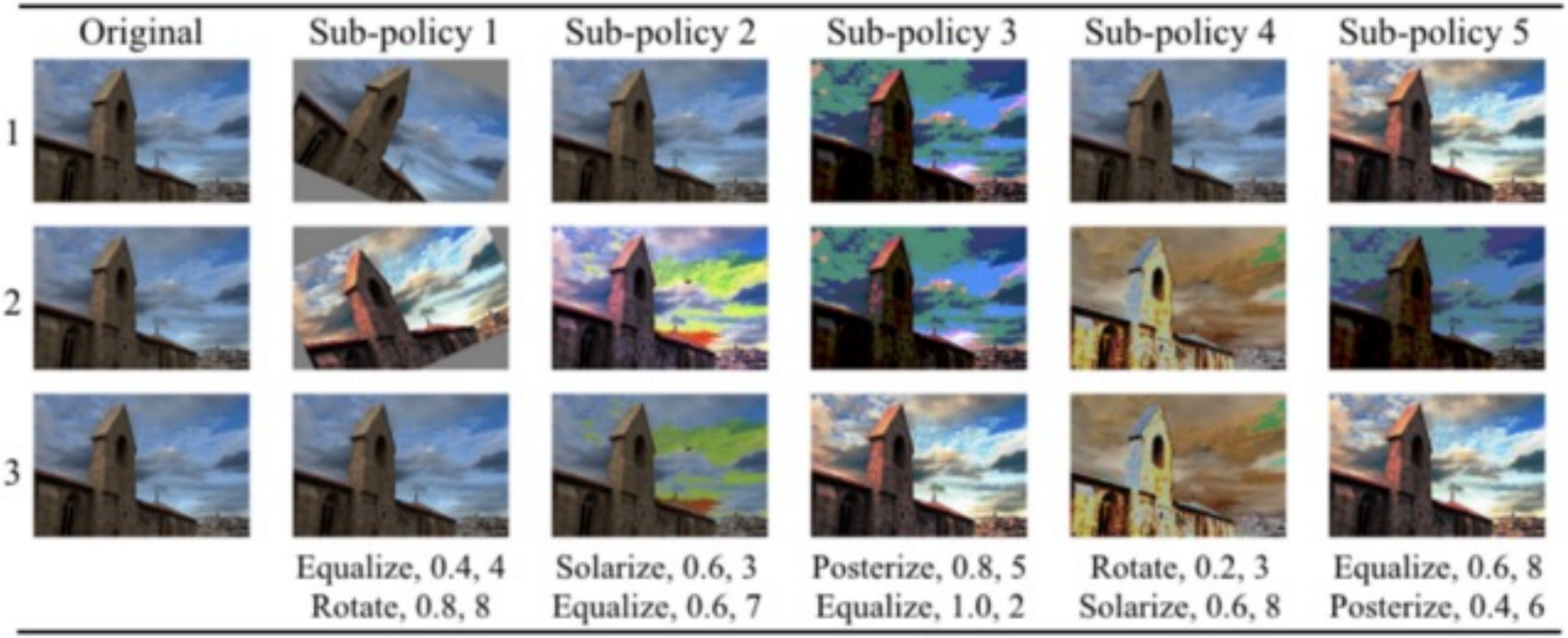


Photometric Distortion คือ การเพิ่มข้อมูลที่จะเปลี่ยนแปลงค่าพิกเซล อาทิเช่น contrast, sharpness, blurring, brightness และ color changes เป็นต้น

Geometric Distortion คือ การเพิ่มข้อมูลที่จะเปลี่ยนแปลงรูปร่าง อาทิเช่น cropping, rotation, scaling และ translation เป็นต้น

https://medium.com/lunit/photometric-data-augmentation-in-projection-radiography-bed3ae9f55c3

Distortion Data Augmentation

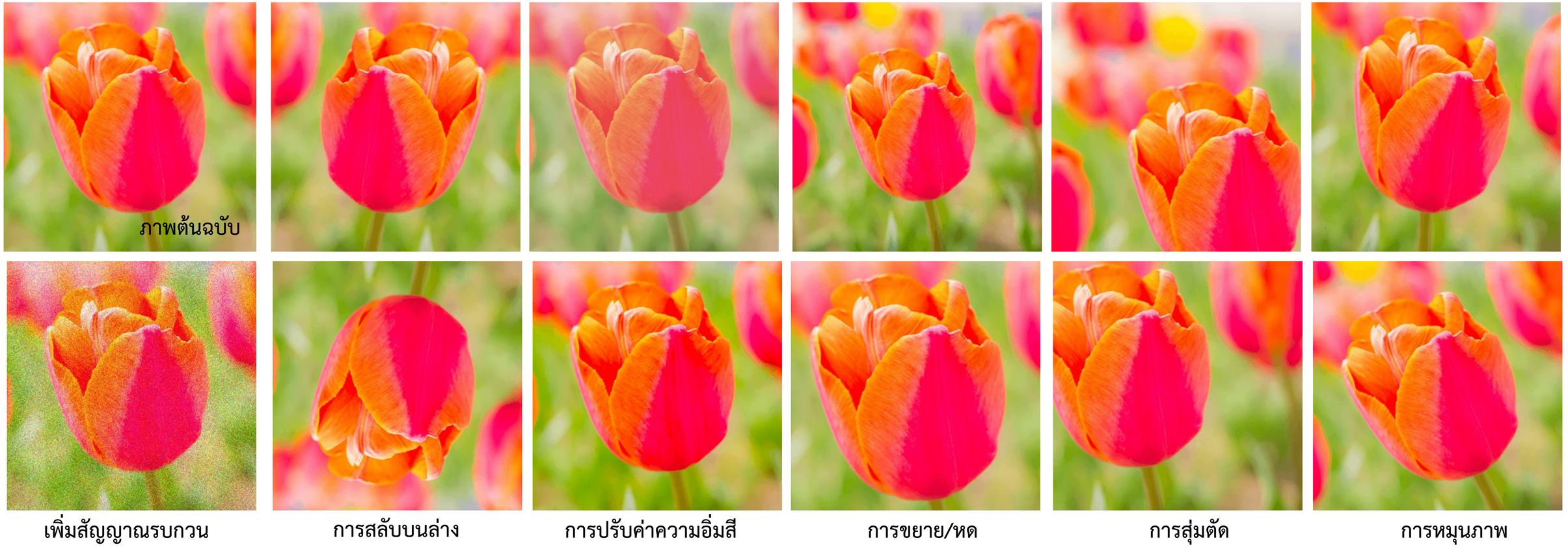


Photometric Distortion คือ การเพิมข้อมูลทีจะเปลียนแปลงค่าพิกเซล อาทิเช่น contrast, sharpness, blurring, brightness และ color changes เป็นต้น

Geometric Distortion คือ การเพิ่มข้อมูลที่จะเปลี่ยนแปลงรูปร่าง อาทิเช่น cropping, rotation, scaling และ translation เป็นต้น

https://medium.com/lunit/photometric-data-augmentation-in-projection-radiography-bed3ae9f55c3

Distortion Data Augmentation



https://www.bualabs.com/archives/1170/what-is-data-augmentation-benefits-data-augmentaion-deep-learning-training-model-deep-neural-network-regularization-ep-1/

Image Occlusion



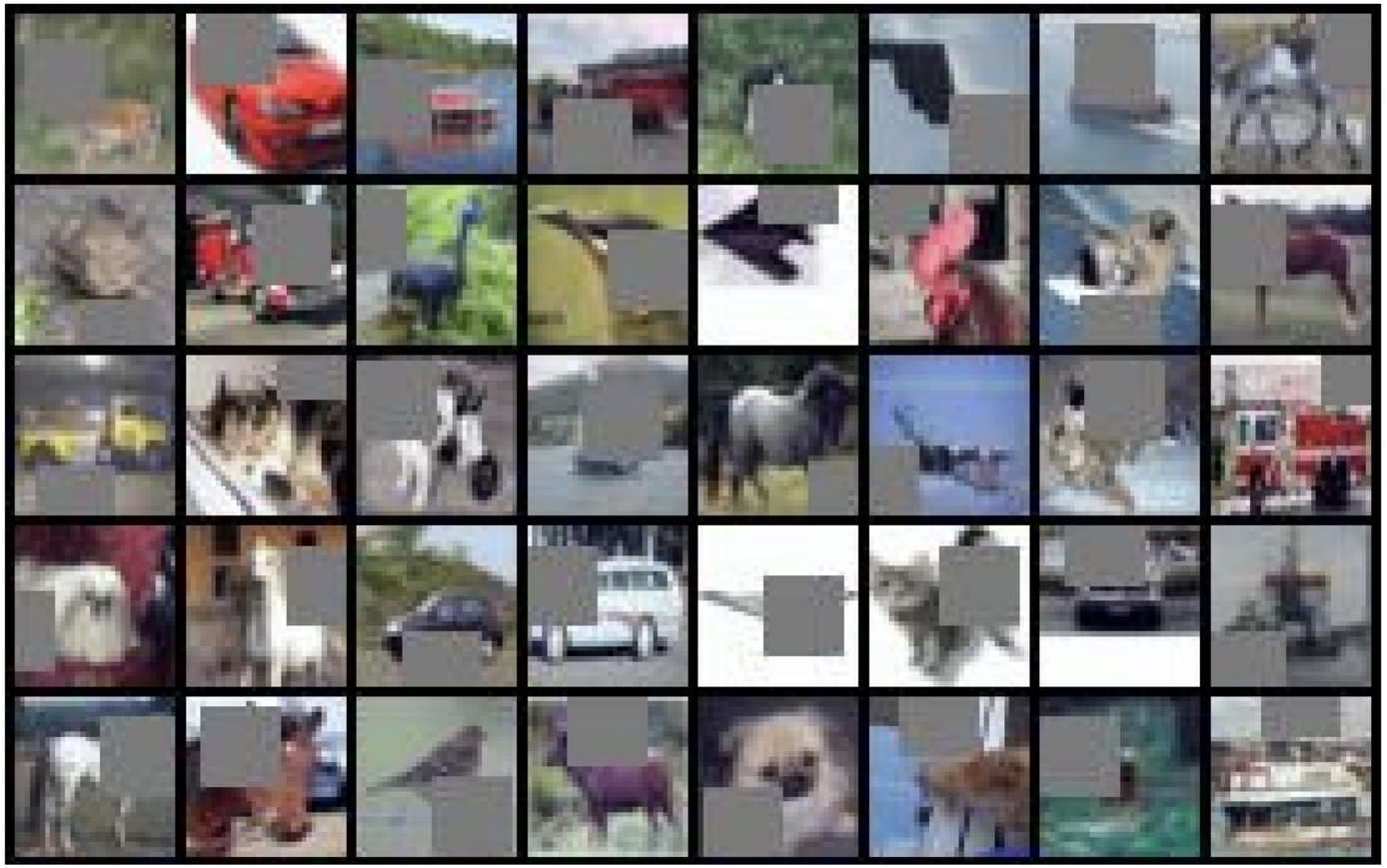
ลบแบบสุ่ม (Random Erase)

คือ วิธีการเพิ่มข้อมูลที่จะแทนที่ข้อมูลบริเวณของภาพด้วยค่าสุ่ม (random) หรือ ค่าพิกเซลเฉลี่ย (Mean) ของชุดการฝึก

ซึ่งขอบเขตและสัดสวนของบริเวณที่จะถูกแทนที่จะใช้หลากหลายขนาดและหลากหลายอัตราส่วน วิธีการนี้กลายเป็นวิธีการมาตรฐานที่สามารถป้องกันการเกิดปัญหา Overfitting ในขั้นตอนการฝึกสอนกับแบบจำลอง

https://blog.roboflow.com/yolov4-data-augmentation/

Image Occlusion

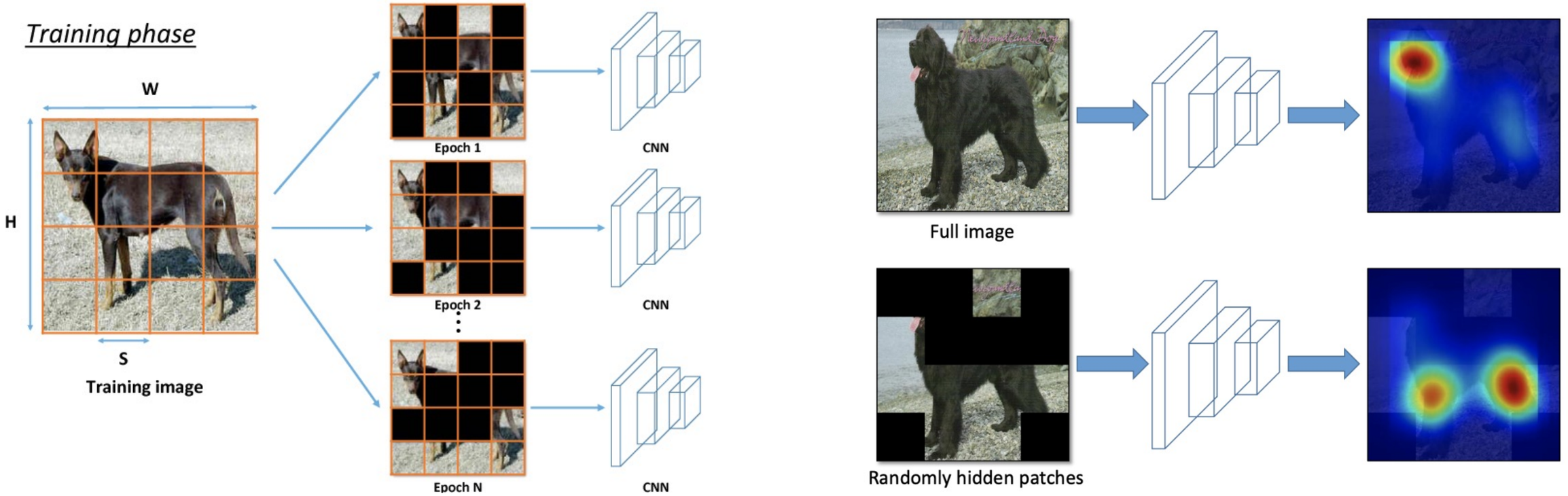


ตัดทิ้ง (Cutout)

คือ วิธีการเพิ่มข้อมูลที่จะตัดพื้นที่ของรูปทิ้งไปเป็นขนาดคงที่ในลักษณะสี่เหลี่ยมจัตุรัสตลอดช่วงฝึกสอนแบบจำลอง ซึ่งมีลักษณะคล้ายกับการลบแบบสุ่ม มีวัตถุประสงค์มีป้องกันการเกิดปัญหา Overfitting ในขั้นตอนการฝึกสอนกับแบบจำลอง

https://blog.roboflow.com/yolov4-data-augmentation/

Image Occlusion



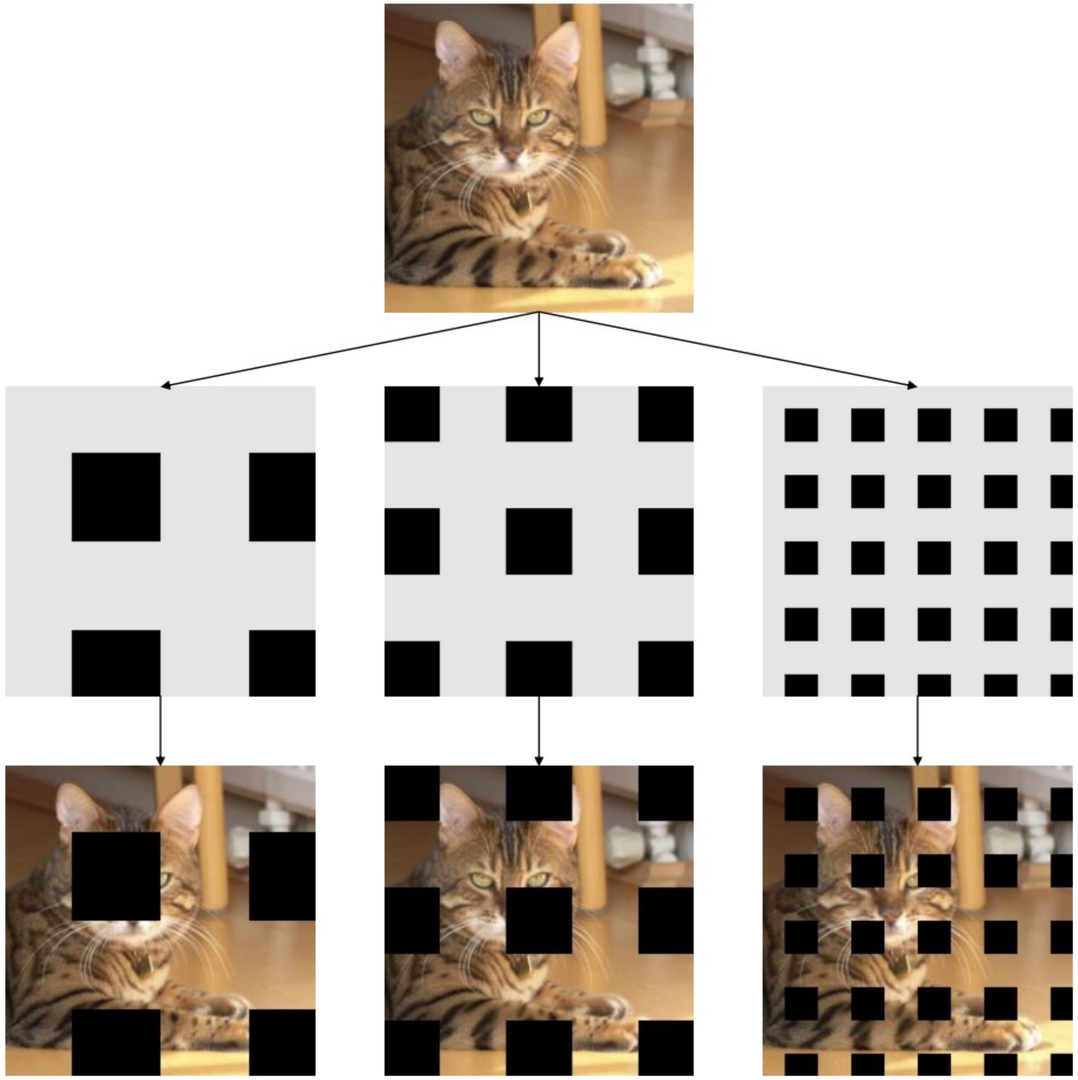
อ้างอิง Singh, K., Yu, H., Sarmasi, A., Pradeep, G., & Lee, Y.J. (2018). Hide-and-Seek: A Data Augmentation Technique for Weakly-Supervised Localization and Beyond. *ArXiv, abs/1811.02545*.

ซ่อนหา (Hide and Seek)

คือ วิธีการเพิ่มข้อมูลที่จะแบ่งภาพออกเป็นส่วนย่อย (Image Patch) จำนวน NXN และเลือกที่จะซ่อนแต่ละ Image Patch ตามค่าความน่าจะเป็นของแต่ละส่วน ทำให้แบบจำลองสามารถเรียนรู้ได้ว่าวัตถุมีลักษณะอย่างไร โดยไม่ต้องเรียนรู้เฉพาะส่วนเดียวของวัตถุ

อ้างอิง https://blog.roboflow.com/yolov4-data-augmentation/

Image Occlusion

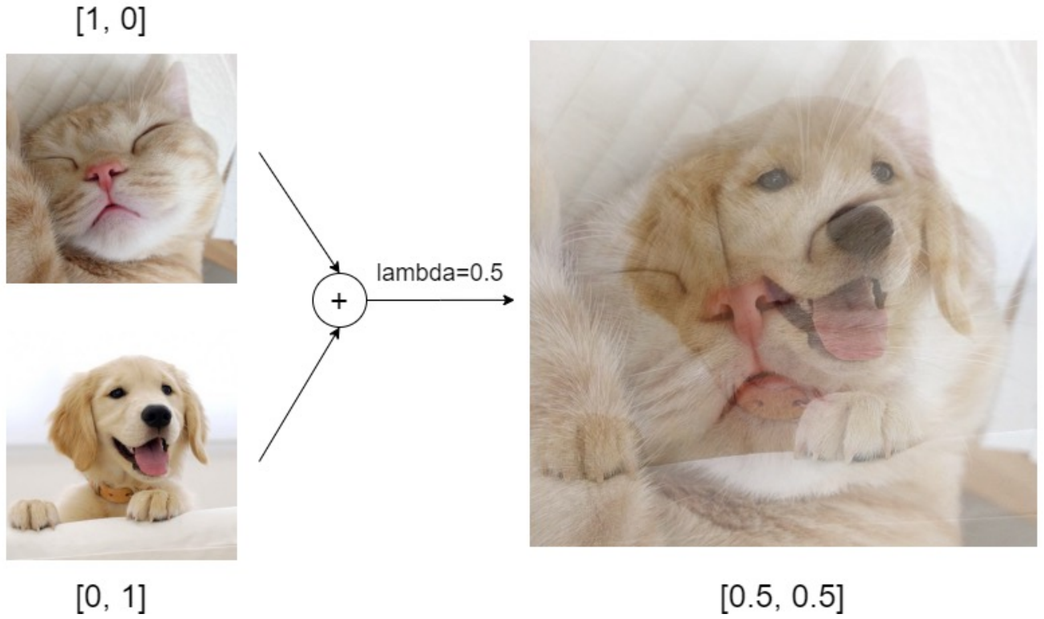


หน้ากากแบบกริด (Grid Mask)

คือ วิธีการเพิ่มข้อมูลที่จะซ่อนบางส่วนของภาพในลักษณะกริด คล้ายกับวิธีการซ่อนหา วิธีการนี้บังคับให้แบบจำลองพยายามเรียนรู้ส่วนต่าง ๆ ของวัตถุภายในภาพจากสิ่งที่ประกอบขึ้นเป็นวัตถุแต่ละชิ้น

อ้างอิง https://blog.roboflow.com/yolov4-data-augmentation/

Image Occlusion



อ้างอิง https://medium.com/@wolframalphav1.0/easy-way-to-improve-image-classifier-performance-part-1-mixup-augmentation-with-codes-33288db92de5

การผสมผสาน (Mixup)

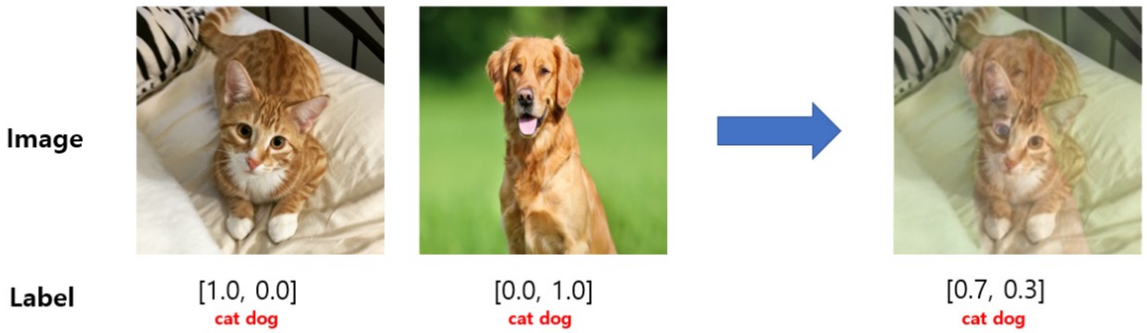
คือ การซ้อนทับหรือผสมผสานสองภาพเข้าด้วยกัน แต่เดิมแล้ววิธีการผสมผสานได้รับการออกแบบมาสำหรับงานทางด้านการจำแนกประเภท (Classification) แต่ก็ยังสามารถใช้งานได้ดีสำหรับงานตรวจจับวัตถุ (Detection) ซึ่งสามารถคำนวณได้จากสมการต่อไปนี้

$$\begin{aligned} \hat{x} &= \lambda x_i + (1-\lambda)x_j, \\ \hat{y} &= \lambda y_i + (1-\lambda)y_j, \end{aligned}$$

where $\lambda \in [0,1]$ is a random number

where x_i, x_j are raw input vectors

where y_i, y_j are one-hot label encodings

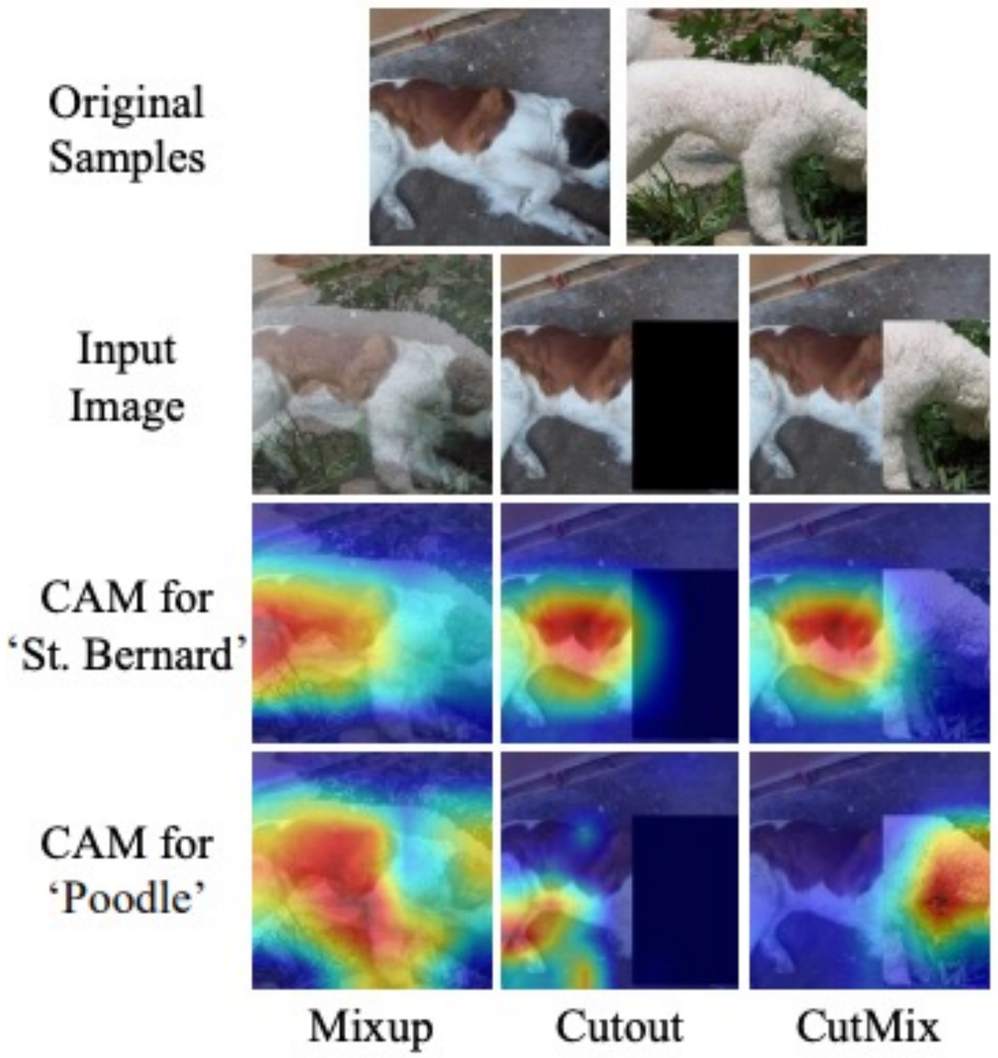


อ้างอิง https://www.kaggle.com/code/kaushal2896/data-augmentation-tutorial-basic-cutout-mixup

จากผลการทดลองพบว่าค่า α ที่เหมาะสม ได้แก่ 0.2, 0.3 และ 0.4

อ้างอิง https://medium.com/mlearning-ai/yolox-explanation-mosaic-and-mixup-for-data-augmentation-3839465a3adf

Image Occlusion



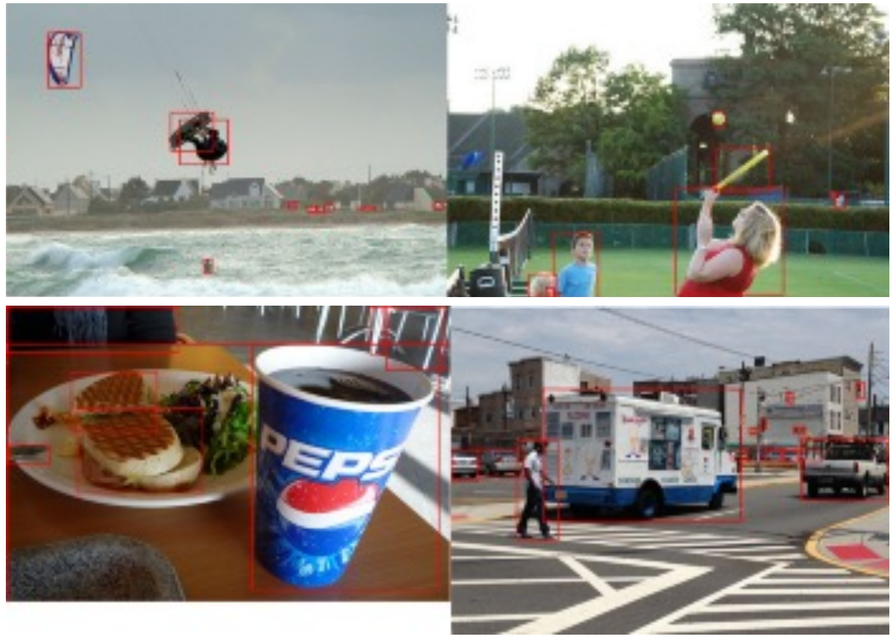
ตัดผสม (CutMix)

คือ เทคนิคที่ดำเนินการตัดส่วนของภาพฝึกสอนและนำไปวางบนภาพการฝึกสอนอื่น ๆ แบบสุ่ม เพื่อหลีกเลี่ยงการสูญเสียข้อมูลในชุดข้อมูลฝึกสอนและเป็นการเพิ่มประสิทธิภาพของแบบจำลองที่เหมาะสมกว่าแนวทางการสุ่มตัดข้อมูลบางส่วนทิ้ง ซึ่งวิธีการตัดผสมสามารถเพิ่มความสามารถของแบบจำลองในงานทางด้านตรวจจับวัตถุได้

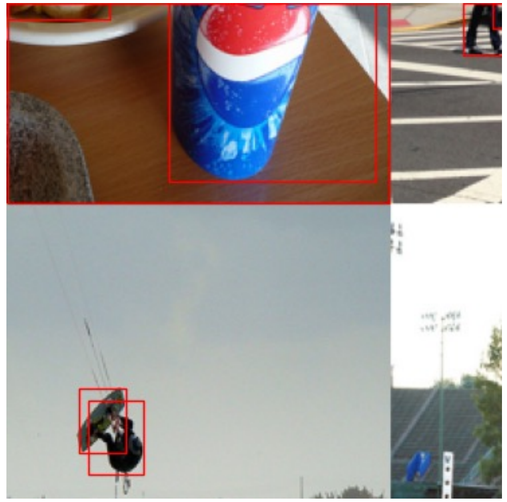
อ้างอิง https://sarthakforwet.medium.com/cutmix-a-new-strategy-for-data-augmentation-bbc1c3d29aab

Data Augmentation

Mosaic Data Augmentation เป็นวิธีการเพิ่มจำนวนรูปภาพฝึกสอนที่ได้นำมาใช้งานครั้งแรกใน YOLO เวอร์ชั่น 4 โดยอาศัยหลักการ “รวมภาพหลายภายให้เป็นภาพเพียงภายเดียว” โดยการปรับขนาดภาพทั้ง 4 ภาพ จากนั้น จึงนำมาต่อเข้าด้วยกัน แล้วสุ่มตัดภาพที่เย็บออกเพื่อให้ได้ภาพในลักษณะโมเสก ซึ่งสามารถสรุปขั้นตอนได้ดังนี้



Four Input Images



Final Mosaic image

https://medium.com/mlearning-ai/yolox-explanation-mosaic-and-mixup-for-data-augmentation-3839465a3adf

Data Augmentation

ขั้นที่ 1 ปรับขนาดภาพให้เป็นขนาด 256 × 256 พิกเซล

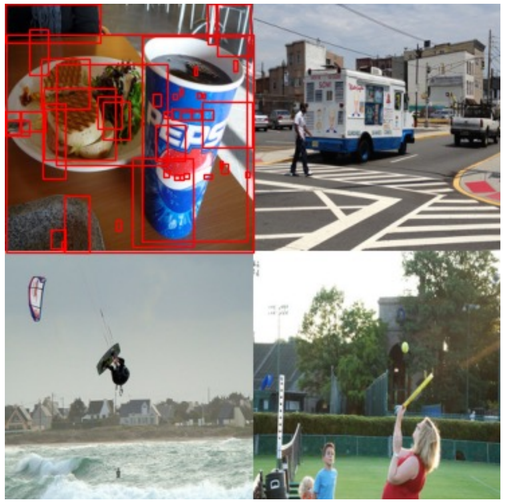


อ้างอิง https://medium.com/mlearning-ai/yolox-explanation-mosaic-and-mixup-for-data-augmentation-3839465a3adf

Data Augmentation

ขั้นที่ 2 ต่อภาพทั้งหมดเข้าด้วยกันให้เป็นภาพเดียวขนาด 512 × 512 พิกเซล

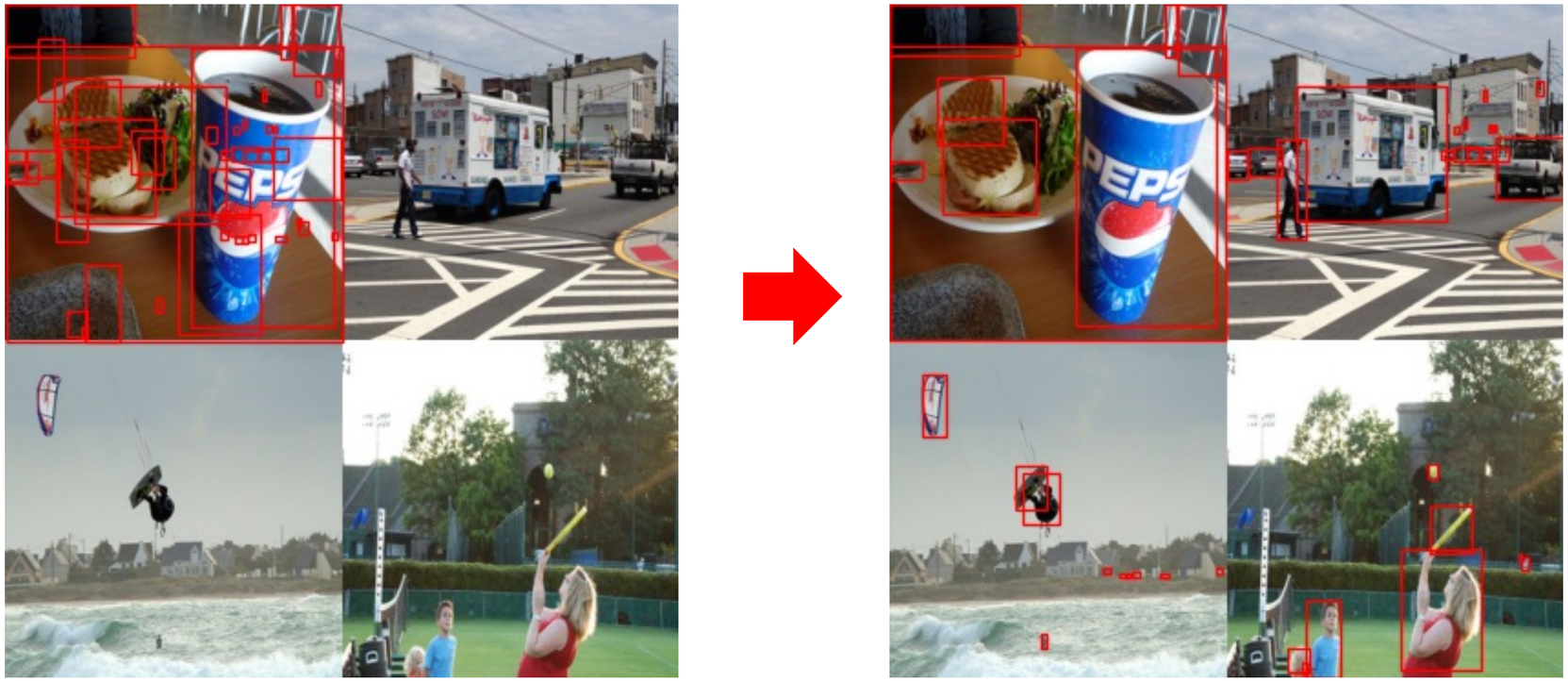




อ้างอิง https://medium.com/mlearning-ai/yolox-explanation-mosaic-and-mixup-for-data-augmentation-3839465a3adf

Data Augmentation

ขั้นที่ 3 ขยับขอบเขตของบริเวณที่น่าสนใจ (Bounding Box)

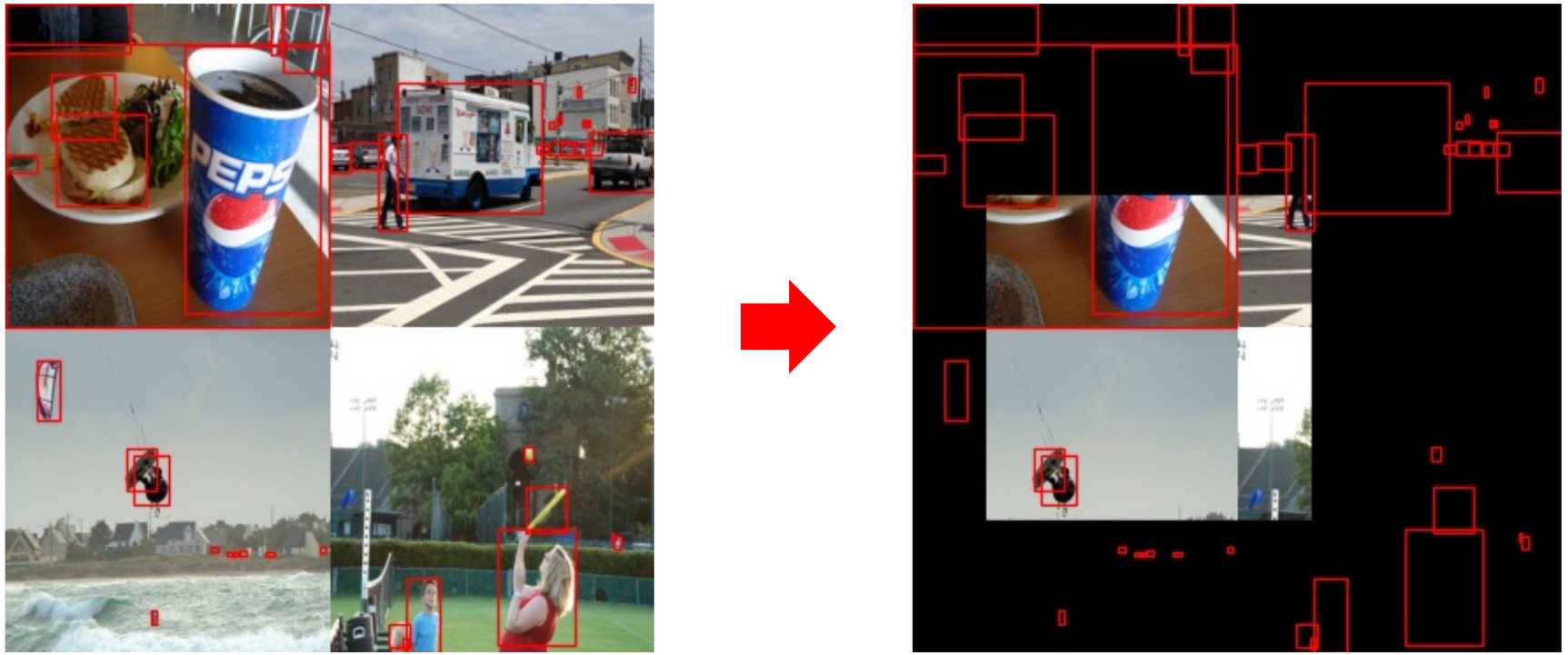


- Bounding Box ของภาพบริเวณซ้ายบน ☛ ไม่ต้องดำเนินการขยับ
- Bounding Box ของภาพบริเวณขวาบน ☛ ให้ขยับไปทางขวา 256 พิกเซล (x_new = x_old +256)
- Bounding Box ของภาพบริเวณซ้ายล่าง ☛ ให้ขยับลงด้านล่าง 256 พิกเซล (y_new = y_old +256)
- Bounding Box ของภาพบริเวณขวาล่าง ☛ ให้ขยับไปทางขวา 256 พิกเซล และลงด้านล่าง 256 พิกเซล (x_new = x_old +256 และ y_new = y_old +256)

อ้างอิง https://medium.com/mlearning-ai/yolox-explanation-mosaic-and-mixup-for-data-augmentation-3839465a3adf

Data Augmentation

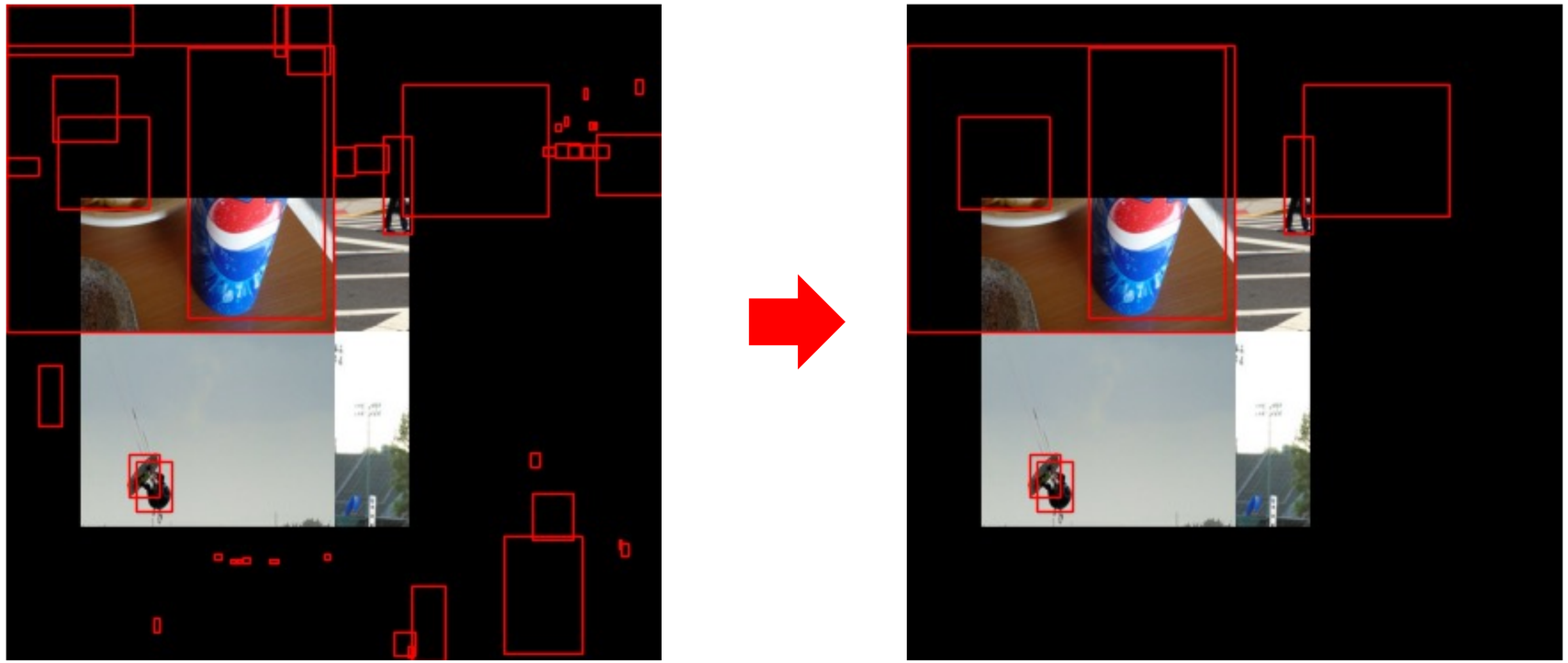
ขั้นที่ 4 สุ่มตัดภาพแบบจัตุรัสให้มีขนาดอยู่ในช่วง 16 ถึง 240 พิกเซลและต้องติดภาพทั้ง 4 ภาพ



อ้างอิง https://medium.com/mlearning-ai/yolox-explanation-mosaic-and-mixup-for-data-augmentation-3839465a3adf

Data Augmentation

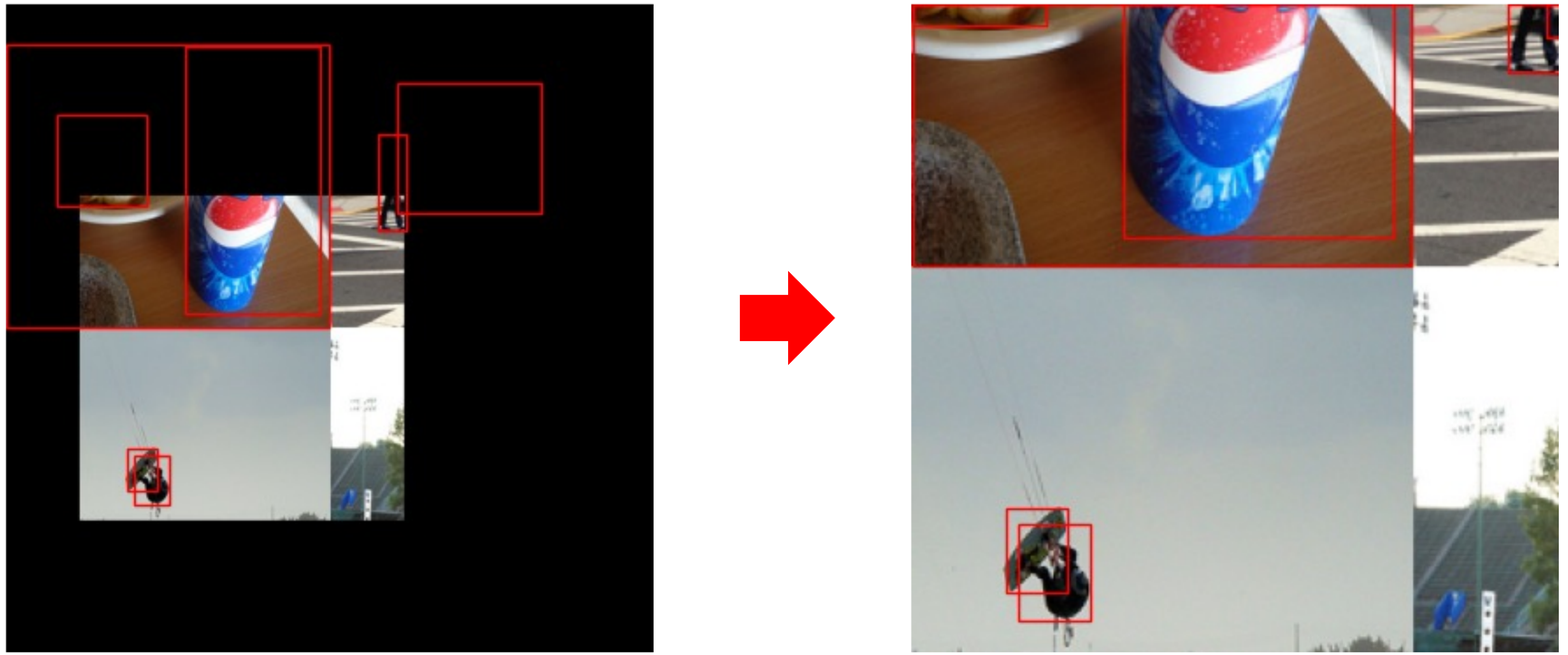
ขั้นที่ 5 ลบขอบเขตของบริเวณที่น่าสนใจที่ถูกสุ่มตัดออก



อ้างอิง https://medium.com/mlearning-ai/yolox-explanation-mosaic-and-mixup-for-data-augmentation-3839465a3adf

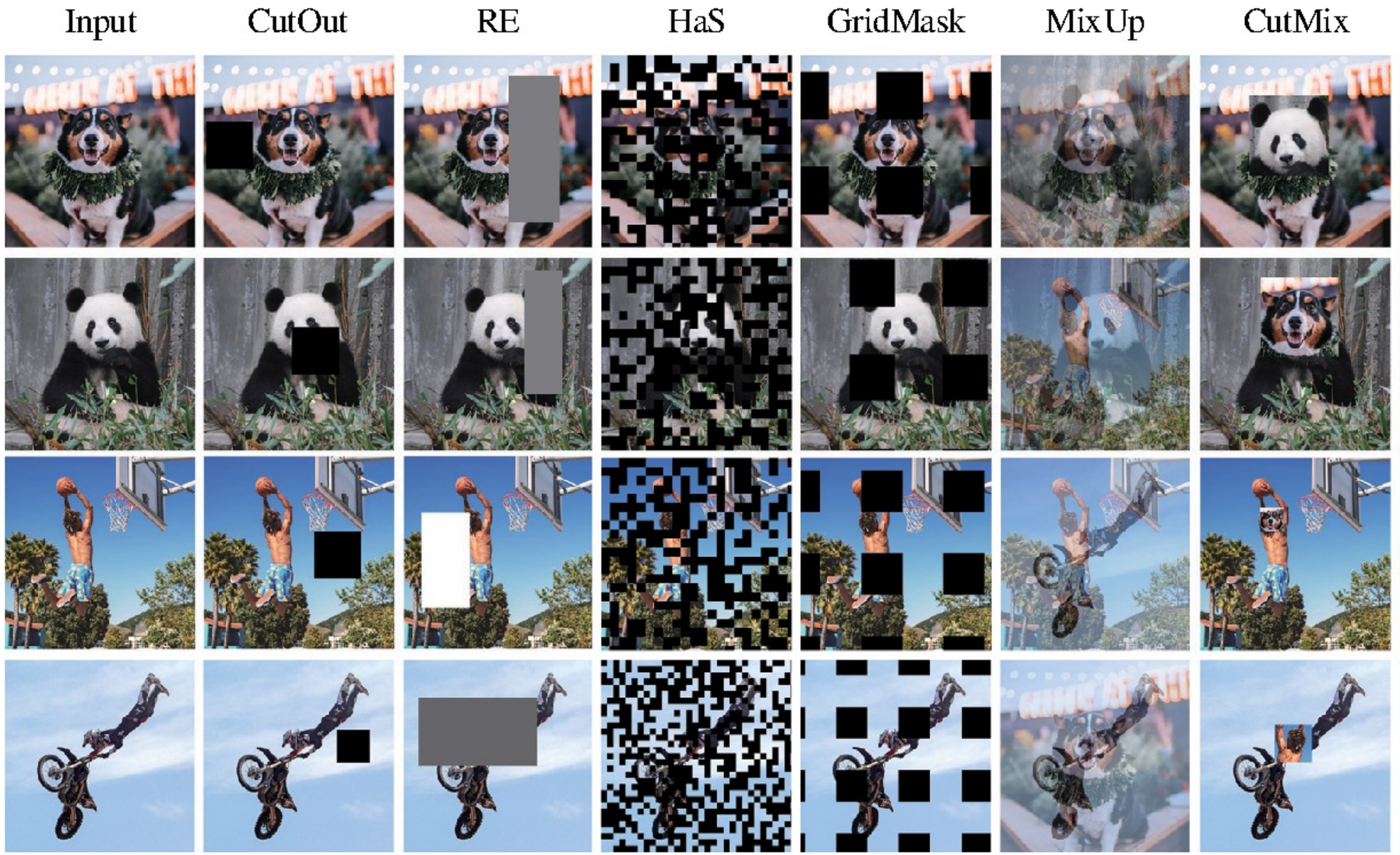
Data Augmentation

ขั้นที่ 6 ปรับภาพให้เหลือแต่บริเวณที่ไม่ถูกสุ่มตัดออกและปรับภาพผลลัพธ์ให้อยู่ในขนาดที่เหมาะสมกับแบบจำลอง



อ้างอิง https://medium.com/mlearning-ai/yolox-explanation-mosaic-and-mixup-for-data-augmentation-3839465a3adf

Data Augmentation



อ้างอิง Li, Chenghao, et al. "In-and-Out: a data augmentation technique for computer vision tasks." Journal of Electronic Imaging 31.1 (2022): 013023-013023.

หลักการ Dynamic Augmentation Per Epoch

การปรับเปลี่ยนวิธีการทำ Data Augmentation สำหรับแต่ละ Epoch คือ การเปลี่ยนหรือปรับพฤติกรรมของการแปลงภาพ (augmentation) ในแต่ละรอบการฝึก (epoch) แทนที่จะใช้แบบเดียวกันตลอดการฝึกโมเดล

แบบดั้งเดิม (Static)	แบบปรับทุก epoch (Dynamic)
• ใช้ policy augmentation เดิมตลอดการฝึก	• ปรับ intensity / type ของ augmentation ตาม epoch
• เหมาะกับงานทั่วไป	• เหมาะกับการฝึกแบบ curriculum / robust learning
• ไม่รู้สถานะของโมเดล	• ตอบสนองต่อ loss, accuracy หรือ sample difficulty ได้

การปรับ Data Augmentation เริ่มจาก Weak Augmentation เพื่อให้โมเดลเรียนรู้พื้นฐานได้ง่าย จากนั้น ค่อยๆ เพิ่มความซับซ้อนด้วย Medium หรือ Strong Augmentation เพื่อเสริม Robustness และลด Overfitting ปรับตามสถานะของโมเดล

ปัญหาของ Traditional Data Augmentation

- **การใช้ Data Augmentation เดียวกันสำหรับทุก epoch**
  - Static Configuration คือ กำหนด augmentation policy ตั้งแต่เริ่มต้นแล้วใช้ตลอดทั้งกระบวนการเทรน
  - One-Size-Fits-All คือ ใช้ parameter เดียวกัน เช่น rotation 30°, brightness ±0.3 ตั้งแต่ epoch 1 จนจบ
  - ไม่มีการปรับตัว คือ ไม่ว่าโมเดลจะเรียนรู้ไปแค่ไหนแล้ว ยังคงได้รับข้อมูลที่มีความซับซ้อนเท่าเดิม
- **ไม่สนใจว่าโมเดลมีความพร้อมแค่ไหน หรือการละเลยสถานะของโมเดล**
  - ไม่พิจารณา Learning Progress คือ ไม่ดูว่าโมเดลเรียนรู้ basic features แล้วหรือยัง
  - ไม่คำนึงถึง Model Capacity คือ ไม่สนใจว่าโมเดลมีความสามารถรับมือกับความซับซ้อนแค่ไหน
  - ขาดการ Monitor คือ ไม่ติดตามว่า augmentation ปัจจุบันเหมาะสมกับระดับการเรียนรู้หรือไม่

ปัญหาของ Traditional Data Augmentation

- **โมเดลใหม่ยังไม่แข็งแกร่ง แต่ได้รับข้อมูลที่ผิดเพี้ยนมาก มักเกิดปัญหาในช่วงต้นการเทรน (Early Training)**
  - Overwhelming Complexity คือ โมเดลใน epoch แรกๆ ยังไม่รู้จัก basic pattern เลยการให้ข้อมูลที่ rotate 30°, มี noise สูง, หรือ cutout ทำให้สับสน เหมือนการสอนเด็กอนุบาลด้วยตำราระดับมหาวิทยาลัย
  - Gradient Instability คือ การ augment มากเกินไปทำให้ gradient กระเด็นไปมา Loss function ไม่เสถียร เพราะข้อมูลแต่ละ batch แตกต่างกันมากการ converge ช้าลงหรืออาจไม่ converge เลย
  - Feature Learning Disruption คือ การเรียนรู้ basic features (edges, shapes, colors) ถูกรบกวน โดยโมเดลพยายามจำข้อมูลที่ผิดเพี้ยนแทนที่จะเรียนรู้ pattern จริง
- **โมเดลแก่แล้วยังได้รับข้อมูลง่ายๆ มักเกิดปัญหาในช่วงหลังการเทรน (Late Training)**
  - Insufficient Challenge คือ โมเดลใน epoch 80-100 มีความสามารถสูงแล้ว การให้ข้อมูลที่ไม่ท้าทายทำให้เกิด overfitting เหมือนการให้นักเรียนมหาวิทยาลัยทำแบบฝึกหัดอนุบาล
  - Missed Generalization Opportunity คือ โมเดลที่แข็งแกร่งแล้ว ควรได้รับข้อมูลที่หลากหลายมากขึ้น การใช้ augmentation เบาๆ ทำให้พลาดโอกาสในการเรียนรู้ robust features ส่งผลทำให้ Generalization performance ต่ำกว่าที่ควรจะเป็น
  - Resource Wastage คือ ใช้เวลาและ computational resources ไปกับข้อมูลที่ไม่ได้ช่วยปรับปรุงโมเดล ส่งผลให้การเทรนใช้เวลานานกว่าที่จำเป็น

ผลกระทบของ Traditional Data Augmentation

- **ด้านประสิทธิภาพ**
  - Slower Convergence: การ converge ช้าลง
  - Lower Final Accuracy: ประสิทธิภาพสุดท้ายต่ำกว่าที่ควรจะเป็น
  - Poor Generalization: ทำงานกับข้อมูลใหม่ได้ไม่ดี

- **ด้านทรัพยากร**
  - Longer Training Time: ใช้เวลาเทรนนานขึ้น
  - Computational Waste: ใช้ GPU/CPU ไม่มีประสิทธิภาพ
  - Higher Cost: ค่าใช้จ่ายในการเทรนสูงขึ้น

- **ด้านความเสถียร**
  - Unstable Training: การเทรนไม่เสถียร
  - Inconsistent Results: ผลลัพธ์ไม่สม่ำเสมอ
  - Need Multiple Runs: ต้องเทรนหลายรอบเพื่อให้ได้ผลที่ดี

สรุป การทำ Data Augmentation แบบดังเดิม หรือ (Fixed Policy) เป็นแนวทางที่ได้เริ่มได้รับความนิยมน้อยลง การปรับ Data Augmentation ให้เข้ากับระดับการเรียนรู้ของแบบจำลองจะช่วยแก้ปัญหาข้างต้น และทำให้ได้รับแบบจำลองที่มีประสิทธิภาพ

Weak To Strong Augmentation

ระดับ	ตัวอย่าง Augmentation	รายละเอียด
เบามาก (Very Mild)	• Flip (horizontal/vertical) • Brightness/contrast (เล็กน้อย)	ไม่เปลี่ยน semantics ของภาพ
เบา (Mild)	• Rotate ±5–10° • Translate ±5% • Color jitter ต่ำ • Gaussian noise (low std) • Random crop + resize	รักษารูปร่างหลักของภาพ
ปานกลาง (Moderate)	• Shear ±10–20° • Cutout เล็ก • AutoContrast / Equalize • Brightness/Contrast กลาง	แก้ geometry และ color โครงสร้างภาพ
แรง (Strong)	• Rotate ±30–45° • Color jitter รุนแรง • CutMix / MixUp / Random erasing ใหญ่ • Posterize / Solarize • Translate ±30–50%	ทำให้รูปเปลี่ยนมาก การจำแนกยากขึ้น
รุนแรงมาก (Very Strong)	• Rotate ±90°+ / Perspective warp • High std noise, strong blur, JPEG artefacts • Adversarial transforms	เบี่ยงเบนจากข้อมูลจริงมาก ใช้เฉพาะโมเดลที่มั่นใจว่าเรียนรู้แล้ว

การปรับ Data Augmentation แบบยืดหยุ่น

การใช้ Data Augmentation แบบ Dynamic หมายถึง การที่ augmentation ไม่ถูกกำหนดตายตัว แต่มีการเปลี่ยนแปลงในแต่ละ epoch หรือแม้กระทั่งในแต่ละ sample เพื่อเพิ่มประสิทธิภาพการเรียนรู้ของโมเดล

วิธี	ลักษณะการทำงาน	จุดเด่น
Progressive	เพิ่มความแรงของ augment ตามเวลา	คล้ายเรียนจากง่ายไปยาก
Adaptive Scheduling	ปรับตาม training loss / val metric	โมเดลขับเคลื่อนการปรับเอง
Sample-Aware	ปรับตาม class หรือ sample difficulty	เน้นความยุติธรรมและ customization

Progressive Augmentation

เป็นหลักการที่เชื่อว่าโมเดลในช่วงต้นของการเทรนยังไม่แข็งแกร่งพอที่จะรับมือกับ Strong Augmentation การเริ่มต้นด้วย Weak augmentation จะช่วยให้โมเดลเรียนรู้ pattern พื้นฐานก่อนได้ดีกว่า อาทิเช่น

- Epoch 1-20: เริ่มด้วย rotation ±5°, brightness ±0.1
- Epoch 21-50: เพิ่มเป็น rotation ±15°, brightness ±0.2, เพิ่ม gaussian noise
- Epoch 51+: ใช้ augmentation เต็มรูปแบบ เช่น rotation ±30°, cutout, mixup

ประโยชน์

- หลีกเลี่ยง underfitting ช่วงแรก ที่โมเดลยังไม่เข้าใจข้อมูล
- คล้ายแนวคิด curriculum learning ที่ให้เรียนจากง่ายไปยาก

อ้างอิง Ye, M., Shen, S., Zhang, T., Dai, P., & Fang, B. (2021). Efficient contrastive learning via novel data augmentation and curriculum learning. *Proceedings of EMNLP 2021*, 1832-1838.



ตัวอย่าง Progressive Augmentation

```
import torch
import torchvision
import torchvision.transforms as transforms
from torch import nn, optim
from torch.utils.data import DataLoader
import torchvision.datasets as datasets

# Define CNN
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 8 * 8, 100),
            nn.ReLU(),
            nn.Linear(100, 10)
        )

    def forward(self, x):
        return self.fc(self.conv(x))
```



ตัวอย่าง Progressive Augmentation

```
# Define Progressive Augmentation Scheduler
def get_transform(epoch):
    if epoch < 10:
        print("Progressive Augmentation >>> Case 1")
        return transforms.Compose([
            transforms.RandomHorizontalFlip(),
            transforms.ToTensor()
        ])
    elif epoch < 20:
        print("Progressive Augmentation >>> Case 2")
        return transforms.Compose([
            transforms.RandomHorizontalFlip(),
            transforms.ColorJitter(brightness=0.2, contrast=0.2),
            transforms.ToTensor()
        ])
    else:
        print("Progressive Augmentation >>> Case 3")
        return transforms.Compose([
            transforms.RandomHorizontalFlip(),
            transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3),
            transforms.RandomRotation(15),
            transforms.RandomErasing(p=0.2),
            transforms.ToTensor()
        ])
```



ตัวอย่าง Progressive Augmentation

```
# Parameter Set Up
model = SimpleCNN()

# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# Loss Function
criterion = nn.CrossEntropyLoss()

# Optimizer
optimizer = optim.Adam(model.parameters(), lr=0.001)

total_epochs = 30
```



ตัวอย่าง Progressive Augmentation

```
# Training Model
for epoch in range(total_epochs):
  # Progressive Transform
  transform = get_transform(epoch)

  # Dataset with Progressive Transform
  trainset = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
  trainloader = DataLoader(trainset, batch_size=64, shuffle=True)

  model.train()
  total_loss = 0

  for images, labels in trainloader:
    images, labels = images.to(device), labels.to(device)

    outputs = model(images)
    loss = criterion(outputs, labels)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    total_loss += loss.item()

  # แสดงค่า learning rate และ loss
  current_lr = optimizer.param_groups[0]['lr']
  print(f"Epoch {epoch+1}: LR = {current_lr:.6f}, Loss = {total_loss:.4f}")
```



ตัวอย่าง Progressive Augmentation

```
Progressive Augmentation >>> Case 1
Epoch 1: LR = 0.001000, Loss = 1246.3496
Progressive Augmentation >>> Case 1
Epoch 2: LR = 0.001000, Loss = 1015.6079
Progressive Augmentation >>> Case 1
Epoch 3: LR = 0.001000, Loss = 924.2589
Progressive Augmentation >>> Case 1
Epoch 4: LR = 0.001000, Loss = 849.6002
Progressive Augmentation >>> Case 1
Epoch 5: LR = 0.001000, Loss = 800.0217
Progressive Augmentation >>> Case 1
Epoch 6: LR = 0.001000, Loss = 756.4806
Progressive Augmentation >>> Case 1
Epoch 7: LR = 0.001000, Loss = 723.6660
Progressive Augmentation >>> Case 1
Epoch 8: LR = 0.001000, Loss = 696.9016
Progressive Augmentation >>> Case 1
Epoch 9: LR = 0.001000, Loss = 673.8261
Progressive Augmentation >>> Case 1
Epoch 10: LR = 0.001000, Loss = 652.9446
Progressive Augmentation >>> Case 2
Epoch 11: LR = 0.001000, Loss = 651.8515
Progressive Augmentation >>> Case 2
Epoch 12: LR = 0.001000, Loss = 637.8007
Progressive Augmentation >>> Case 2
Epoch 13: LR = 0.001000, Loss = 621.3867
Progressive Augmentation >>> Case 2
Epoch 14: LR = 0.001000, Loss = 606.7203
Progressive Augmentation >>> Case 2
Epoch 15: LR = 0.001000, Loss = 592.3356
Progressive Augmentation >>> Case 2
Epoch 16: LR = 0.001000, Loss = 582.4802
Progressive Augmentation >>> Case 2
Epoch 17: LR = 0.001000, Loss = 568.6250
Progressive Augmentation >>> Case 2
Epoch 18: LR = 0.001000, Loss = 564.1299
Progressive Augmentation >>> Case 2
Epoch 19: LR = 0.001000, Loss = 551.9912
Progressive Augmentation >>> Case 2
Epoch 20: LR = 0.001000, Loss = 543.5560
```

Adaptive Scheduling

จะติดตาม training metrics (อาทิเช่น Loss หรือ Error) แล้วปรับ augmentation policy ให้เหมาะสม ตัวอย่างเช่น

- ถ้า validation loss ยังสูง ⟶ คงความเบาของ augmentation
- แต่ถ้า loss เริ่มลด ⟶ เพิ่มความรุนแรงเพื่อหลีกเลี่ยง overfitting

ซึ่งอาจจะสังเกตได้ว่าโมเดลอาจ "สบายใจ" เกินไป ได้ดังนี้

- เมื่อ validation loss ไม่ลดลงใน 5 epochs ติดต่อกัน
- เมื่อ accuracy ไม่เพิ่มขึ้นเกิน threshold ที่กำหนด
- เมื่อ gradient มีขนาดเล็กเกินไป

ประโยชน์

- ให้ augmentation สอดคล้องกับ learning stage ของโมเดล
- ช่วยลดความเสี่ยงของ over/under augmentation

อ้างอิง Hou, J., Li, M., Wang, Y., & Chen, X. (2023). When to learn what: Model-adaptive data augmentation curriculum. Proceedings of ICCV 2023. และ Li, X., et al. (2022). Universal adaptive data augmentation. arXiv preprint arXiv:2207.06658.



ตัวอย่าง Adaptive Scheduling

```
import torch
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
import torchvision.datasets as datasets
from torch import nn, optim

# CNN โมเดลพื้นฐาน
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 8 * 8, 100),
            nn.ReLU(),
            nn.Linear(100, 10)
        )
    def forward(self, x):
        return self.fc(self.conv(x))
```



ตัวอย่าง Adaptive Scheduling

```
# ฟังก์ชันสร้าง multi-augmentation ตามค่า magnitude
def build_transform(mag):
    return transforms.Compose([
        transforms.RandomHorizontalFlip(),
        transforms.ColorJitter(
            brightness=mag,
            contrast=mag,
            saturation=mag
        ),
        transforms.RandomRotation(degrees=mag * 45),  # ปรับตาม mag
        transforms.RandomAffine(degrees=0, translate=(mag * 0.2, mag * 0.2)),
        transforms.ToTensor(),
        transforms.RandomErasing(p=mag * 0.5)
    ])
```



ตัวอย่าง Adaptive Scheduling

```
# Parameter Set Up
model = SimpleCNN()

# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# Loss Function
criterion = nn.CrossEntropyLoss()

# Optimizer
optimizer = optim.Adam(model.parameters(), lr=0.001)

total_epochs = 30

# ค่าเริ่มต้นของความแรง
magnitude = 0.1
last_val_loss = None
```



ตัวอย่าง Adaptive Scheduling

```
# Training Model
for epoch in range(total_epochs):

  # สร้าง transform ใหม่ตาม magnitude
  transform = build_transform(magnitude)

  # Dataset with Adaptive Transform
  trainset = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
  trainloader = DataLoader(trainset, batch_size=64, shuffle=True)

  model.train()
  total_loss = 0

  for images, labels in trainloader:
    images, labels = images.to(device), labels.to(device)

    outputs = model(images)
    loss = criterion(outputs, labels)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    total_loss += loss.item()


  # แสดงค่า learning rate และ loss
  current_lr = optimizer.param_groups[0]['lr']
  print(f"Epoch {epoch+1}: LR = {current_lr:.6f}, Loss = {total_loss:.4f}")
```

```
for epoch in range(total_epochs):
  #... สไลน์ก่อนหน้า
  # Dataset for Validation Only
  valset = datasets.CIFAR10(root='./data', train=False, download=True, transform=transforms.ToTensor())
  valloader = DataLoader(valset, batch_size=64, shuffle=False)
  val_loss = 0

 # ประเมินบน validation set
  model.eval()
  val_loss = 0
  with torch.no_grad():
    for images, labels in valloader:
      images, labels = images.to(device), labels.to(device)
      outputs = model(images)
      loss = criterion(outputs, labels)
      val_loss += loss.item()
    val_loss /= len(valloader)

  # ปรับ magnitude ตาม trend ของ validation loss
  if last_val_loss is not None:
    if val_loss > last_val_loss:  # โมเดลแย่ลง → ลดความแรง
      magnitude = max(0.05, magnitude - 0.05)
    else:  # โมเดลดีขึ้น → เพิ่มความแรง
      magnitude = min(0.6, magnitude + 0.05)
  last_val_loss = val_loss
```



ตัวอย่าง Adaptive Scheduling

```
Epoch 1: LR = 0.001000, Loss = 1285.9273
Epoch 2: LR = 0.001000, Loss = 1055.9353
Epoch 3: LR = 0.001000, Loss = 978.0699
Epoch 4: LR = 0.001000, Loss = 929.2464
Epoch 5: LR = 0.001000, Loss = 916.8001
Epoch 6: LR = 0.001000, Loss = 919.0911
Epoch 7: LR = 0.001000, Loss = 854.6223
Epoch 8: LR = 0.001000, Loss = 869.8755
Epoch 9: LR = 0.001000, Loss = 886.1702
Epoch 10: LR = 0.001000, Loss = 837.8419
Epoch 11: LR = 0.001000, Loss = 859.6033
Epoch 12: LR = 0.001000, Loss = 810.0184
Epoch 13: LR = 0.001000, Loss = 829.2921
Epoch 14: LR = 0.001000, Loss = 784.4774
Epoch 15: LR = 0.001000, Loss = 813.1688
Epoch 16: LR = 0.001000, Loss = 842.5635
Epoch 17: LR = 0.001000, Loss = 794.0153
Epoch 18: LR = 0.001000, Loss = 827.0069
```

Sample-Aware Augmentation

คือ การเลือกใช้เทคนิคการเพิ่มข้อมูล (augmentation) ให้เหมาะสมกับแต่ละตัวอย่างหรือแต่ละคลาส แทนที่จะใช้รูปแบบเดียวกันกับข้อมูลทั้งหมด กล่าวคือ เป็นการปรับระดับหรือประเภทของ augmentation ตามความยากของภาพ

❑ กลยุทธ์ย่อย Difficulty-Aware Per-sample

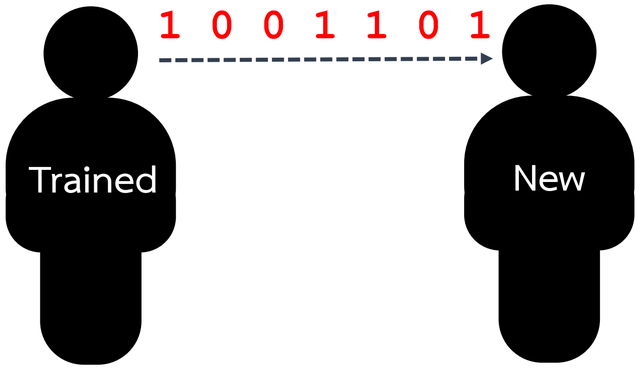
- ถ้า sample ที่โมเดลทำนายถูกต้องอย่างมั่นใจ (confidence > 0.9) ⟶ ให้ใช้ Strong Augmentation เพื่อเพิ่มความท้าทาย
- ถ้า sample ที่โมเดลทำนายผิดหรือไม่มั่นใจ (confidence < 0.6) ⟶ ให้ใช้ Weak Augmentation เพื่อไม่ให้ยากเกินไป
- ถ้า sample ที่อยู่ระหว่างกลาง ⟶ ให้ใช้ Medium Augmentation

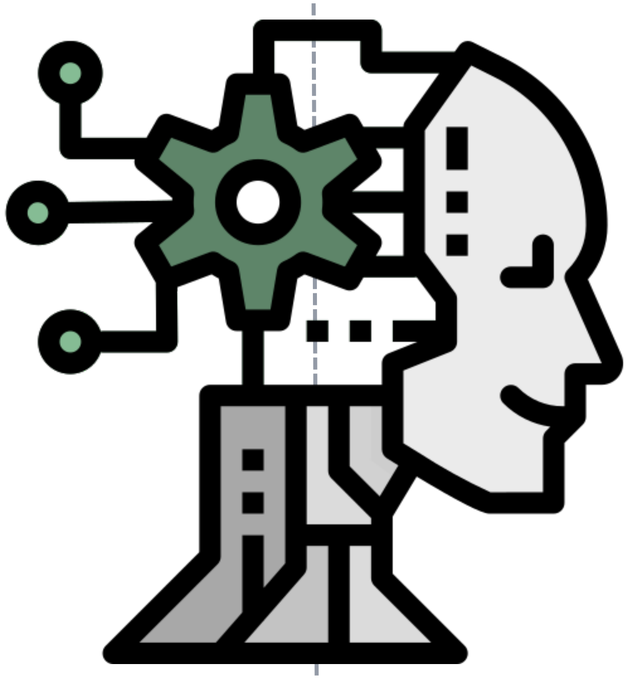
❑ กลยุทธ์ย่อย Class-Aware

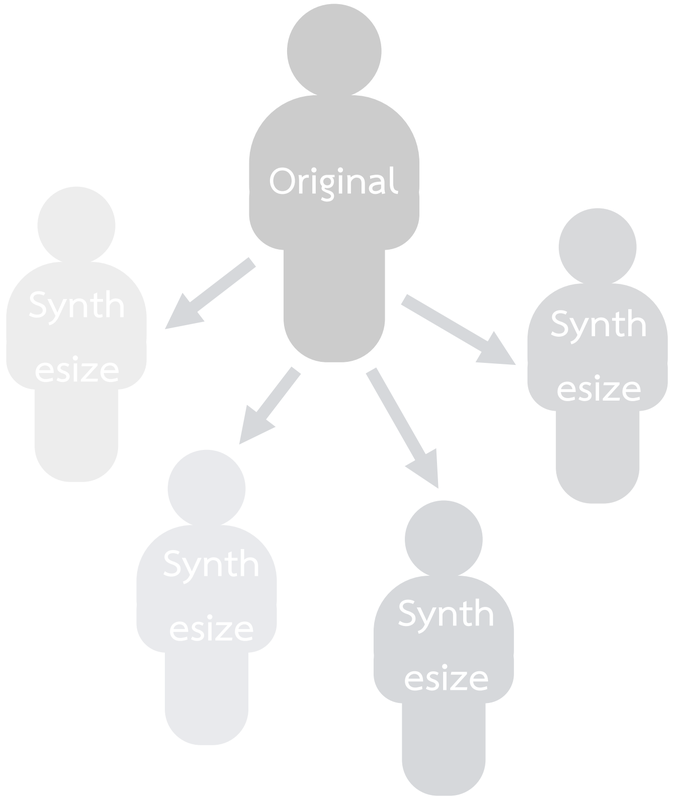
- ถ้าคลาสไหนมี accuracy ต่ำ ⟶ ให้ใช้ Strong Augmentation (เพิ่มข้อมูล)
- ถ้าคลาสไหนมี accuracy สูง ⟶ ให้ใช้ Weak Augmentation เพื่อหลีกเลี่ยง noise

อ้างอิง Ye, M., Shen, S., Zhang, T., Dai, P., & Fang, B. (2021). Efficient contrastive learning via novel data augmentation and curriculum learning. *Proceedings of EMNLP 2021*, 1832-1838.

ปัญหาของการเรียนรู้เชิงลึก



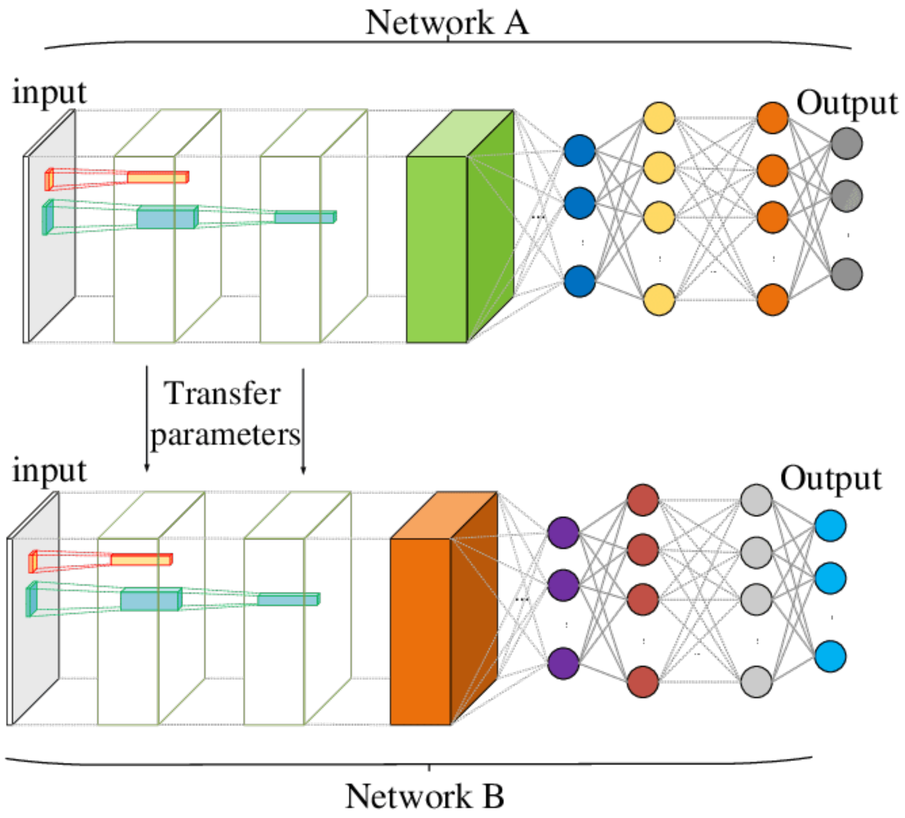




Transfer Learning

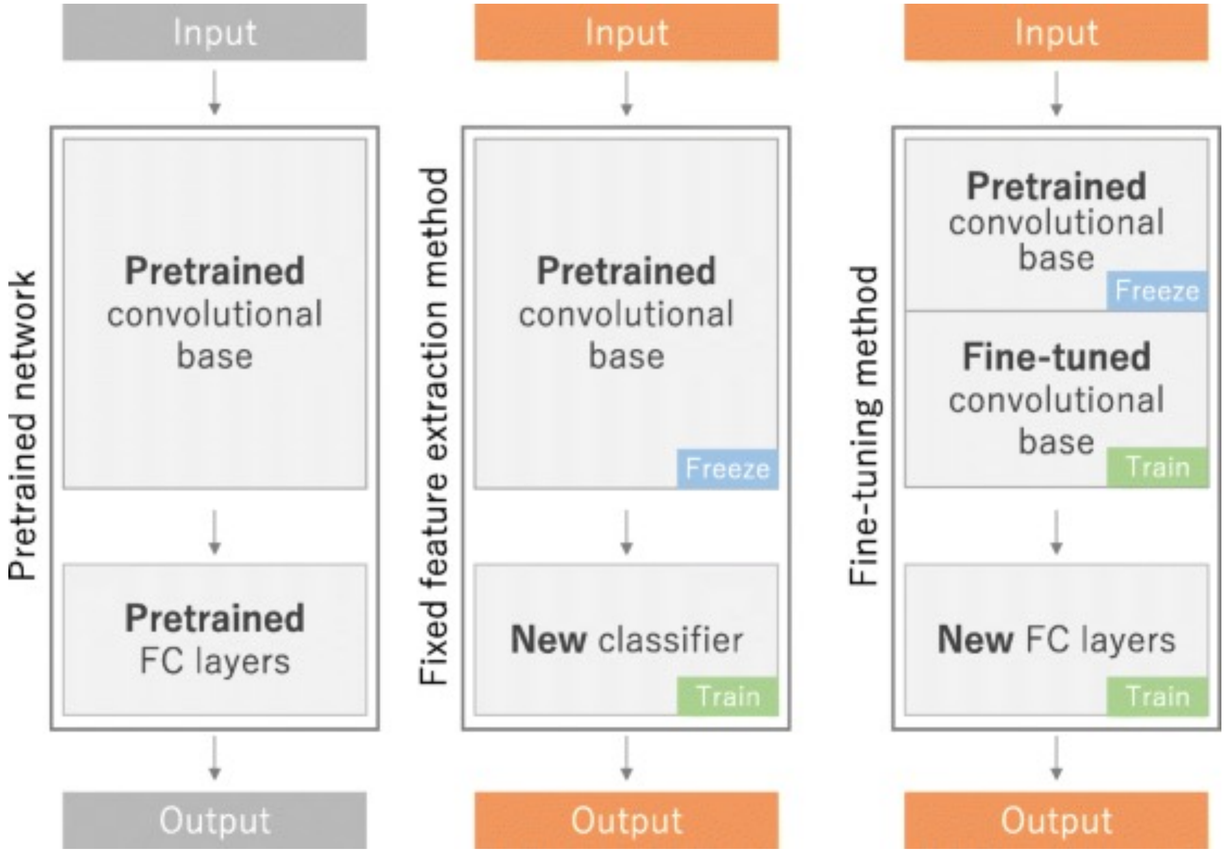
Data Augmentation

Transfer Learning



คือ การนำบางส่วนของแบบจำลองที่ผ่านกระบวนการฝึกสอนเรียบร้อยมาใช้ใหม่อีกครั้ง โดยนำมาเชื่อมต่อหรือประกอบเข้ากับแบบจำลองใหม่ เนื่องจากแบบจำลองส่วนใหญ่จะมีความซับซ้อนและมีตัวแปรที่ต้องกำหนดค่าค่อนข้างมาก ทำให้ต้องใช้ทรัพยากรคอมพิวเตอร์และเวลาในการประมวลผลค่อนข้างมากสำหรับการฝึกสอนเพื่อหาค่าตัวแปรต่าง ๆ ที่เหมาะสมและทำให้แบบจำลองมีประสิทธิภาพสูงสุด ซึ่งอาจจะกินเวลาเนินนานเป็น วัน สัปดาห์ เดือน เป็นต้น ดังนั้น การอาศัยเทคนิค Transfer Learning (หรือสามารถเรียกว่า Pretrained Networks) จะช่วยลดเวลาในการฝึกสอนแบบจำลองลงได้ค่อนข้างมาก โดยอาศัยบางส่วนของแบบจำลองที่มีลักษณะงานใกล้เคียงกันมาใช้ในแบบจำลองใหม่

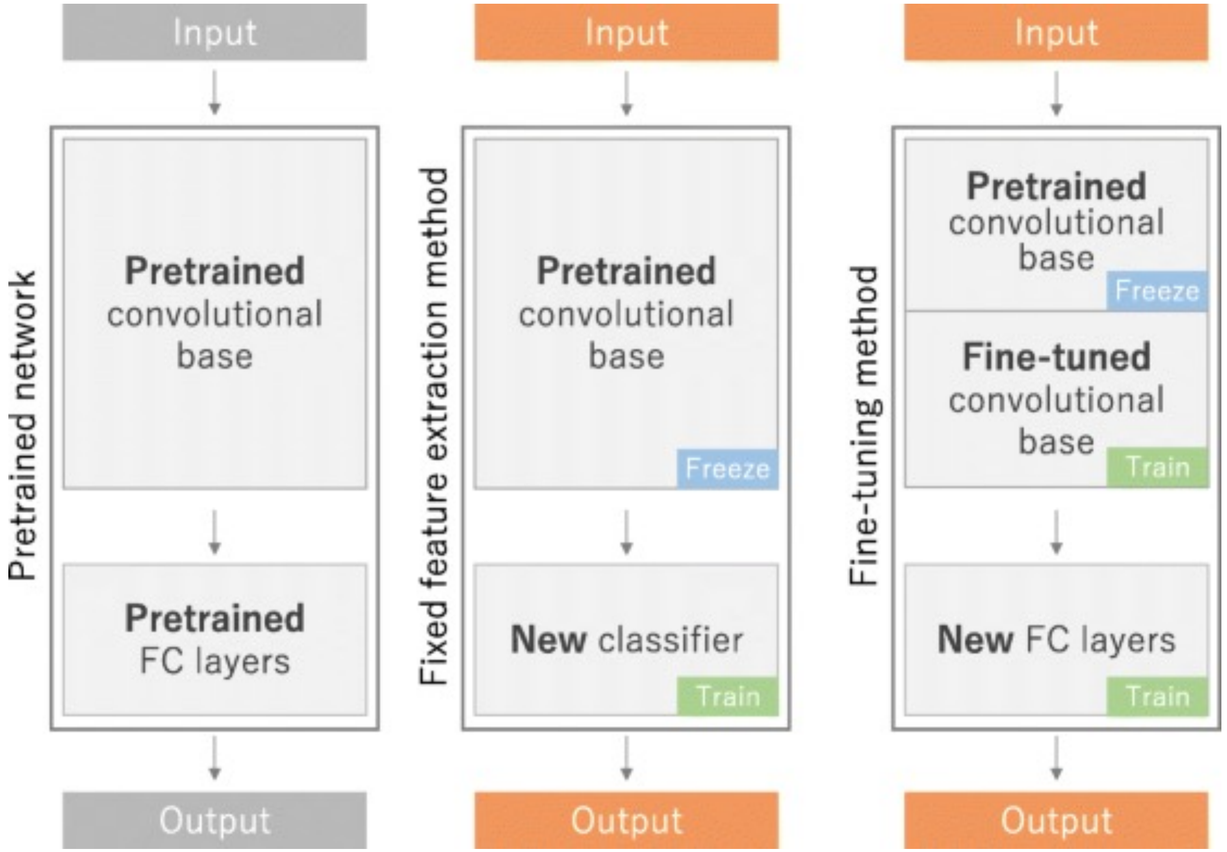
Transfer Learning



นอกจากนี้ การเรียนรู้แบบย้ายถ่าย (transfer learning) นิยมนำมาใช้งานกับการฝึกบนชุดข้อมูลขนาดเล็ก เนื่องจากเป็นกลยุทธ์ที่ใช้บ่อยและมีประสิทธิภาพในการฝึกโมเดลบนชุดข้อมูลขนาดเล็ก โดยที่โมเดลจะถูกฝึกบนชุดข้อมูลขนาดใหญ่มาก เช่น ImageNet ซึ่งประกอบด้วยภาพ 1.4 ล้านภาพที่มี 1,000 ประเภท จากนั้น นำกลับมาใช้ใหม่และประยุกต์ใช้กับงานที่สนใจ สมมติฐานพื้นฐานของการเรียนรู้แบบย้ายถ่าย คือ คุณสมบัติทั่วไปที่เรียนรู้จากชุดข้อมูลขนาดใหญ่สามารถแชร์ระหว่างชุดข้อมูลที่ดูเหมือนแตกต่างกันได้ ทำให้สามารถพกพาของคุณสมบัติทั่วไปที่เรียนรู้มาใช้ประโยชน์ในงานโดเมนต่าง ๆ ที่มีชุดข้อมูลขนาดเล็ก ปัจจุบัน หลายโมเดลที่ถูกฝึกบนชุดข้อมูล ImageNet ได้เปิดให้สาธารณชนเข้าถึงและใช้งานได้ง่าย พร้อมกับเคอร์เนลและน้ำหนักที่เรียนรู้มา เช่น AlexNet, VGG, ResNet, Inception และ DenseNet ในทางปฏิบัติมี 2 วิธีในการใช้โมเดลที่ถูกฝึกไว้แล้ว (1) การสกัดคุณสมบัติที่ถูกตรึง (fixed feature extraction) และ (2) การปรับแต่ง (fine-tuning)

- Yamashita, R., Nishio, M., Do, R.K.G. *et al.* Convolutional neural networks: an overview and application in radiology. *Insights Imaging* **9**, 611–629 (2018).

Transfer Learning

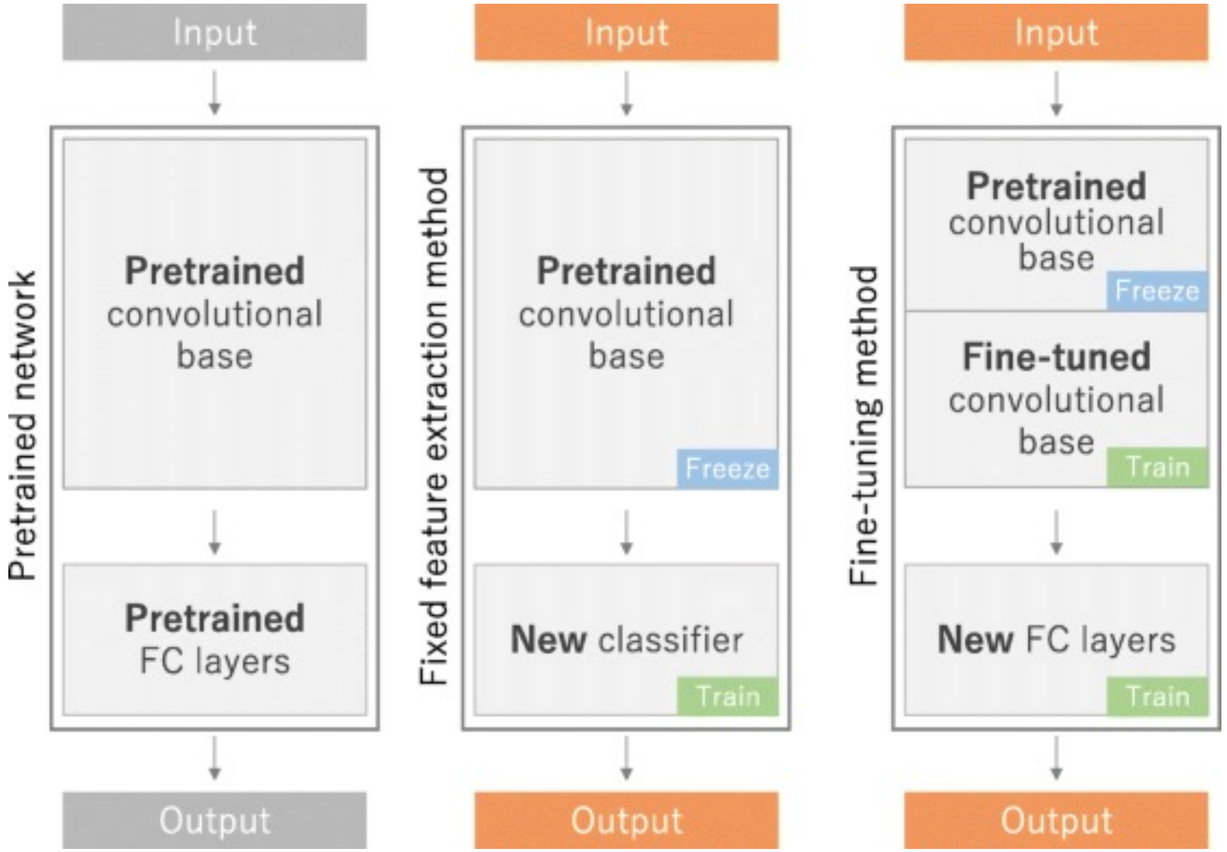


การสกัดคุณสมบัติที่ถูกตรึง (fixed feature extraction)

เป็นกระบวนการที่นำ FC layer ออกจากเครือข่ายที่ถูกฝึกบน ImageNet และรักษาเครือข่ายที่เหลือ ซึ่งประกอบด้วยชุดของเลเยอร์การคอนโวลูชันและการพูลไว้ เรียกว่าเบสคอนโวลูชัน (convolutional base) เพื่อเป็นตัวสกัดคุณสมบัติที่ถูกตรึง ในสถานการณ์นี้ เครื่องจำแนกประเภทใด ๆ ก็ได้ เช่น ป่าแบบสุ่ม (random forests) และเครื่องจักรเวกเตอร์สนับสนุน (support vector machines) รวมถึง FC layer ที่ใน CNNs สามารถถูกเพิ่มไว้บนตัวสกัดคุณสมบัติที่ถูกตรึง ส่งผลให้การฝึกถูกจำกัดอยู่แค่ตัวจำแนกที่เพิ่มขึ้นบนชุดข้อมูลที่สนใจ วิธีนี้ไม่ค่อยพบในงานวิจัยการเรียนรู้เชิงลึกบนภาพทางการแพทย์เนื่องจากความไม่เหมือนกันระหว่าง ImageNet และภาพทางการแพทย์ที่ให้มา

- Yamashita, R., Nishio, M., Do, R.K.G. *et al.* Convolutional neural networks: an overview and application in radiology. *Insights Imaging* **9**, 611–629 (2018).

Transfer Learning

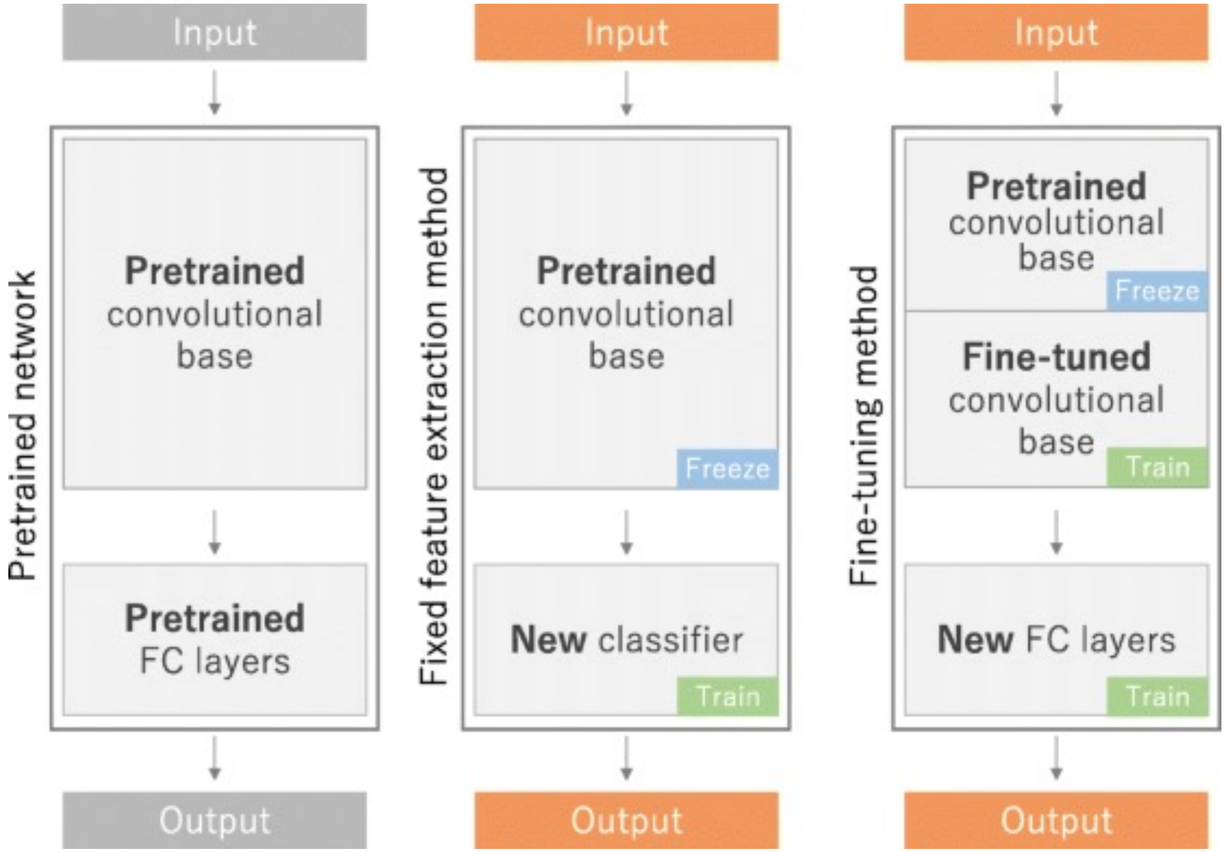


การปรับแต่ง (fine-tuning)

คือ การไม่เพียงแค่แทนที่ FC layer ของโมเดลที่ถูกฝึกด้วยชุด FC layer ใหม่เพื่อฝึกบนชุดข้อมูลที่ให้มา แต่ยังปรับแต่งเคอร์เนลทั้งหมดหรือบางส่วนในเบสคอนโวลูชันที่ถูกฝึกไว้แล้วด้วยการถอยกลับ (backpropagation) เลเยอร์ทั้งหมดในเบสคอนโวลูชันสามารถถูกปรับแต่งหรือ เลเยอร์บางส่วนที่อยู่ก่อนหน้าอาจถูกตรึงไว้ในขณะที่ปรับแต่งเลเยอร์ที่ลึกกว่าที่เหลือ แรงจูงใจของวิธีนี้มาจากการสังเกตว่า คุณสมบัติของเลเยอร์ที่อยู่ก่อนหน้าดูเหมือนจะเป็นคุณสมบัติทั่วไป รวมถึงคุณสมบัติเช่นขอบที่สามารถประยุกต์ใช้กับชุดข้อมูลและงานที่หลากหลาย ในขณะที่คุณสมบัติในเลเยอร์หลัง ๆ จะค่อย ๆ เฉพาะเจาะจงมากขึ้นกับชุดข้อมูลหรืองานใดงานหนึ่ง ซึ่งถูกประยุกต์ใช้ในการวิจัยรังสีวิทยามากกว่า

- Yamashita, R., Nishio, M., Do, R.K.G. *et al.* Convolutional neural networks: an overview and application in radiology. *Insights Imaging* **9**, 611–629 (2018).

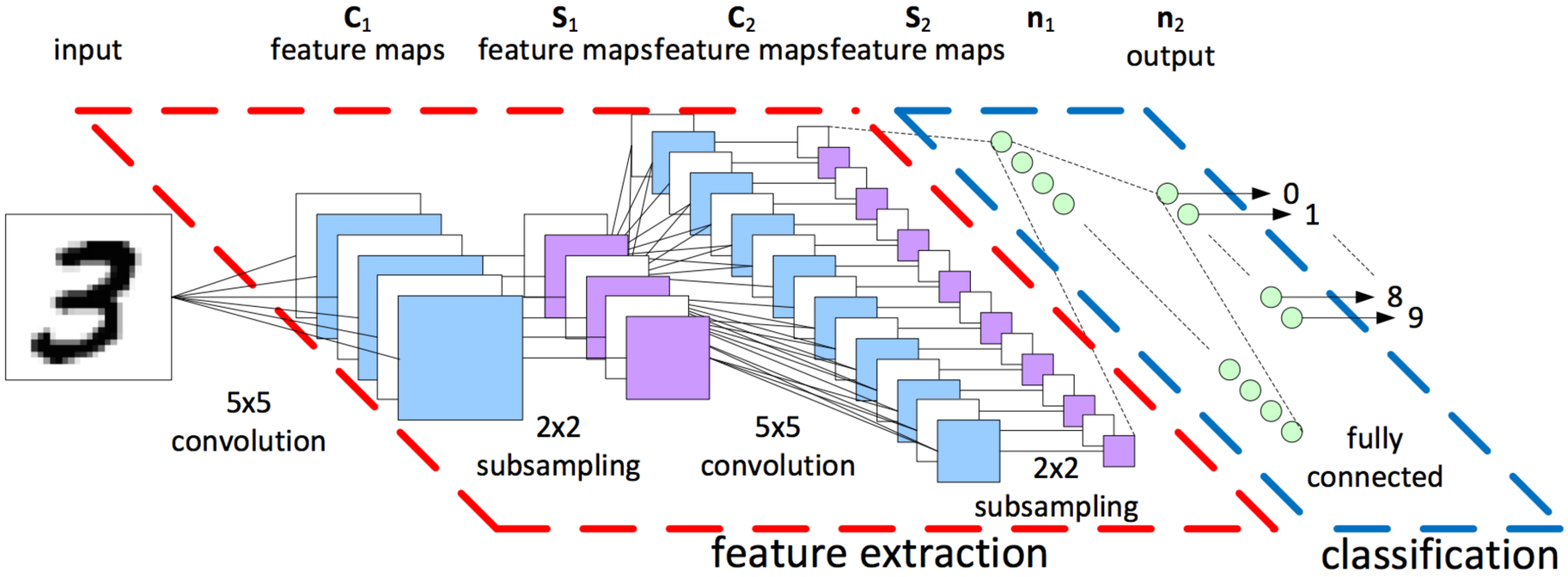
Transfer Learning



ข้อเสียของการเรียนรู้แบบย้ายถ่าย คือ ข้อจำกัดในมิติของข้อมูลนำเข้า ภาพที่นำเข้าต้องเป็นภาพสีบนปริภูมิ RGB เนื่องจากชุดข้อมูล ImageNet ประกอบด้วยภาพสีบนปริภูมิ RGB ในขณะที่ ภาพเกรย์สเกลทางการแพทย์มีเพียงช่องสีเดียว (ระดับของสีเทา) อย่างไรก็ตาม ความสูงและความกว้างของภาพที่นำเข้าอาจเป็นไปตามต้องการ แต่ไม่เล็กเกินไป โดยการเพิ่ม Global Pooling Layer ระหว่างเบสคอนโวลูชันและ FC Layer ที่เพิ่มขึ้น

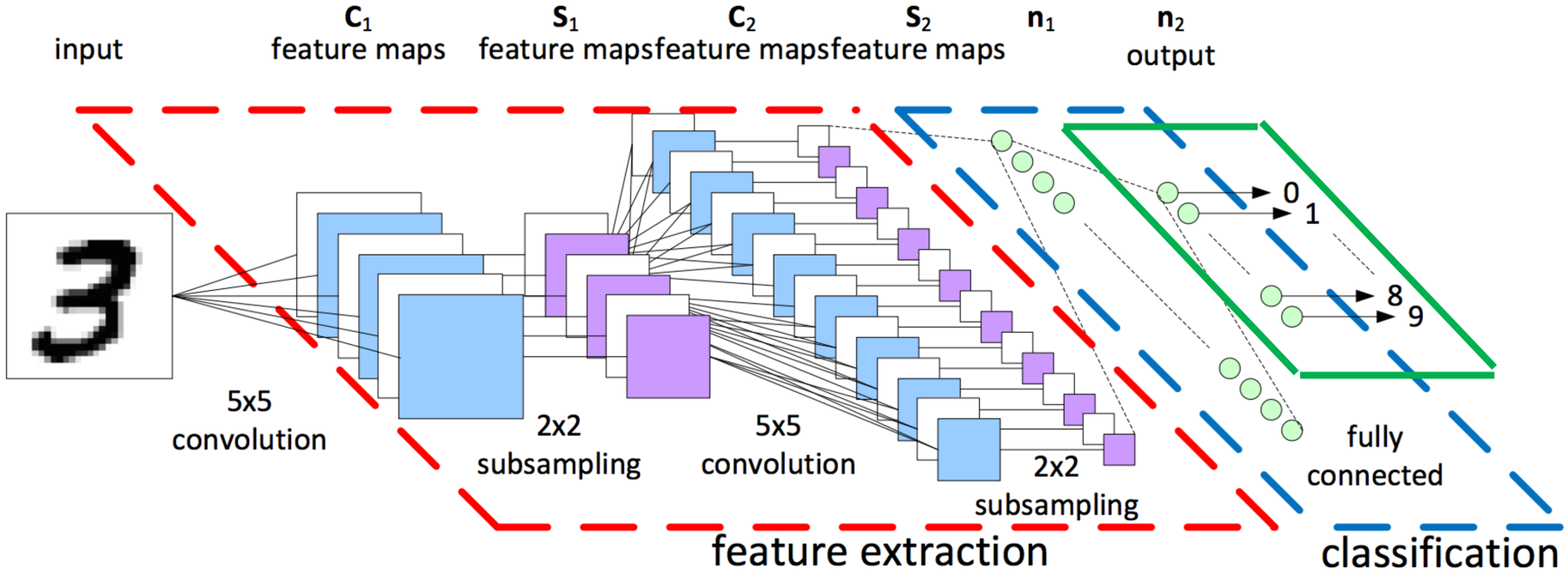
- Yamashita, R., Nishio, M., Do, R.K.G. *et al.* Convolutional neural networks: an overview and application in radiology. *Insights Imaging* **9**, 611–629 (2018).

Transfer Learning



แบบที่ 1 นำแบบจำลองที่ฝึกสอนเรียบร้อยแล้วมาฝึกสอนเพิ่มเติมกับข้อมูลชุดใหม่ โดยไม่มีการนำส่วนใดของแบบจำลองเดิมออก เพื่อให้ได้ประสิทธิภาพที่สูงขึ้นกับงานใหม่ ***(ใช้ทั้งแบบจำลอง)***

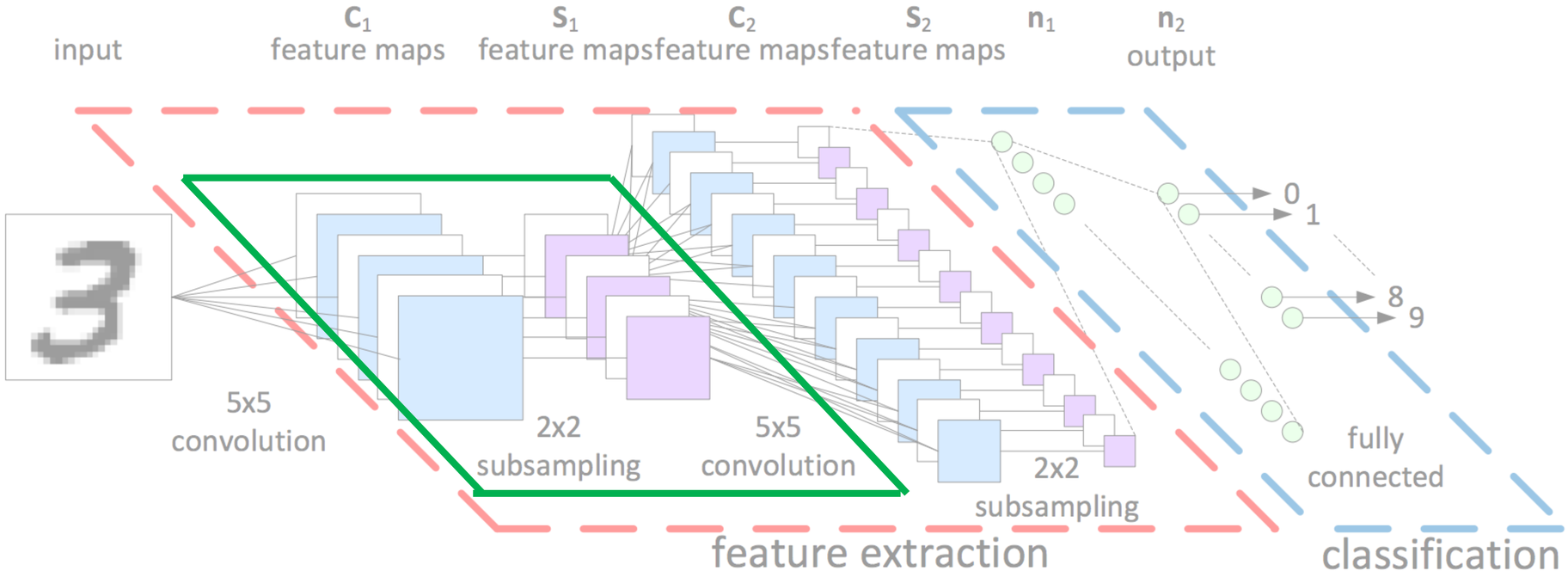
Transfer Learning



แบบที่ 2 นำส่วนของ Feature Extraction ของแบบจำลองที่ฝึกสอนแล้วมาใช้งาน ตัวอย่างเช่น นำ แบบจำลองของ ConvNet มาใช้งาน โดยการนำส่วนของ Dense Layer สุดท้ายออก จากนั้น **สร้าง Classifier ใหม่**และนำไปฝึกสอนใหม่กับชุดข้อมูลฝึกสอนใหม่ที่มีขนาดเล็กกว่าหรือข้อมูลที่มีความจำเพาะมากยิ่งขึ้น ***(ใช้ส่วนการสกัดคุณลักษณะเท่านั้น)***

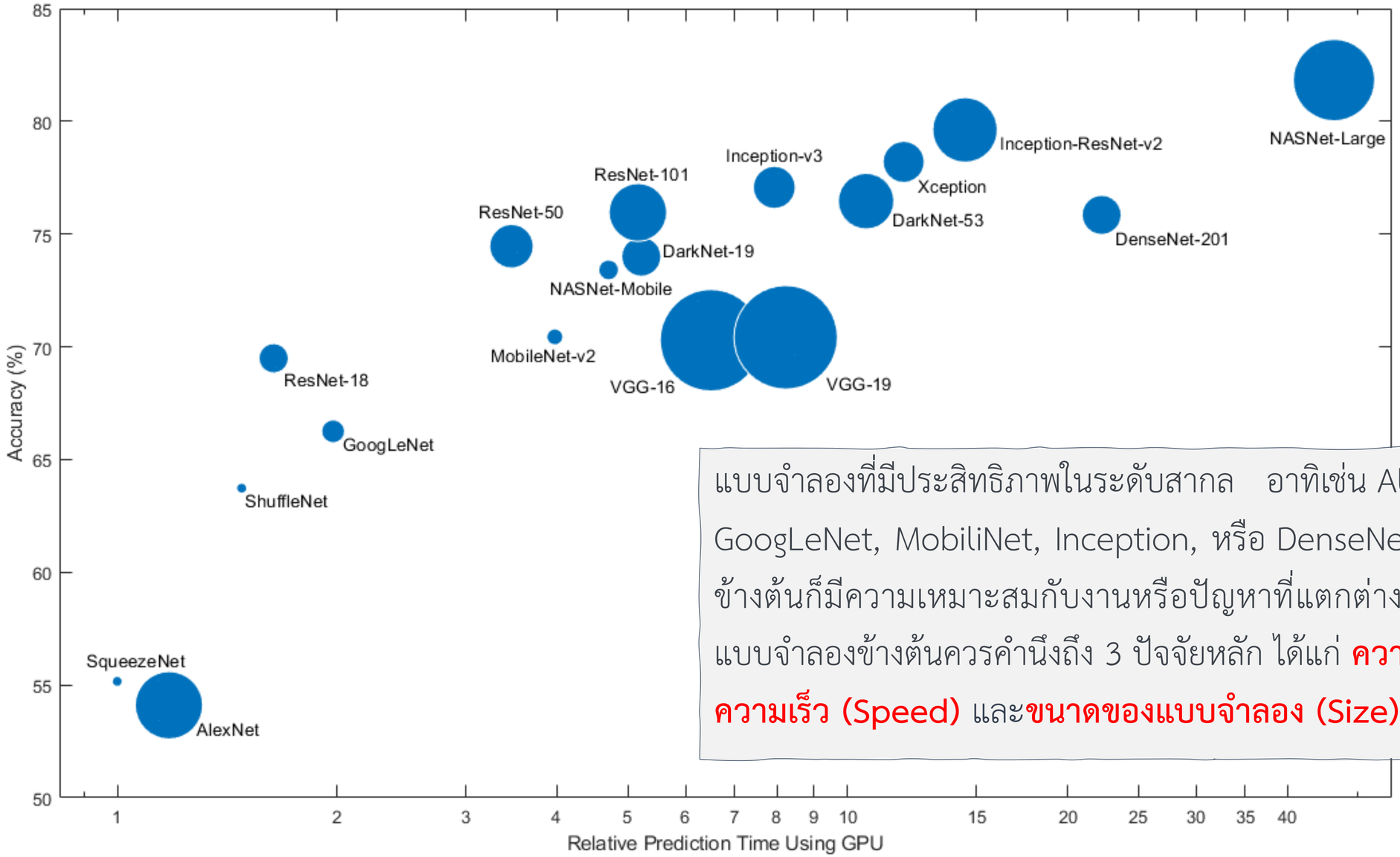
https://www.bualabs.com/archives/3493/what-is-transfer-learning-build-headless-mobilenet-model-transfer-learning-machine-learning-mobilenet-json-retrain-webcam-tensorflow-js-tfjs-ep-10/

Transfer Learning



แบบที่ 3 อาศัย***บางส่วน***ของแบบจำลองที่มีลักษณะงานใกล้เคียงกันมาใช้ในแบบจำลองใหม่ ***(ใช้แค่บางส่วนโดยเฉพาะอย่างยิ่งส่วนของการสกัดคุณลักษณะ)***

Transfer Learning



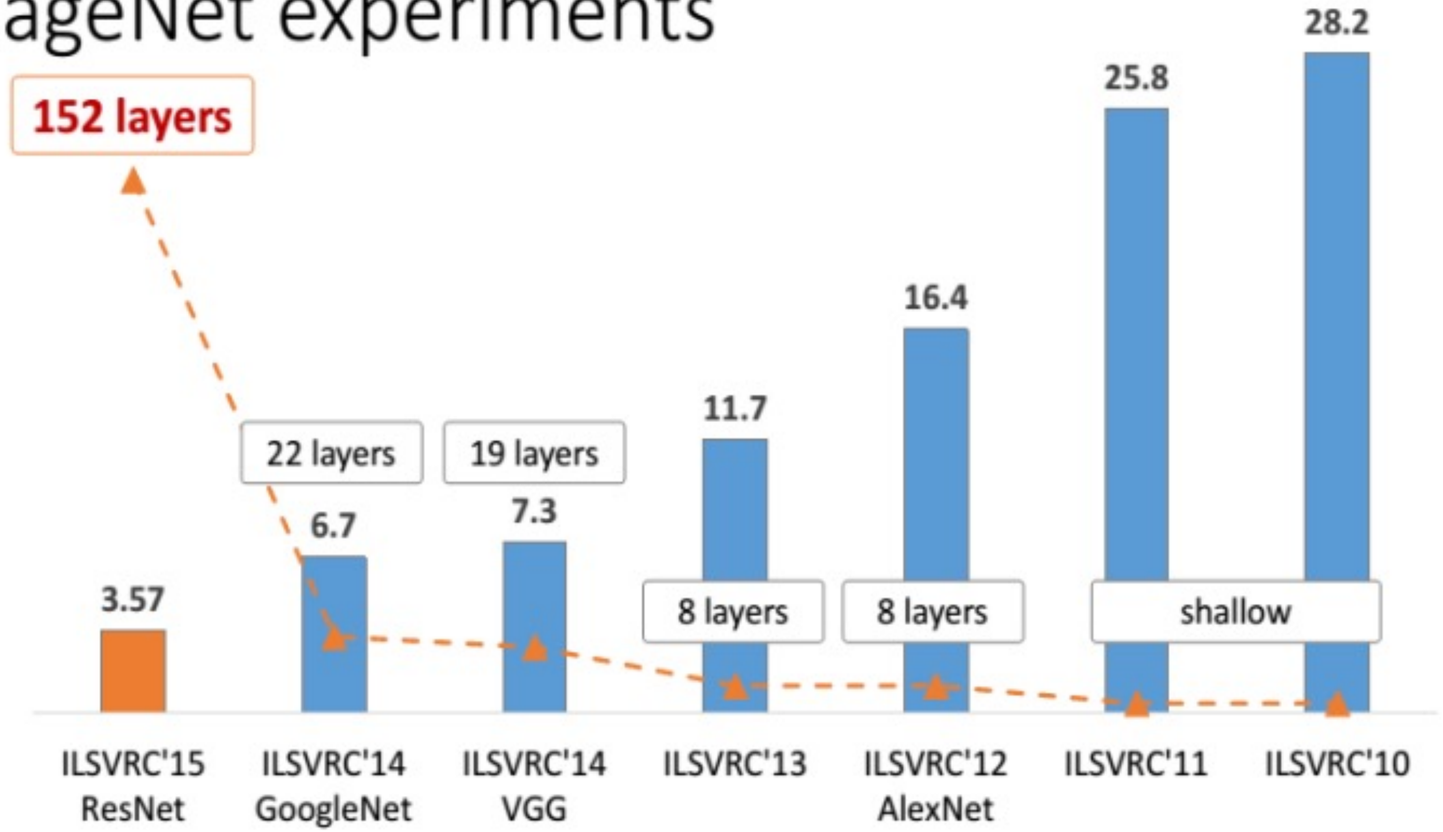
แบบจำลองที่มีประสิทธิภาพในระดับสากล อาทิเช่น AlexNet, ResNet, VGG-19, GoogLeNet, MobiliNet, Inception, หรือ DenseNet เป็นต้น ซึ่งแต่ละแบบจำลองข้างต้นก็มีความเหมาะสมกับงานหรือปัญหาที่แตกต่างกันออกไป ซึ่งการเลือกใช้งานแบบจำลองข้างต้นควรคำนึงถึง 3 ปัจจัยหลัก ได้แก่ **ความแม่นยำ (Accuracy) ความเร็ว (Speed)** และ**ขนาดของแบบจำลอง (Size)**

แนะนำเกี่ยวกับโครงข่ายประสาทเชิงคอนโวลูชันที่เป็นที่นิยม

Introduction to Conventional Convolutional Neural Networks

Microsoft Research

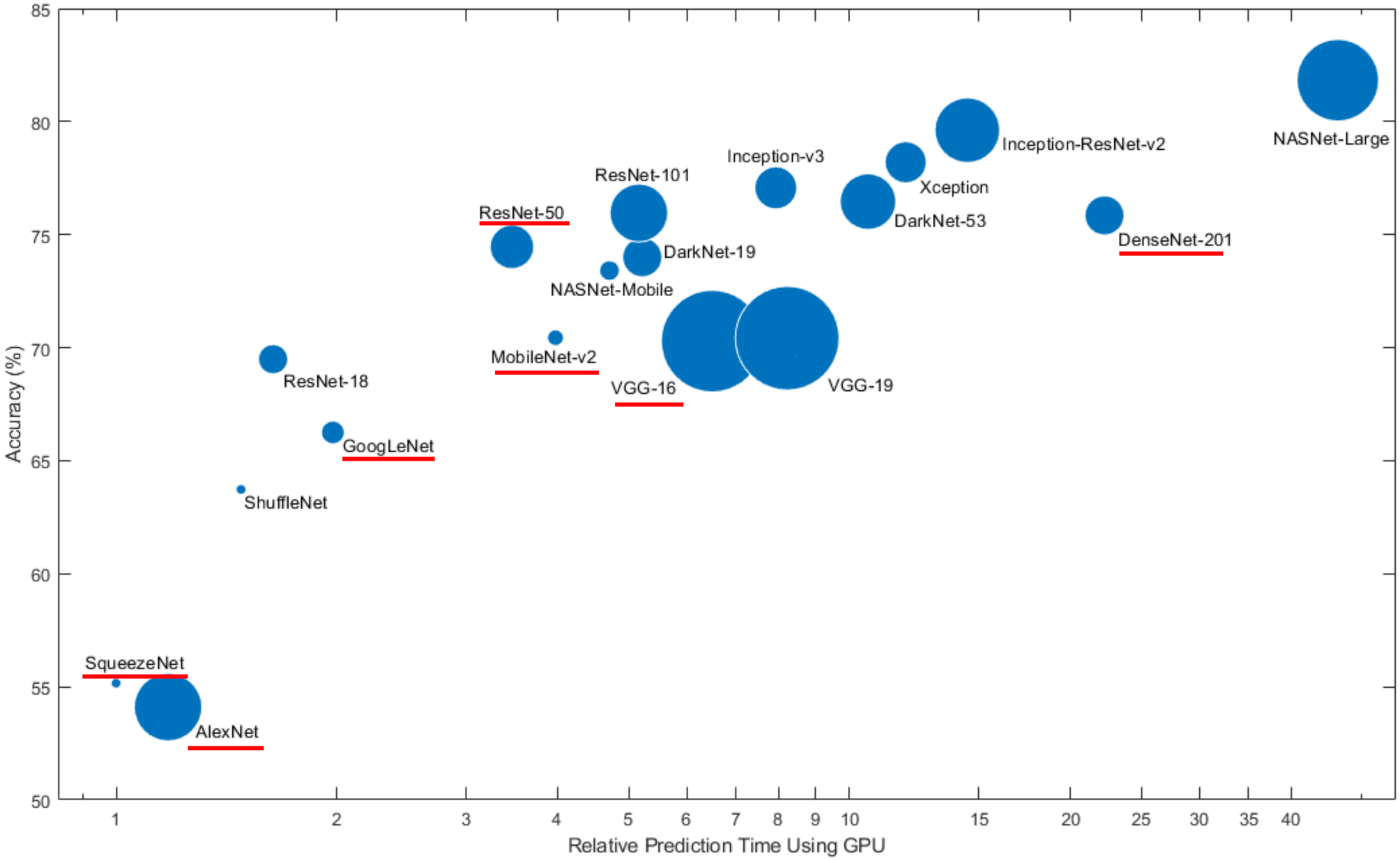
ImageNet experiments



ImageNet Classification top-5 error (%)

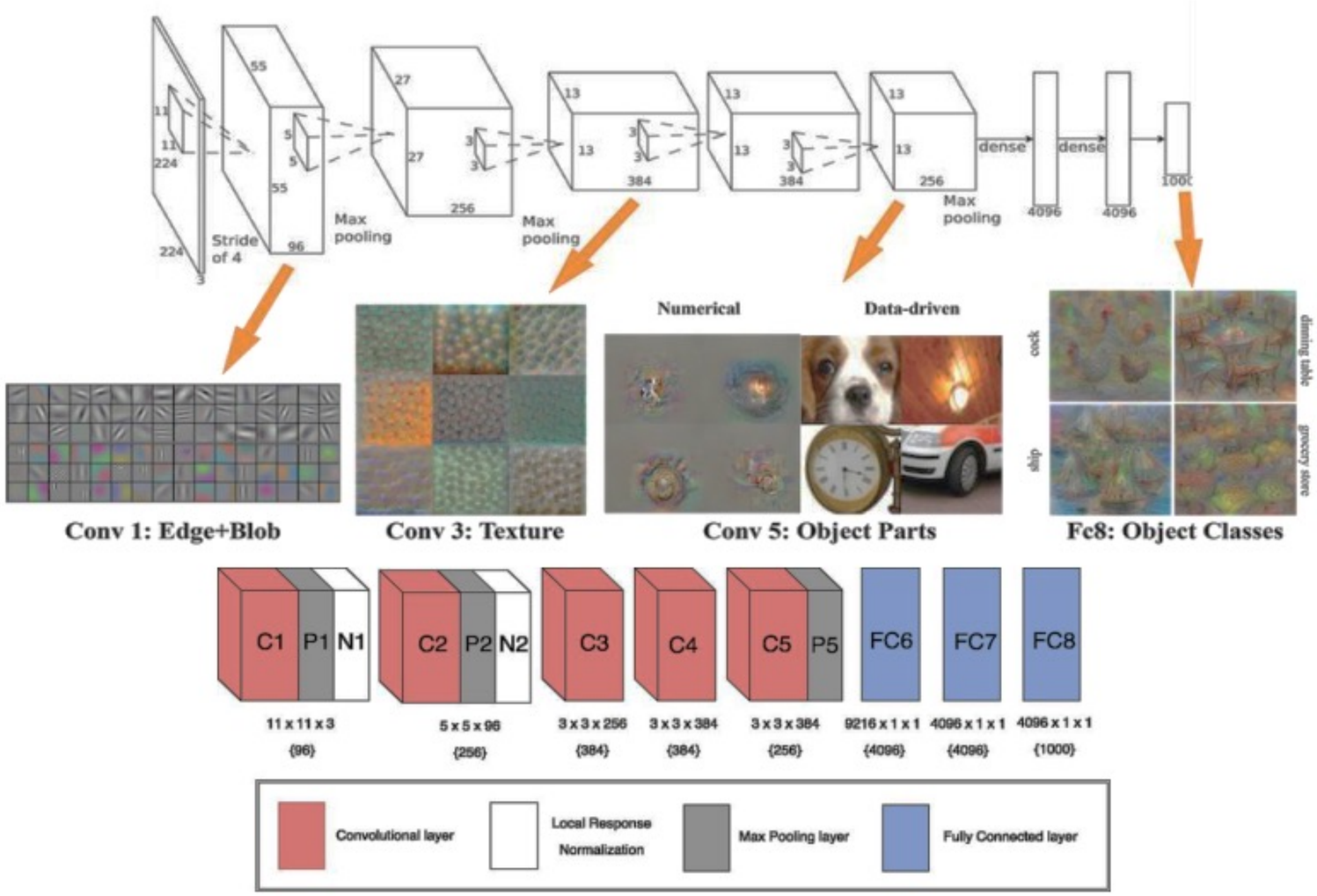
ICCV15

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". arXiv 2015.


Accuracy (%)
85
80
75
70
65
60
55
50
1
2
3
4
5
6
7
8
9
10
15
20
25
30
35
40
Relative Prediction Time Using GPU
NASNet-Large
Inception-ResNet-v2
Inception-v3
Xception
ResNet-101
DarkNet-53
ResNet-50
DenseNet-201
DarkNet-19
NASNet-Mobile
MobileNet-v2
VGG-16
VGG-19
ResNet-18
GoogLeNet
ShuffleNet
SqueezeNet
AlexNet

AlexNet Architecture

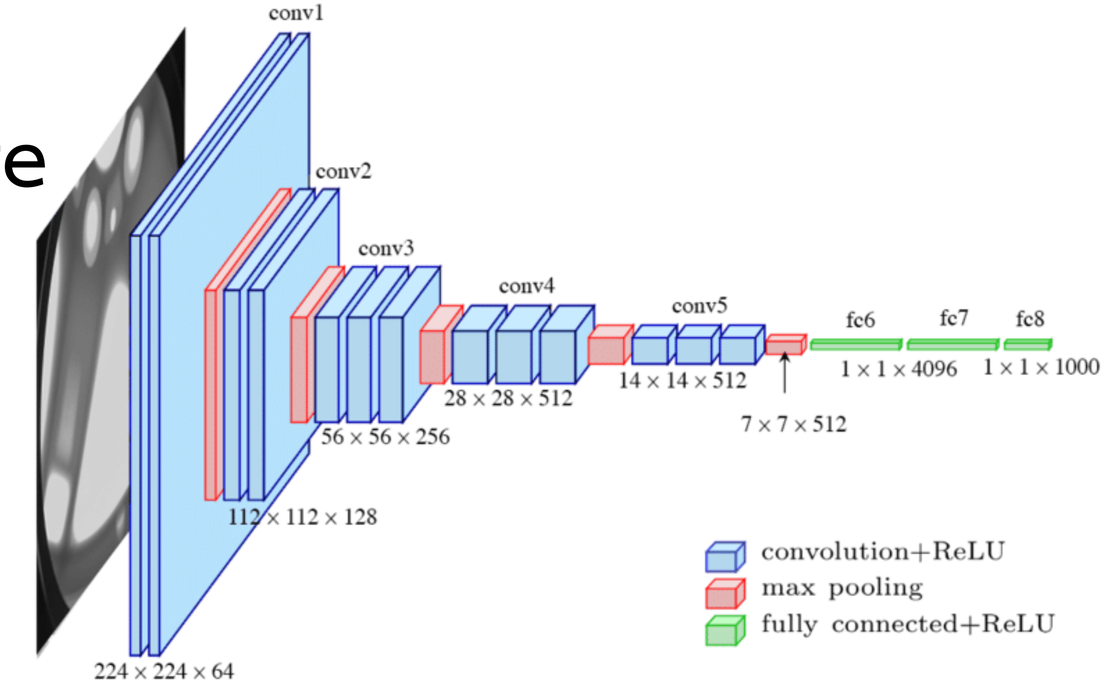
AlexNet เป็นสถาปัตยกรรมของโครงข่ายประสาทเทียมแบบคอนโวลูชั่นที่ได้รับการนำเสนอขึ้นในปี 2012 โดย Alex Krizhevsky และคณะในการแข่งขัน ImageNet ซึ่งสถาปัตยกรรมแบบ AlexNet ประกอบไปด้วย convolutional layers จำนวน 5 เลเยอร์ และ fully-connected layers จำนวน 3 เลเยอร์ อย่างไรก็ตาม สถาปัตยกรรมแบบ AlexNet ประสบกับปัญหา overfitting เนื่องจากมีจำนวนพารามิเตอร์ 60 ล้านตัว นอกจากนี้ AlexNet ได้ (1) ใช้งาน ReLU แทนการใช้งาน Tanh แบบเดิม (2) มีการนำเทคนิค Dropout มาใช้งานเพื่อหลีกเลี่ยงปัญหา overfitting แต่ก็ทำให้ใช้เวลาในการคำนวณมากขึ้นถึง 2 เท่า



AlexNet (original):Krizhevsky et.al., ImageNet Classification with Deep Convolutional Neural Networks, 2012.
CaffeNet: Jia et.al., Caffe: Convolutional Architecture for Fast Feature Embedding, 2014.
Image credit: Roberto Matheus Pinheiro Pereira, "Deep Learning Talk".
Srinivas et.al "A Taxonomy of Deep Convolutional Neural Nets for Computer Vision", 2016.

อ้างอิง https://www.jeremyjordan.me/convnet-architectures/

VGG Architecture

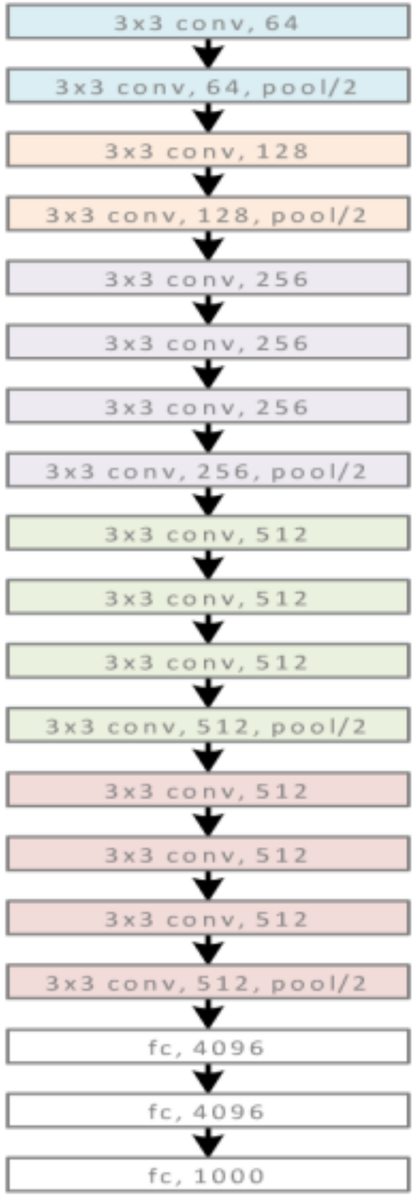


VGG เป็นสถาปัตยกรรมของโครงข่ายประสาทเทียบแบบคอนโวลูชั่นรูปแบบหนึ่งที่ได้รับการนำเสนอขึ้นในปี 2014 โดย Simonyan และคณะจาก University of Oxford ก่อนหน้านี้ สถาปัตยกรรมแบบ AlexNet ได้มุ่งเน้นไปที่หน้าต่าง (Window size) และการขยับ (Stride) ขนาดเล็กในเลเยอร์คอนโวลูชั่นต้น ซึ่งในสถาปัตยกรรมแบบ VGG กล่าวถึงอีกแง่มุมที่สำคัญมาก CNNs คือ ผลกระทบของความลึกของเครือข่ายประสาทที่ส่งผลต่อความแม่นยำ

สถาปัตยกรรมแบบ VGG รองรับภาพสีขนาด 224 x 224 พิกเซล จากนั้น จะมีการทำ Pre-processing โดยการ Normalization ช่วงของข้อมูลให้อยู่ในช่วง 0 ถึง 255 ก่อนนำผลลัพธ์ที่ได้ไปลบด้วยค่าเฉลี่ยสีจากชุดข้อมูลฝึกสอน (ImageNet) ซึ่ง VGG จะประกอบไปด้วย convolutional layers จำนวน 13 เลเยอร์ และ fully connected layers อีก 3 เลเยอร์ ซึ่งทุก convolutional layers จะกำหนดให้ตัวกรองมาก (ลึกมาก) และมีขนาด 3 x 3 พิกเซล ซึ่ง Simonyan และคณะมีความเชื่อว่ามันจะมีประสิทธิภาพไม่ต่างจากตัวกรองขนาด 7 x 7 พิกเซล นอกจากนี้ VGG ได้อาศัยหลักการ 1 x1 convolutional layer สำหรับการทำนายค่าของผลลัพธ์

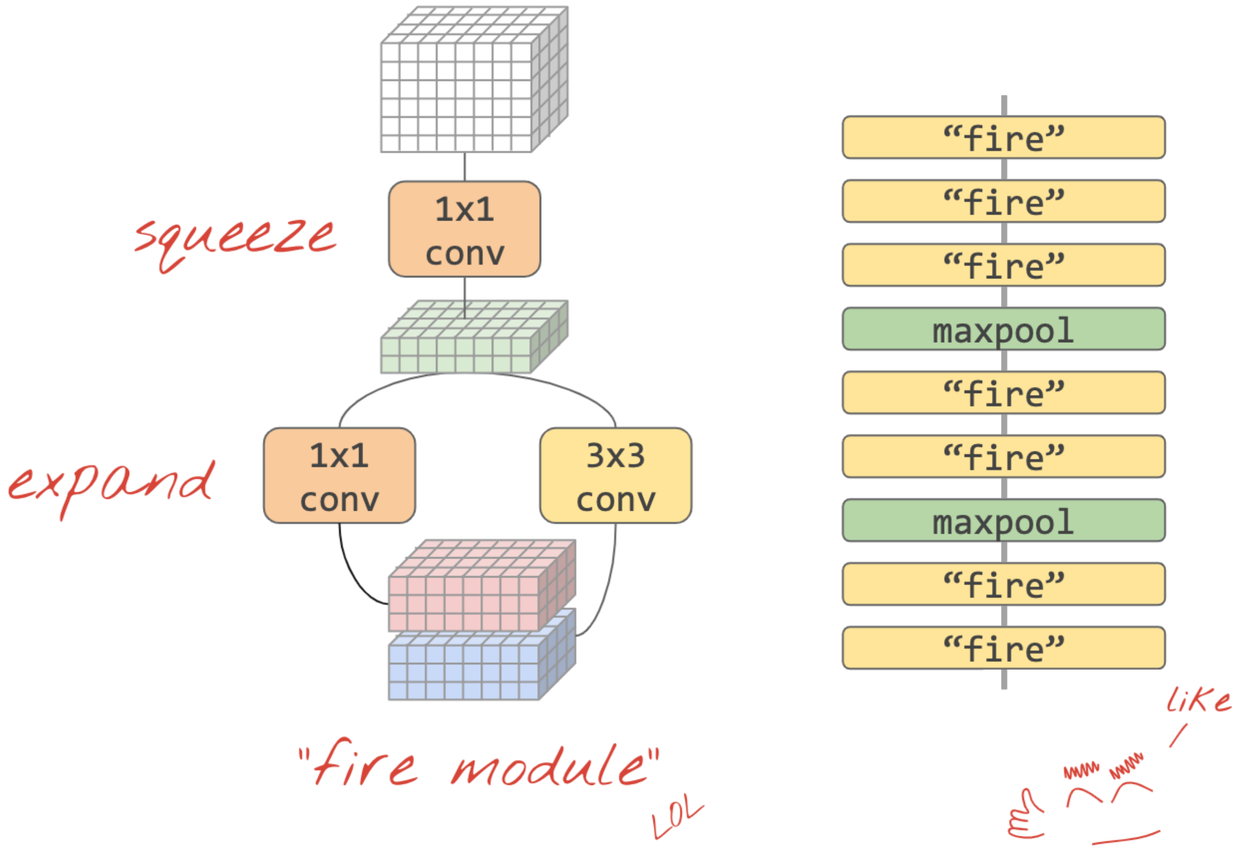
อ้างอิง Ferguson et al. (2017). Automatic localization of casting defects with convolutional neural networks. 1726-1735. และ https://medium.com/analytics-vidhya/vggnet-architecture-explained-e5c7318aa5b6

VGG Architecture



All convolutions are 3x3
Good performance,
but slow

SqueezeNet Architecture



https://codelabs.developers.google.com/codelabs/keras-flowers-squeezenet#6

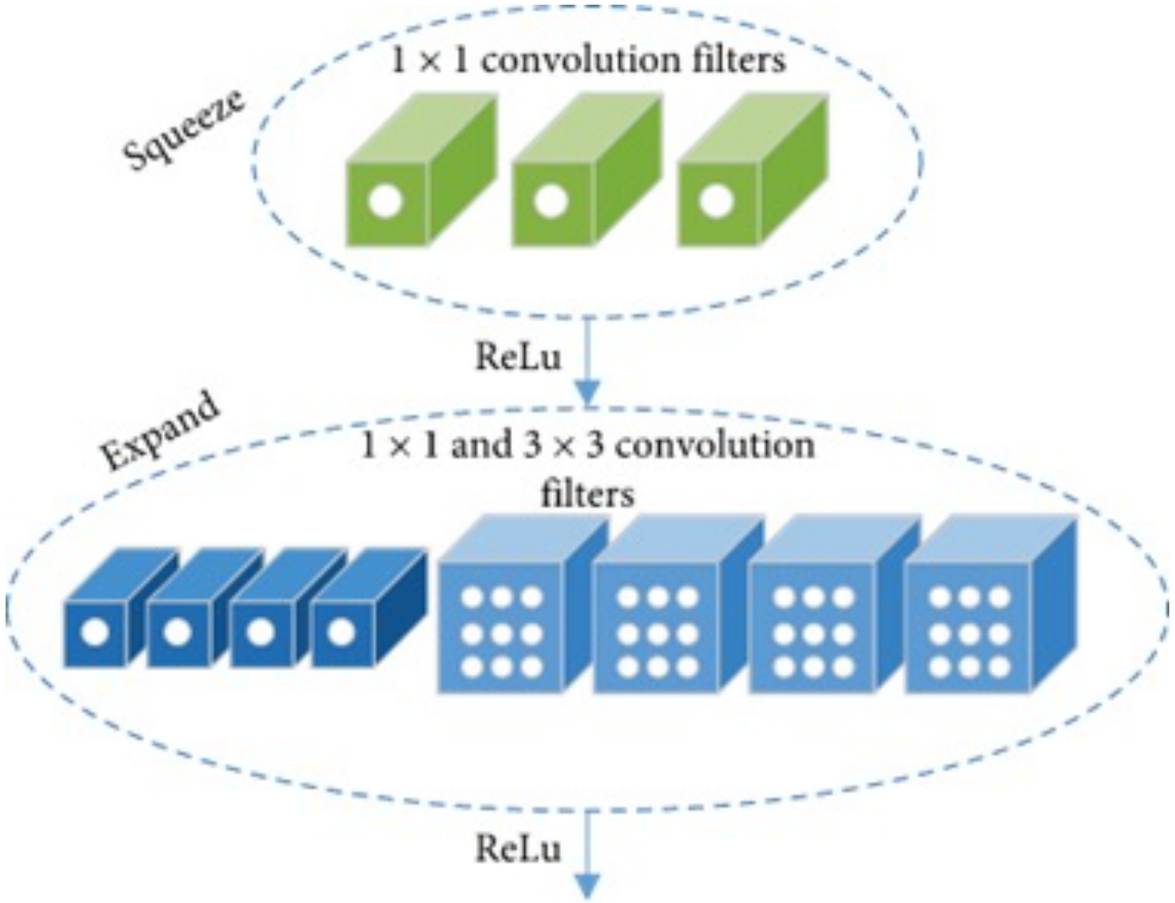
SqueezeNet เป็นสถาปัตยกรรมของโครงข่ายประสาทเทียมแบบคอนโวลูชั่นที่ได้รับการนำเสนอขึ้นในปี 2016 ซึ่งได้นำเสนอโมดูล Fire ที่ได้รับการออกแบบมาจาก 3 แนวคิด ได้แก่

- แทนที่ตัวกรองขนาด 3x3 ด้วยตัวกรองขนาด 1x1 (point-wise filters) เนื่องจากมีจำนวนพารามิเตอร์ลดลง 9 เท่า
- การใช้งานตัวกรองขนาด 1x1 สามารถลดความลึกของแผนที่คุณลักษณะและเวลาที่ใช้ในการคำนวณได้
- การใช้ Pooling ล่าช้า เพื่อเก็บรักษาภาพรวมของแผนที่คุณลักษณะไว้ ส่งผลทำให้เพิ่มประสิทธิภาพการทำงาน

โมดูล Fire จะประกอบด้วย 2 ส่วน ได้แก่ squeeze layer และ expand layer

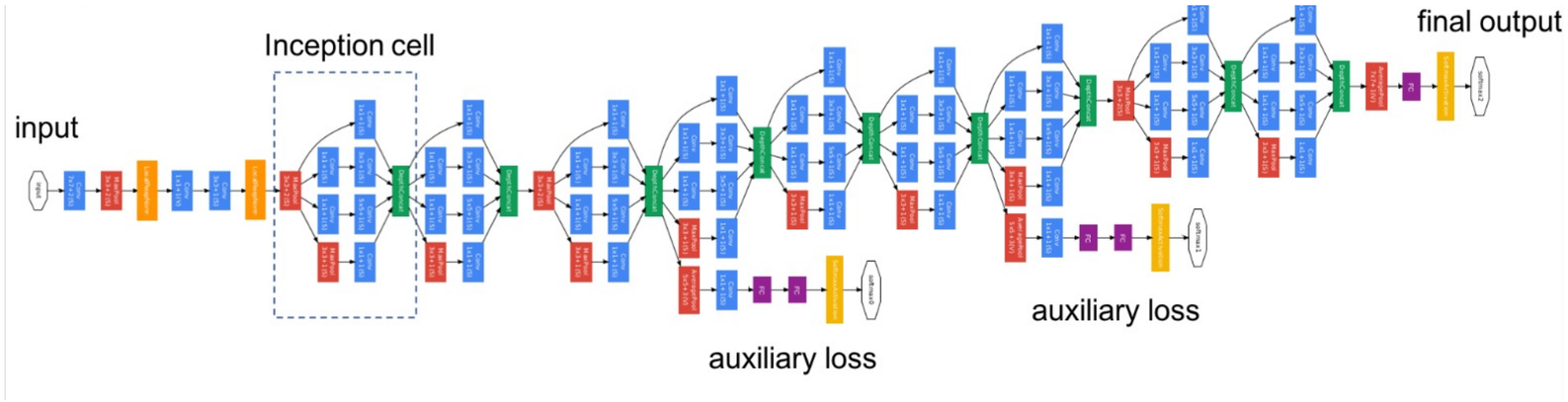
อ้างอิง https://medium.com/@avidrishik/squeezenets-architecture-compressed-neural-network-7741d24ca56f และ https://towardsdatascience.com/review-squeezenet-image-classification-e7414825581a

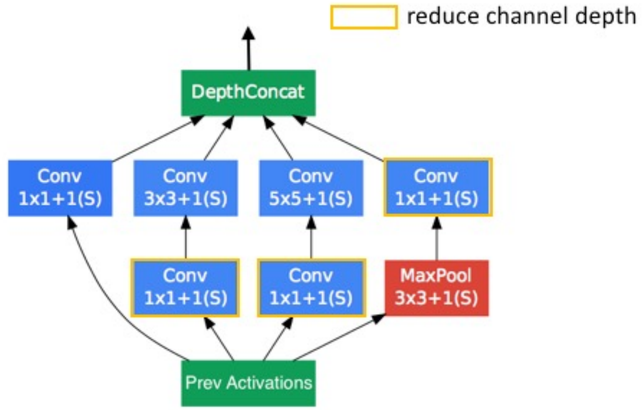
SqueezeNet Architecture



อ้างอิง https://medium.com/@avidrishik/squeezenets-architecture-compressed-neural-network-7741d24ca56f และ https://towardsdatascience.com/review-squeezenet-image-classification-e7414825581a

GoogLeNet (aka Inception) Architecture





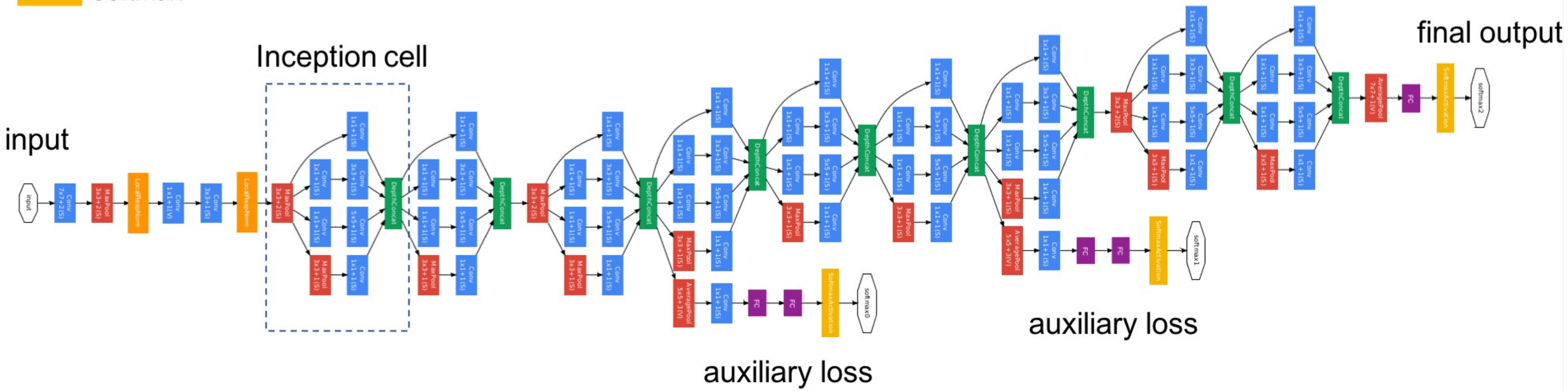
อ้างอิง https://www.jeremyjordan.me/convnet-architectures/

GoogLeNet (หรือ Inception) เป็นสถาปัตยกรรมของโครงข่ายประสาทเทียบแบบคอนโวลูชั่นที่ได้รับการนำเสนอขึ้นในปี 2014 โดยทีมนักวิจัยจาก Google เพื่อ (1) ประหยัดทรัพยากรในการคำนวณ อาทิเช่น ลดจำนวนพารามิเตอร์ ลดจำนวนครั้งของ operator ลง โดยอาศัยหลักการ 1x1 convolution ร่วมกับหลักการ Inception cell จำนวน 9 cell ซึ่งภายใน Inception cell จะประกอบไปด้วยชุดของตัวกรองขนาด 1x1, 3x3 และ 5x5 ที่สามารถเรียนรู้ที่จะสกัดคุณลักษณะในขนาดของวัตถุที่แตกต่างกันจากในภาพได้

อ้างอิง https://www.jeremyjordan.me/convnet-architectures/ และ https://medium.com/coinmonks/paper-review-of-googlenet-inception-v1-winner-of-ilsvlc-2014-image-classification-c2b3565a64e7

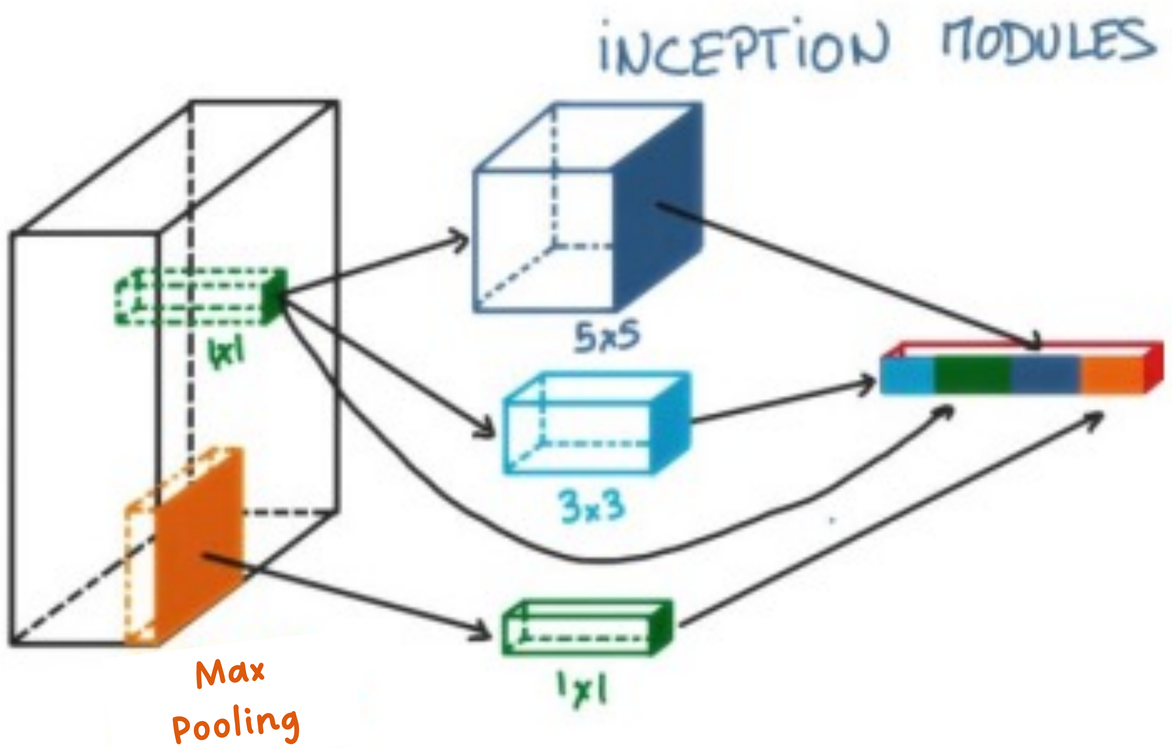
GoogLeNet (aka Inception) Architecture

- convolution
- max pooling
- channel concatenation
- channel-wise normalization
- fully-connected layer
- softmax



อ้างอิง https://www.jeremyjordan.me/content/images/2018/04/inception-model.png

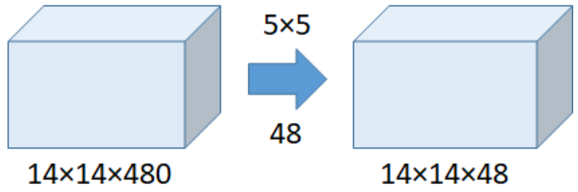
GoogLeNet (aka Inception) Architecture



GoogLeNet (aka Inception) Architecture

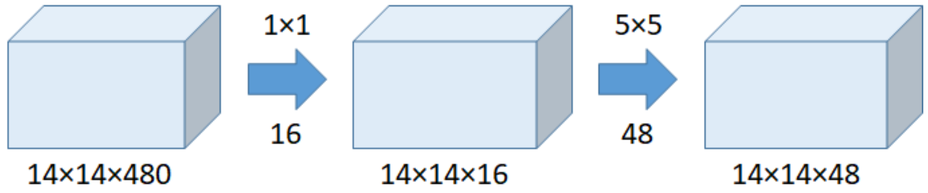
ตัวดำเนินการ 1×1 convolution ได้รับการคิดค้นขึ้นในปี 2014 จากบทความที่มีชื่อว่า Network in Network (NIN) เพื่อทำให้แบบจำลองมีประสิทธิภาพดีขึ้นโดยการเพิ่มความเป็น Non-linearity ของแบบจำลองให้เหมาะสมกับข้อมูลประเภท Non-linear ซึ่งนิยมใช้งานร่วมกับ ReLU แต่ใน GoogLeNet ได้นำ 1×1 convolution มาใช้งานเพื่อลดขนาดของแผนที่คุณลักษณะลง ส่งผลให้ลดการใช้ทรัพยากรสำหรับการคำนวณลง ซึ่งจากรูปจะพบว่า 1×1 และ 5×5 convolution จะใช้จำนวนครั้งในการประมวลผลที่ต่างกัน นอกจากนี้ 1×1 convolution สามารถช่วยลดพารามิเตอร์ของแบบจำลองลง ส่งผลทำให้ช่วยลดปัญหา overfitting ได้

- **กรณี 5×5 convolution**



Number of operations = (14×14×48)×(5×5×480) = 112.9M

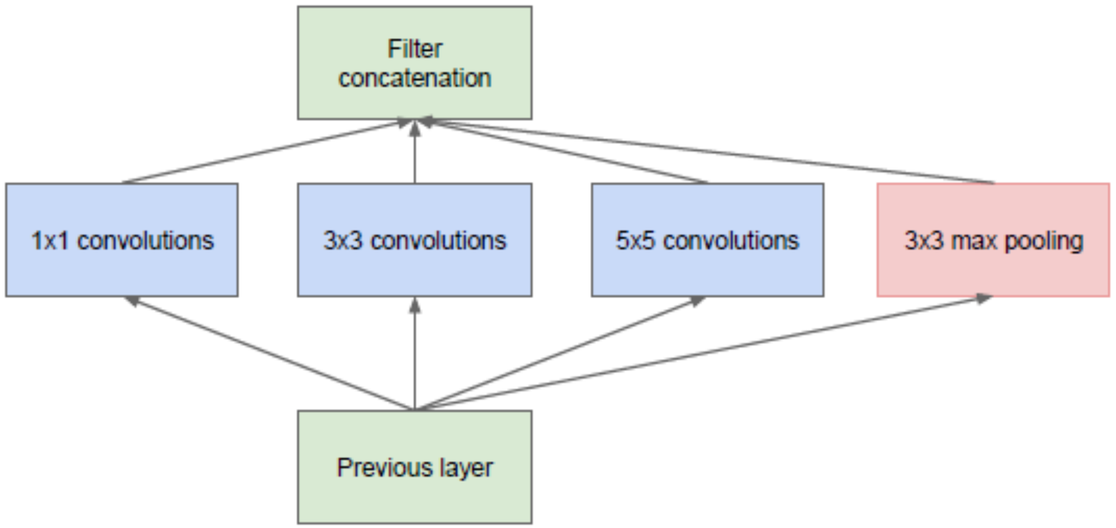
- **กรณี 1×1 convolution**



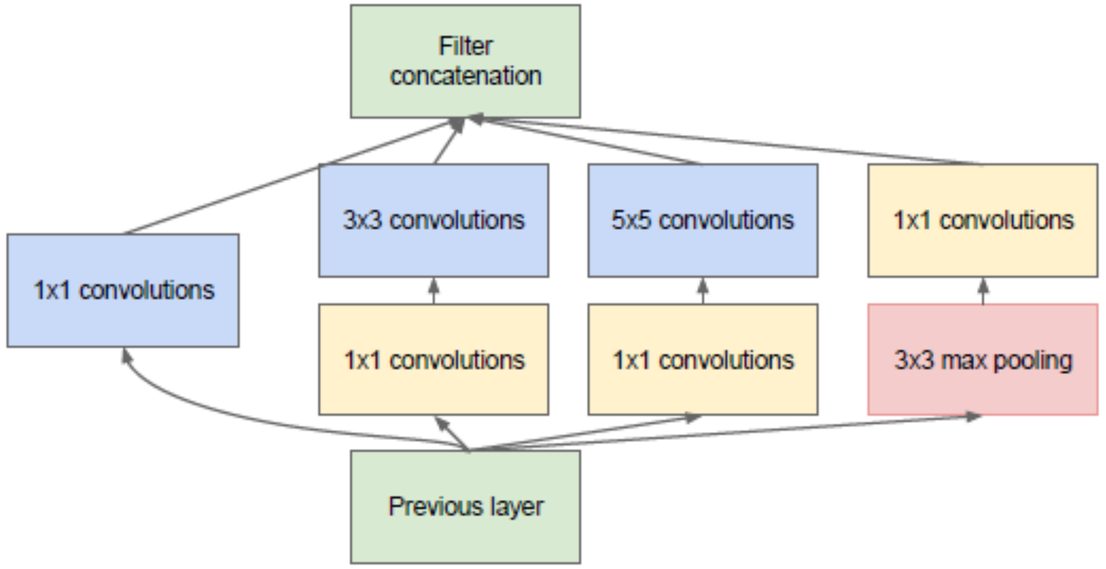
Number of operations for 1×1 = (14×14×16)×(1×1×480) = 1.5M
Number of operations for 5×5 = (14×14×48)×(5×5×16) = 3.8M
Total number of operations = 1.5M + 3.8M = 5.3M

อ้างอิง https://medium.com/coinmonks/paper-review-of-googlenet-inception-v1-winner-of-ilsvlc-2014-image-classification-c2b3565a64e7

GoogLeNet (aka Inception) Architecture



(a) Inception module, naïve version

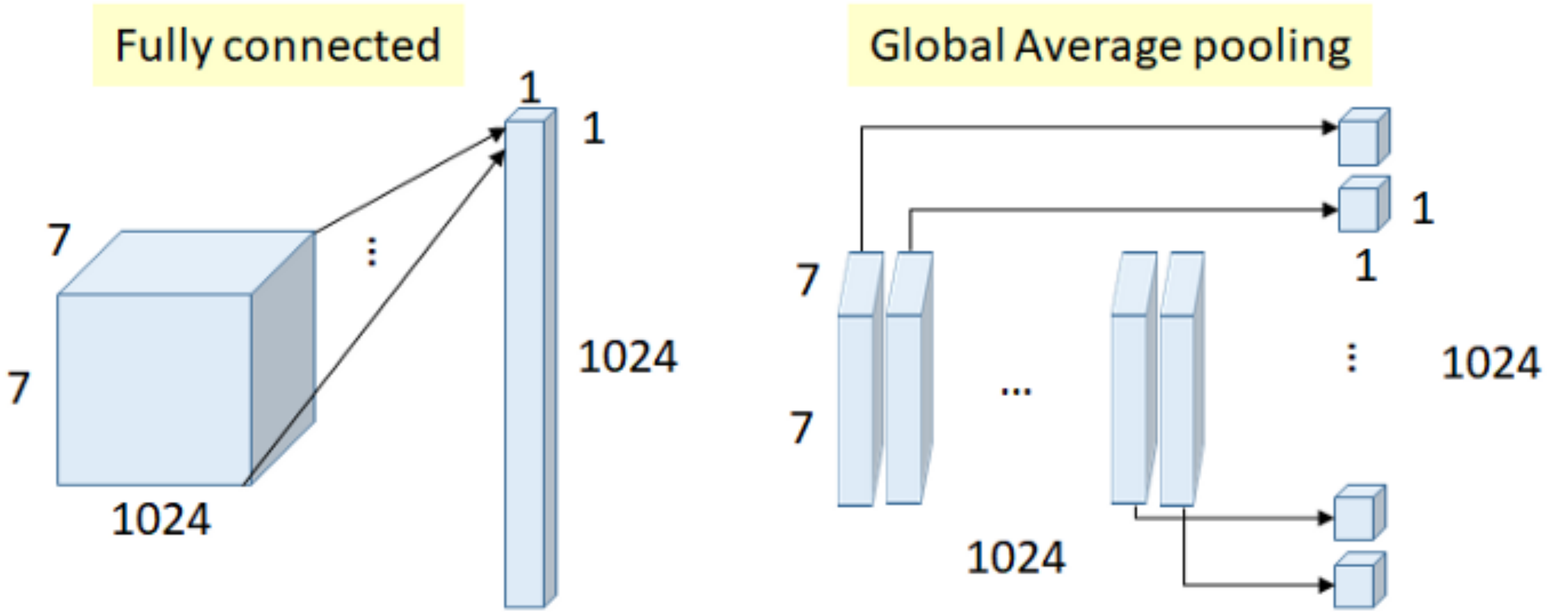


(b) Inception module with dimensionality reduction

โมดูล inception ได้รับการออกแบบเพื่อให้แบบจำลองสามารถสกัดแผนที่คุณลักษณะได้จากตัวกรองแบบคอนโวลูชั่นที่ขนาดที่หลากหลายได้ โดยอาศัย 1×1 conv, 3×3 conv, 5×5 conv, และ 3×3 max pooling นำผลลัพธ์ที่ได้มาต่อกัน แต่เพื่อลดขนาดของแผนที่คุณลักษณะลงและลดการใช้ทรัพยากรสำหรับการคำนวณลง ตัวดำเนินการ 1×1 convolution จึงถูกนำมาใช้งานร่วมด้วย

อ้างอิง https://medium.com/coinmonks/paper-review-of-googlenet-inception-v1-winner-of-ilsvlc-2014-image-classification-c2b3565a64e7

GoogLeNet (aka Inception) Architecture

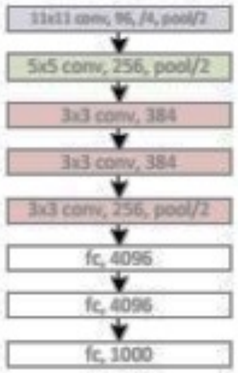


Global average pooling ได้ถูกนำมาใช้งานช่วงท้ายของแบบจำลอง (ก่อน Output Layer) เพื่อลดแผนที่คุณลักษณะลงทำให้แบบจำลองรองรับภาพที่มีขนาดแตกต่างกันได้ อีกทั้งยัง ลดการใช้ทรัพยากรสำหรับการคำนวณลง โดยการหาค่าเฉลี่ยของแผนที่คุณลักษณะขนาด 7×7 ให้เหลือขนาด 1×1

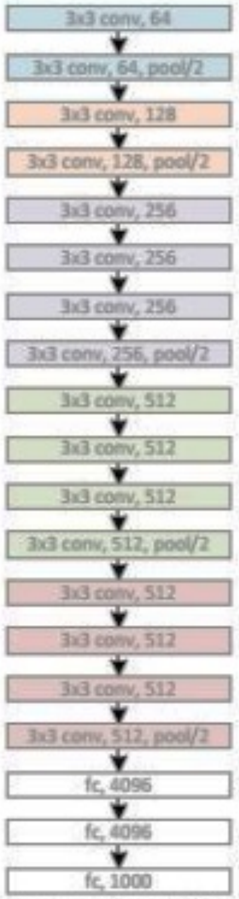
อ้างอิง https://medium.com/coinmonks/paper-review-of-googlenet-inception-v1-winner-of-ilsvlc-2014-image-classification-c2b3565a64e7

เปรียบเทียบสถาปัตยกรรม AlexNet, VGG และ GoogLeNet

AlexNet, 8 layers
(ILSVRC 2012)



VGG, 19 layers
(ILSVRC 2014)

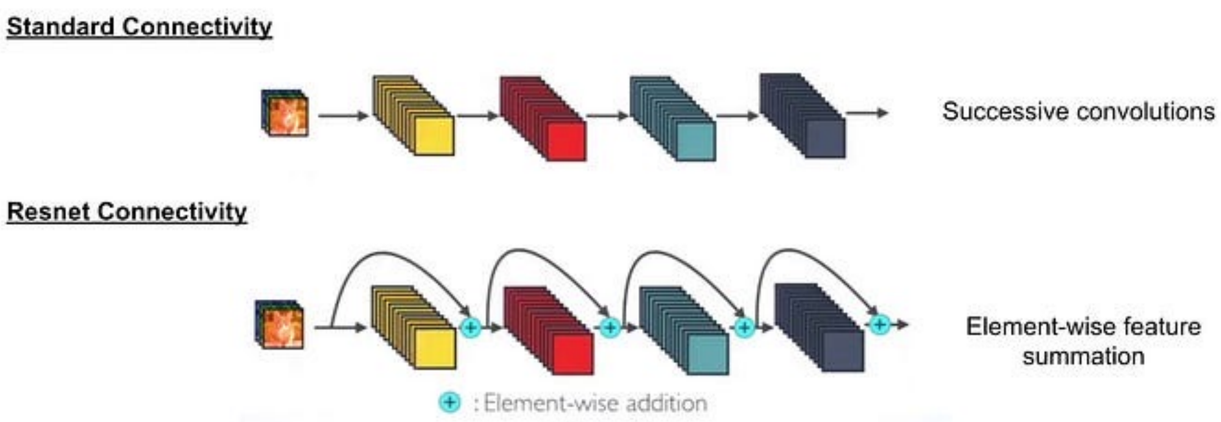


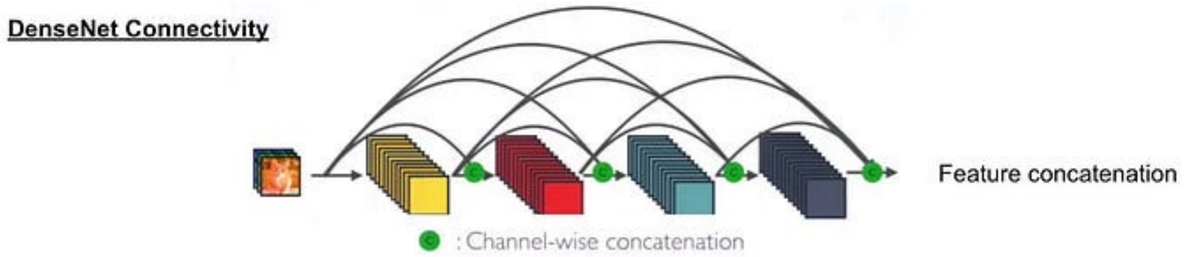
GoogleNet, 22 layers
(ILSVRC 2014)



Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". arXiv 2015.

รูปแบบการเชื่อมต่อระหว่างชั้นใน CNN





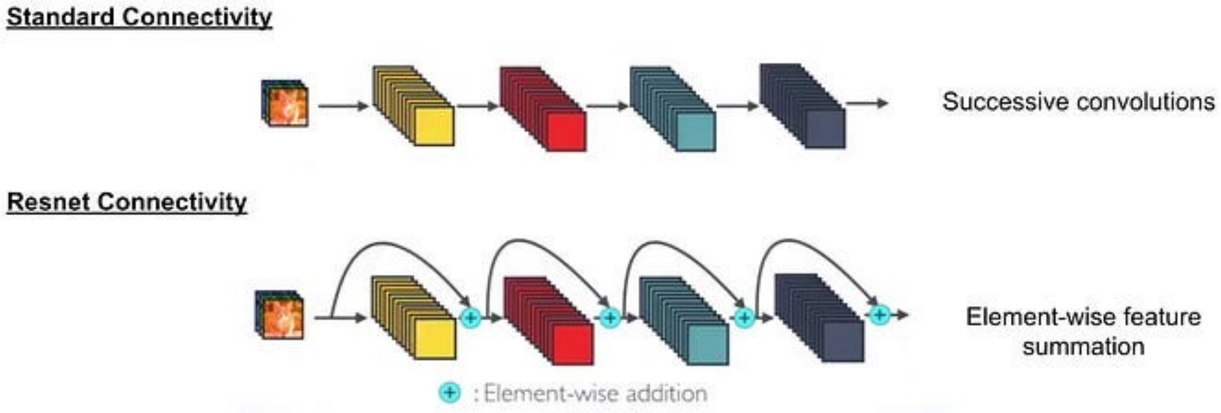
จากภาพได้แสดงให้เห็นถึงรูปแบบการเชื่อมต่อของเลเยอร์ต่างๆ ในโครงข่ายประสาทเทียมแบบคอนโวลูชัน 3 รูปแบบ ได้แก่ Standard Connectivity, ResNet Connectivity และ DenseNet Connectivity ดังนี้

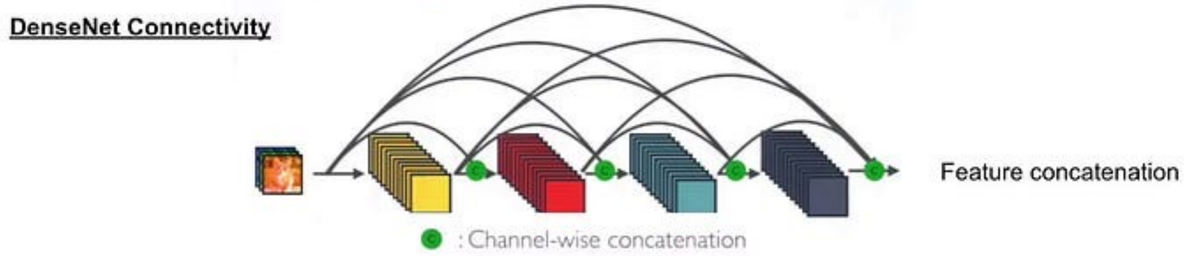
- **การเชื่อมต่อมาตรฐาน (Standard Connectivity)**

ลักษณะการเชื่อมต่อในโครงข่ายประสาทเทียมแบบดั้งเดิม การคอนโวลูชันในแต่ละเลเยอร์จะถูกดำเนินการต่อเนื่องกัน (successive convolutions) โดยผลลัพธ์จากเลเยอร์หนึ่งจะถูกส่งตรงไปยังเลเยอร์ถัดไป ข้อดีของการเชื่อมต่อแบบนี้ คือ โครงสร้างง่ายและตรงไปตรงมา แต่อาจก่อให้เกิดปัญหา Gradient Vanishing หรือ Gradient Exploding ในโครงข่ายที่มีความลึกมาก

อ้างอิง https://cvinvolution.medium.com/why-isnt-densenet-adopted-as-extensive-as-resnet-1bee84101160

รูปแบบการเชื่อมต่อระหว่างชั้นใน CNN





- **การเชื่อมต่อแบบ ResNet Connectivity**

ใช้เทคนิคการเชื่อมต่อแบบ Residual Connection หรือ Skip Connection โดยจะมีการเพิ่มการเชื่อมต่อแบบลัดระหว่างเลเยอร์ ทำให้ข้อมูลสามารถไหลข้ามเลเยอร์ไปได้ ผลลัพธ์จากเลเยอร์ที่ข้ามจะถูกนำมาบวกกันแบบ element-wise addition กับเลเยอร์ที่ตามมา ข้อดีของการเชื่อมต่อแบบนี้ คือ ช่วยลดปัญหา Gradient Vanishing ทำให้สามารถสร้างโครงข่ายที่มีความลึกมากขึ้นได้อย่างมีประสิทธิภาพ แต่โครงสร้างมีความซับซ้อนขึ้นเล็กน้อย

- **การเชื่อมต่อแบบ DenseNet Connectivity**

ใช้เทคนิคการเชื่อมต่อแบบ Dense Connection โดยแต่ละเลเยอร์จะได้รับข้อมูลจากทุกเลเยอร์ก่อนหน้านี้ ทำให้เกิดการรวมข้อมูล (Feature Concatenation) ของทุกเลเยอร์ก่อนหน้า ผลลัพธ์จากการเชื่อมต่อแบบนี้จะถูกนำมารวมกันแบบ channel-wise concatenation ข้อดีของการเชื่อมต่อแบบนี้ คือ เพิ่มการเรียนรู้ฟีเจอร์ที่มีประสิทธิภาพมากขึ้นเนื่องจากทุกเลเยอร์มีการรับข้อมูลจากทุกเลเยอร์ก่อนหน้านี้ แต่การเชื่อมต่อแบบนี้ต้องการหน่วยความจำมากขึ้นในการเก็บฟีเจอร์ที่เกิดขึ้นในทุกเลเยอร์

อ้างอิง https://cvinvolution.medium.com/why-isnt-densenet-adopted-as-extensive-as-resnet-1bee84101160

ResNet-18

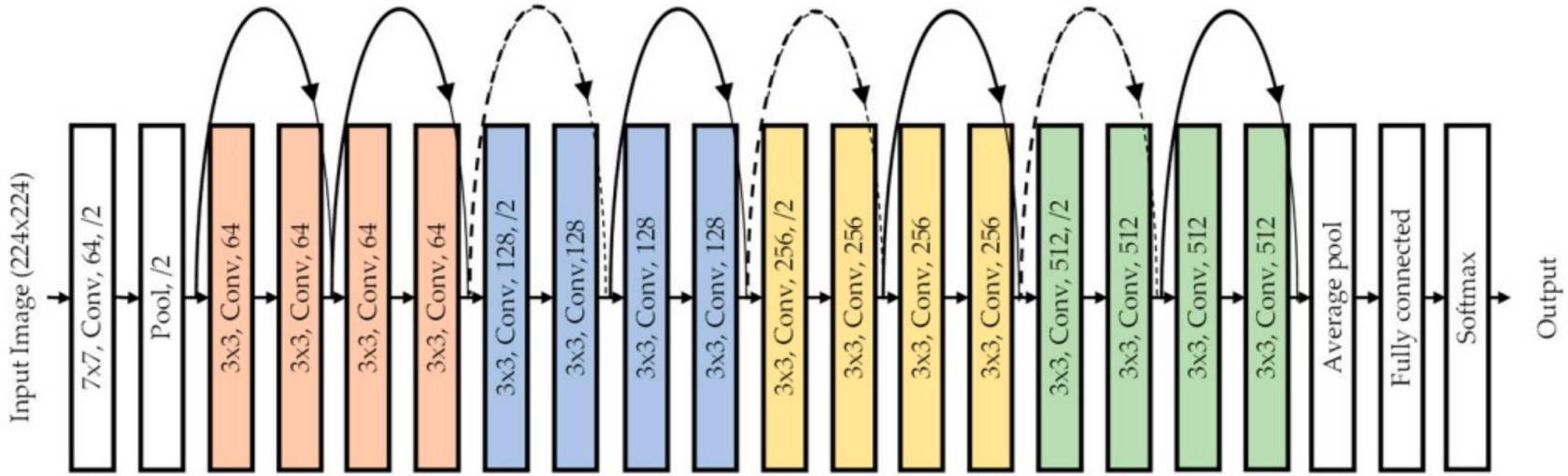
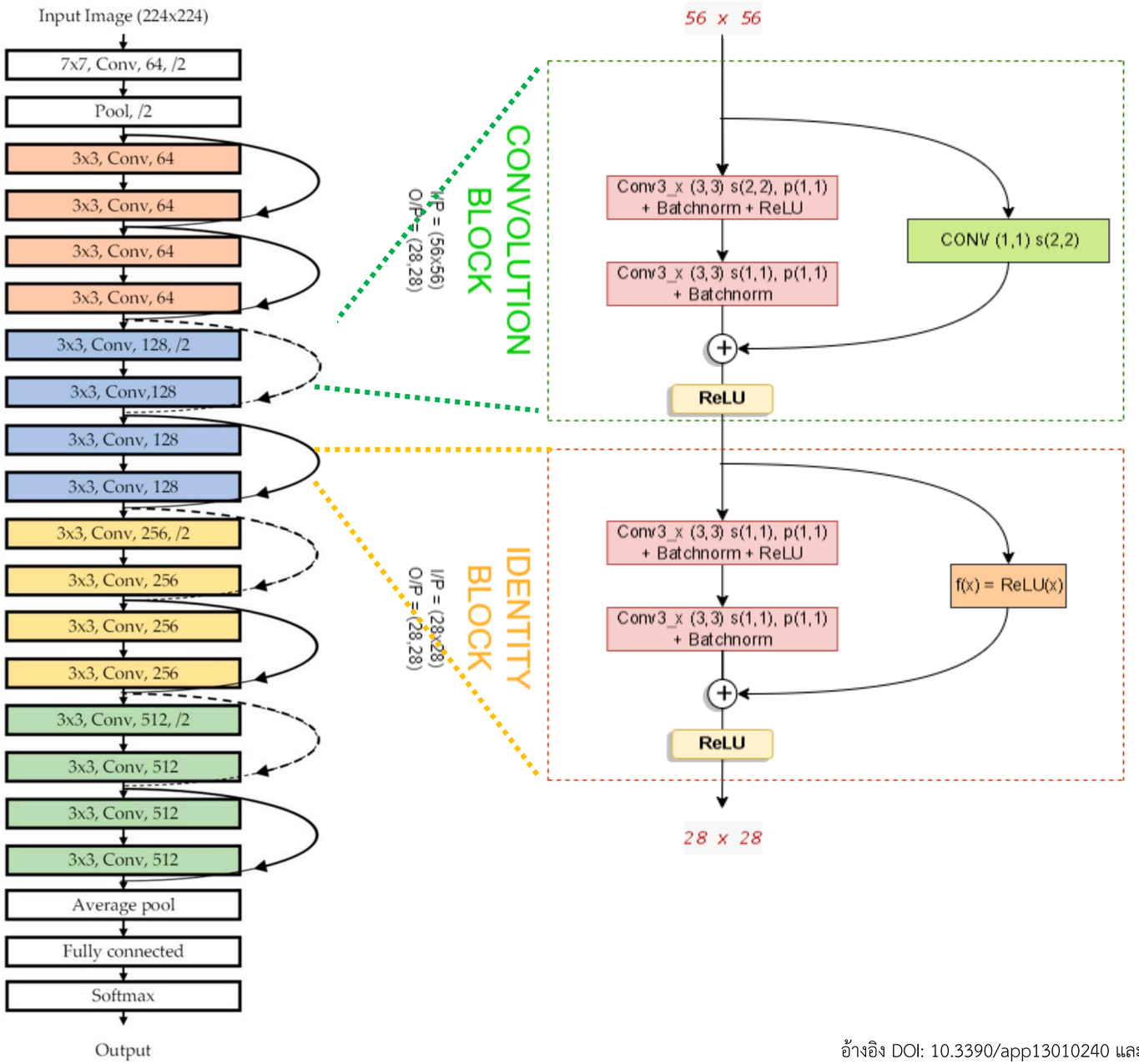


Figure 5. Structure of the Resnet-18 Model.

ResNet-18 เป็นส่วนหนึ่งของตระกูล Residual Networks (ResNets) ที่ถูกพัฒนาโดย Microsoft Research เพื่อแก้ปัญหาที่เกิดขึ้นกับการฝึกสอนโครงข่ายประสาทเทียมที่มีความลึกมาก (Deep Neural Networks) โดยใช้เทคนิค Residual Learning ซึ่งช่วยให้สามารถสร้างโครงข่ายที่มีความลึกมากขึ้นได้อย่างมีประสิทธิภาพ โครงสร้างภายในประกอบไปด้วย (1) Convolution Block, (2) Identity Block และ (3) Skip Connections โดยที่ Convolution Block และ Identity Block คือ Residual Block ที่มี Connection ที่แตกต่างกัน เมื่อใดก็ตามที่ต้องปรับขนาดของผลลัพธ์เพื่อให้สามารถใช้การเชื่อมต่อด้วยหลักการ Skip Connections ได้ จำเป็นต้องใช้ตัวดำเนินการ Convolution มาช่วย ดังนั้น จึงเรียก Residual Block นั้นว่า Convolution Block ขณะที่ เมื่ออินพุตและเอาต์พุตมีขนาดเท่ากัน นักศึกษาต้องใช้การเชื่อมต่อแบบ Identity จึงเรียก Residual Block นั้นว่า Identity Block โดยที่ การเชื่อมต่อแบบลัด (skip connections) คือ การนำเอาผลลัพธ์จากเลเยอร์ก่อนหน้ามาบวกกับผลลัพธ์ของเลเยอร์ปัจจุบัน ซึ่งช่วยลดปัญหา Gradient Vanishing และ Gradient Exploding ทำให้การฝึกสอนโครงข่ายที่ลึกมากเป็นไปได้อย่างมีประสิทธิภาพ

อ้างอิง DOI: 10.3390/app13010240 และ https://medium.com/analytics-vidhya/resnet-understand-and-implement-from-scratch-d0eb9725e0db

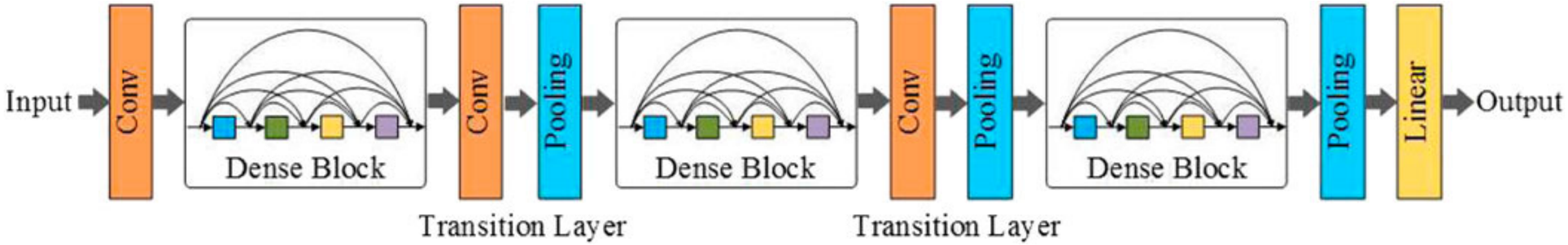


อินพุตของ Convolution Block เป็นภาพที่มีขนาด (56x56) และมี 64 channel ชั้น Convolution แรกจะเปลี่ยนภาพนี้เป็นภาพขนาด (28x28) โดยมี 128 channel โดยใช้ตัวกรองขนาด 3x3, stride 2x2 และ padding 1x1 พร้อมกับใช้ฟังก์ชัน ReLU ต่อจากนั้น ชั้น Convolution ที่สองจะรับภาพนี้เป็นอินพุตและให้ผลลัพธ์เป็นภาพที่มีขนาดเดียวกัน (28x28x128) ในการที่จะใช้การเชื่อมต่อแบบ Skip Connection ต้องนำผลลัพธ์ของบล็อก Residual Block ชั้นก่อนหน้า ที่มีขนาด (56x56x64) มารวมกับผลลัพธ์ของ Convolution ที่สองที่มีขนาด (28x28x128) เพื่อทำเช่นนี้ จำเป็นต้องแปลงผลลัพธ์ของ Residual Block ชั้นก่อนหน้า ให้มีขนาดเป็น (28x28x128) ซึ่งทำได้โดยการใช้ Convolution อีกชั้นหนึ่งด้วยตัวกรองขนาด 1x1 และ stride 2x2 โดยมีช่องทางอินพุตเป็น 64 และช่องทางเอาต์พุตเป็น 128 หรือสามารถกล่าวได้ว่า **บล็อก {Conv(1,1) s(2,2)}** มีหน้าที่ในการแปลงผลลัพธ์จากบล็อกหนึ่งโดยใช้การทำ Convolution เพื่อให้สามารถใช้รวมกับผลลัพธ์ของบล็อก Convolution อื่นได้อย่างมีประสิทธิภาพ หลังจากการรวมเสร็จสิ้น จะมีการใช้ฟังก์ชัน Activation (ReLU) กับผลลัพธ์และส่งไปยัง Identity Block

อินพุตและเอาต์พุตของ Identity Block มีขนาดเท่ากัน ทำให้การเชื่อมต่อแบบ Skip connection ไม่จำเป็นต้องแปลงผลลัพธ์ของ Convolution Block ดังนั้น การใช้ Residual connection ต้องนำผลลัพธ์ของ Convolution Block ก่อนหน้ามารวมกับผลลัพธ์ของชั้น Identity Block จากนั้น นำไปผ่าน ReLU กับผลลัพธ์ที่ได้

อ้างอิง DOI: 10.3390/app13010240 และ https://medium.com/analytics-vidhya/resnet-understand-and-implement-from-scratch-d0eb9725e0db

DenseNet



Densely Connected Convolutional Networks (DenseNet) เป็นสถาปัตยกรรมโครงข่ายประสาทเทียมที่ถูกนำเสนอโดย Gao Huang และคณะ ในปี 2016 สถาปัตยกรรม DenseNet มีวัตถุประสงค์เพื่อแก้ปัญหาการเรียนรู้ในโครงข่ายที่มีขนาดลึกมาก โดยการใช้ประโยชน์จากข้อมูลเชิงลึกที่ได้จากทุกเลเยอร์ในโครงข่าย โครงสร้างของ DenseNet ประกอบด้วย 3 ส่วน

- **Dense Blocks**

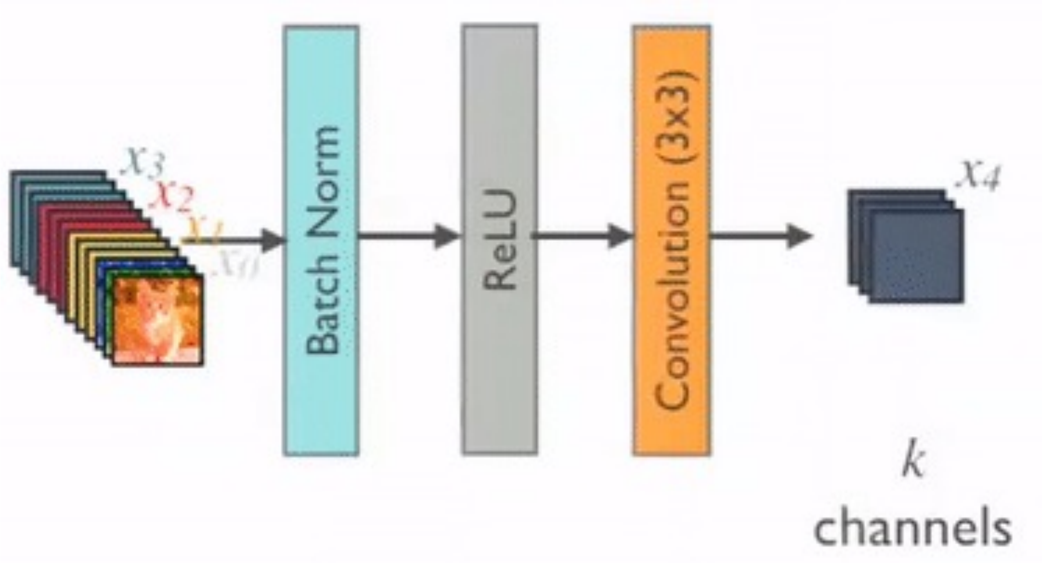
ใน DenseNet แต่ละเลเยอร์จะได้รับข้อมูลจากทุกเลเยอร์ก่อนหน้า ซึ่งต่างจากการเชื่อมต่อแบบลำดับ (Sequential) ทั่วไปด้วยการเชื่อมต่อแบบนี้ ทำให้ Feature map ที่เรียนรู้จากเลเยอร์ก่อนหน้าไม่สูญหาย แต่จะถูกนำมาใช้ในทุกเลเยอร์ภายใน Dense Blocks

- **Transition Layers**

คือ กลุ่มของเลเยอร์ที่เรียงลำดับ Batch Normalization, ReLU, 1x1 Convolution เพื่อลดจำนวน Channels จากนั้น ค่อยผ่าน 2x2 Average Pooling เพื่อลดขนาดของ Feature Map

อ้างอิง https://doi.org/10.3389/fenvs.2022.991173 และ https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

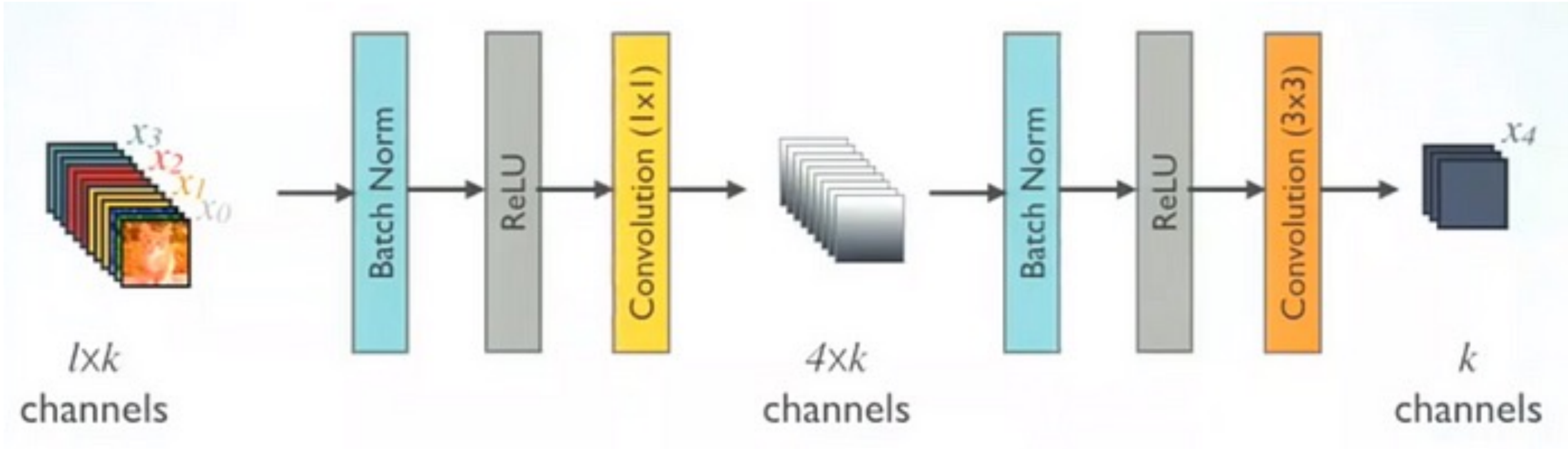
• Dense Blocks - Basic DenseNet Composition Layer



ในแต่ละเลเยอร์การประกอบจะมีการทำ Pre-Activation ซึ่งประกอบไปด้วย Batch Normalization (BN) และ ReLU ก่อนที่จะทำการ Convolution ขนาด 3×3 โดยมี Feature Map เอาต์พุตที่มี Channel จำนวน k ช่อง โดยก่อนที่จะทำการ Convolution จะมีการทำ Batch Normalization และการใช้ฟังก์ชันการกระตุ้น ReLU ก่อน ซึ่งช่วยให้การฝึกสอนโมเดลมีประสิทธิภาพและเสถียรมากขึ้น และการดำเนินการ 3x3 Convolution ทำให้ Feature Map ของข้อมูลเปลี่ยนไปตามจำนวน Channel ที่กำหนด

อ้างอิง https://doi.org/10.3389/fenvs.2022.991173 และ https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

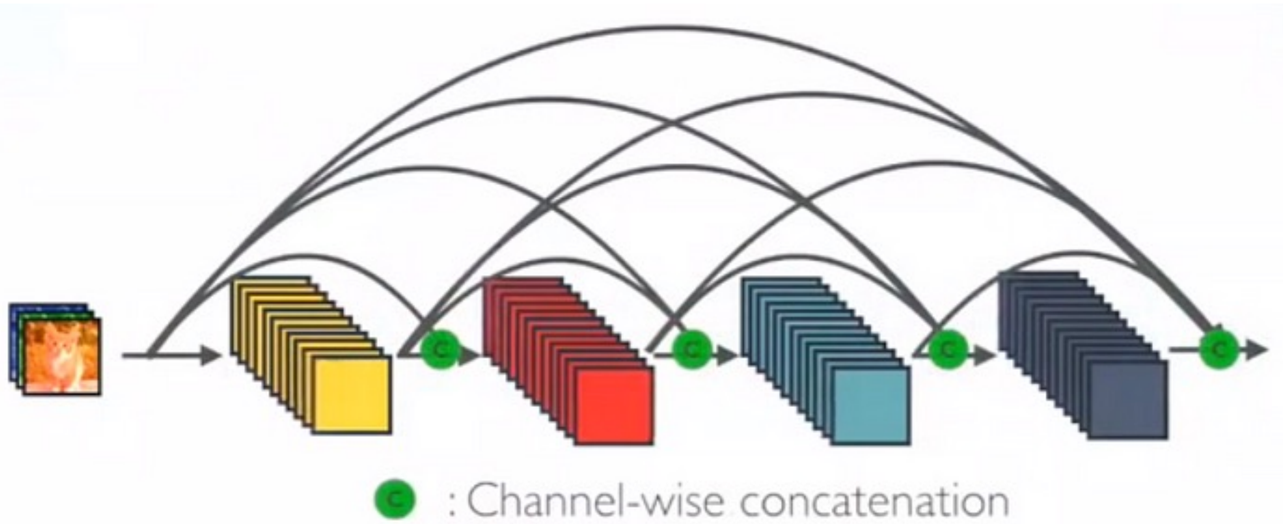
• Dense Blocks - DenseNet Bottleneck Layers



ได้รับการออกแบบมาเพื่อลดความซับซ้อนและขนาดของโมเดล จะประกอบไปด้วย Batch Normalization, ReLU และ 1x1 Conv จากนั้น จะดำเนินการ Batch Normalization, ReLU และ 3x3 Conv ซึ่งการทำ Convolution ขนาด 1x1 เพื่อลดจำนวน Channels ของ Feature Map ก่อน จากนั้น จึงทำ Convolution ขนาด 3x3 เพื่อสกัด Feature Map ใหม่ สิ่งนี้ช่วยลดจำนวนพารามิเตอร์และการคำนวณ ทำให้โมเดลมีขนาดเล็กลงและมีประสิทธิภาพมากขึ้น

อ้างอิง https://doi.org/10.3389/fenvs.2022.991173 และ https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

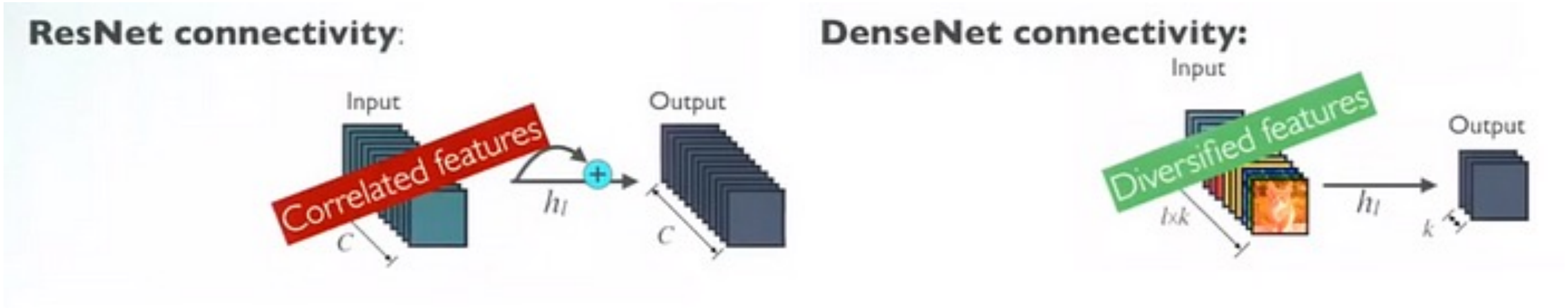
• การเชื่อมต่อแบบ DenseNet Connectivity



ภายใน Dense Block ทุกเลเยอร์จะสร้าง Feature Map ที่มีขนาดเชิงพื้นที่เท่ากัน ทำให้สามารถรวม Feature Map ที่ได้จากหลายเลเยอร์เข้าด้วยกันได้อย่างง่ายดายโดยใช้การ Channel-wise Concatenation สิ่งนี้สามารถช่วยให้ Feature Map มีหลากหลายมากขึ้น (More Diversified Features) และรักษาฟีเจอร์ที่มีความซับซ้อนต่ำ (Maintains Low Complexity Features) นอกจากนี้ การเชื่อมต่อแบบ DenseNet Connectivity ยังถือว่าเป็นการทำ Feature Reuse เนื่องจากช่วยให้ Feature Map ที่ได้จากเลเยอร์ก่อนหน้านี้ไม่สูญหาย แต่ถูกนำมาใช้ซ้ำ ทำให้การเรียนรู้มีประสิทธิภาพและลดจำนวนพารามิเตอร์ที่ต้องการใช้

อ้างอิง https://doi.org/10.3389/fenvs.2022.991173 และ https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

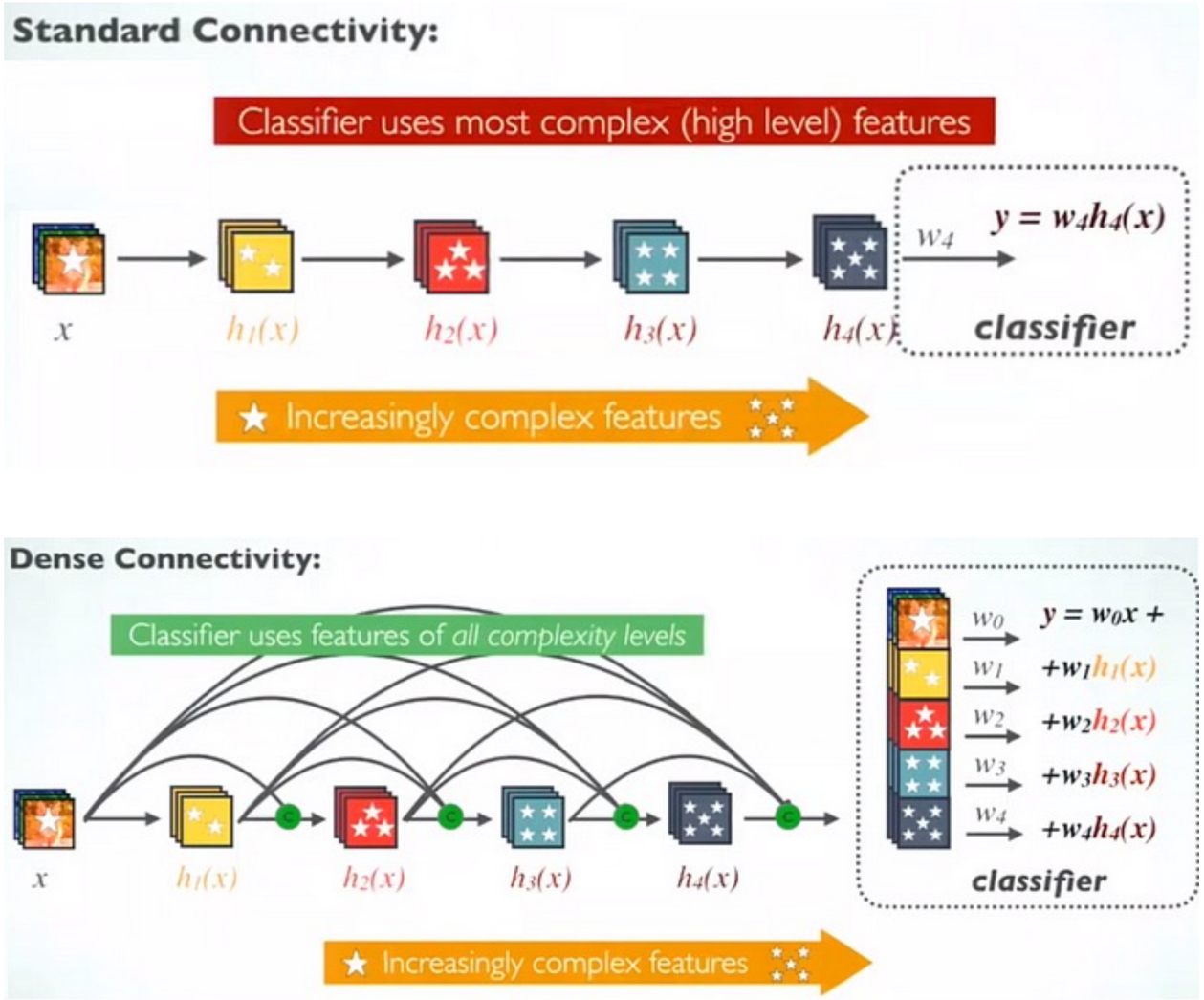
• การเชื่อมต่อแบบ DenseNet Connectivity



ช่วยให้ Feature Map มีหลากหลายมากขึ้น (More Diversified Features) เนื่องจากแต่ละเลเยอร์ใน DenseNet ได้รับข้อมูลจากทุกเลเยอร์ก่อนหน้านี้เป็นอินพุต ฟีเจอร์ที่ได้จึงมีความหลากหลายมากขึ้นและมักจะมีรูปแบบที่สมบูรณ์มากขึ้น

อ้างอิง https://doi.org/10.3389/fenvs.2022.991173 และ https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

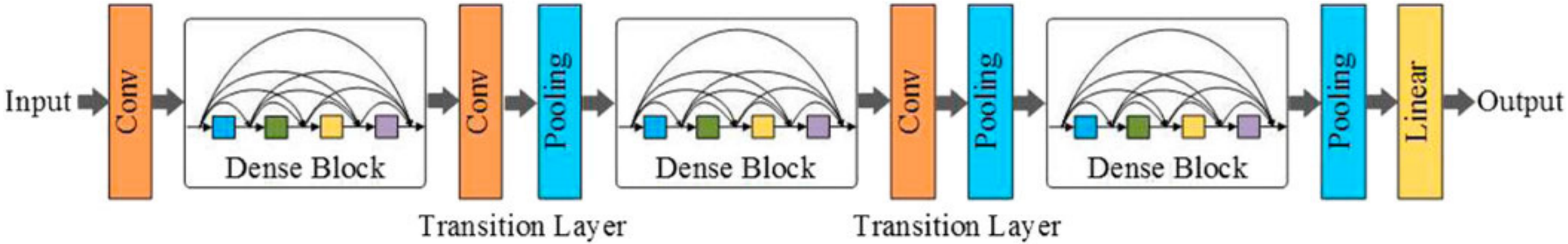
• การเชื่อมต่อแบบ DenseNet Connectivity



การรักษาฟีเจอร์ที่มีความซับซ้อนต่ำ ใน DenseNet ส่วนของตัวจำแนกประเภทใช้ฟีเจอร์ที่มีความซับซ้อนทุกระดับ ทำให้การตัดสินใจมีความราบรื่นมากขึ้น และเป็นผลดีในการเรียนรู้จากข้อมูลที่มีจำนวนจำกัด จึงสามารถทำงานได้ดีแม้มีข้อมูลการฝึกสอนไม่เพียงพอ

อ้างอิง https://doi.org/10.3389/fenvs.2022.991173 และ https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

• Transition Layers

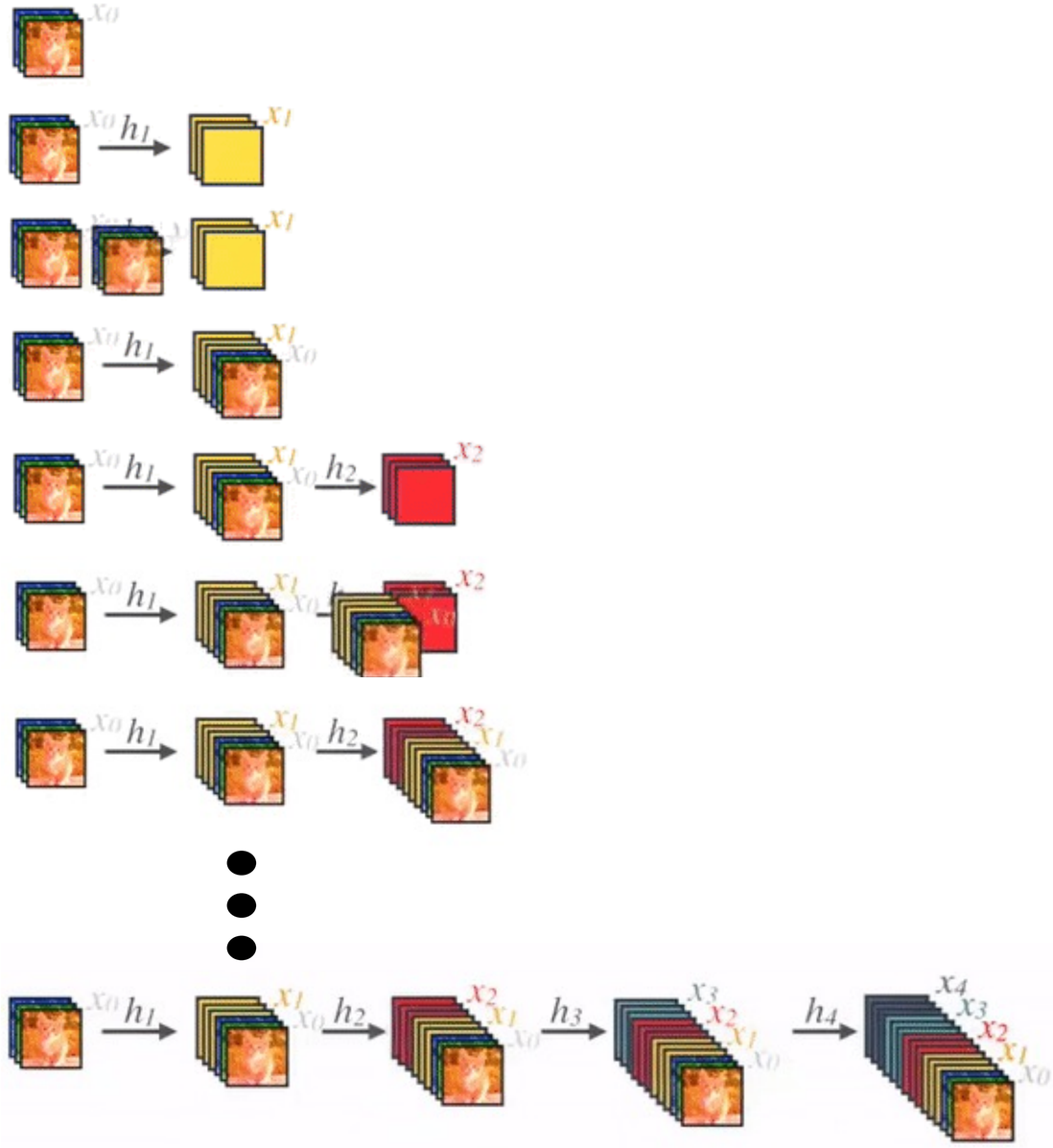


Transition Layers ประกอบด้วย 1×1 Conv ตามด้วย 2×2 Average Pooling ใช้เป็นเลเยอร์ระหว่างกลางระหว่างสอง Dense Blocks ที่ติดกัน โดยที่ การใช้ Convolution ขนาด 1x1 เป็นการลดจำนวน Channels ของฟีเจอร์แมปลง ซึ่งช่วยลดจำนวนพารามิเตอร์และการคำนวณ จากนั้น ใช้ Average Pooling ขนาด 2x2 เพื่อลดขนาดของ Feature Map (Downsampling) ทำให้ Feature Map ที่ได้จาก Dense Block หนึ่งสามารถเชื่อมต่อกับ Dense Block ถัดไปได้อย่างราบรื่น ทำให้ขนาดของ Feature Map จะเหมือนกันภายใน Dense Block ดังนั้น สามารถเชื่อมต่อกันได้อย่างง่ายดาย

อ้างอิง https://doi.org/10.3389/fenvs.2022.991173 และ https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

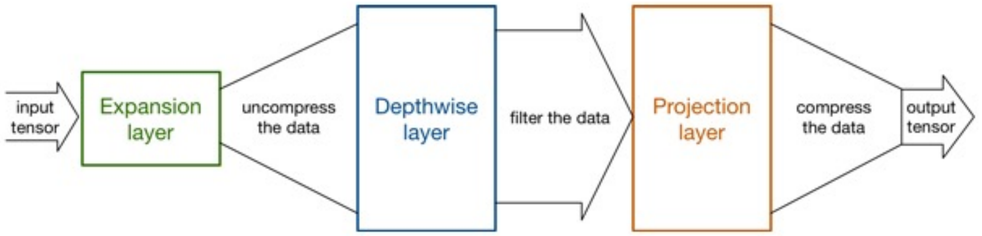
Growth Rate

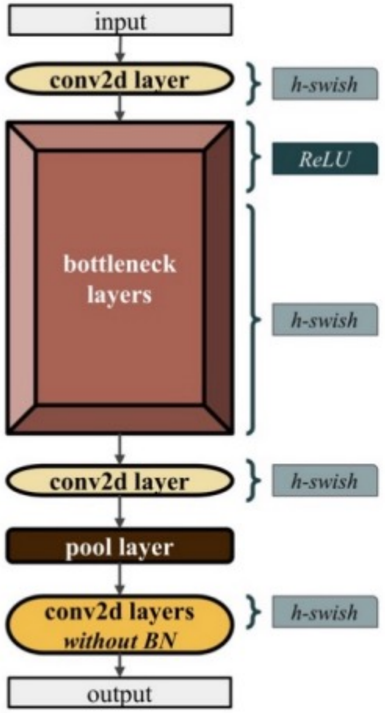
คือ จำนวน Feature Map ที่แต่ละเลเยอร์เพิ่มเข้ามาใน Dense Block ซึ่งจะกำหนดให้แต่ละเลเยอร์เพิ่มจำนวน Feature Map เข้ามาเรื่อย ๆ



อ้างอิง https://doi.org/10.3389/fenvs.2022.991173 และ https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803

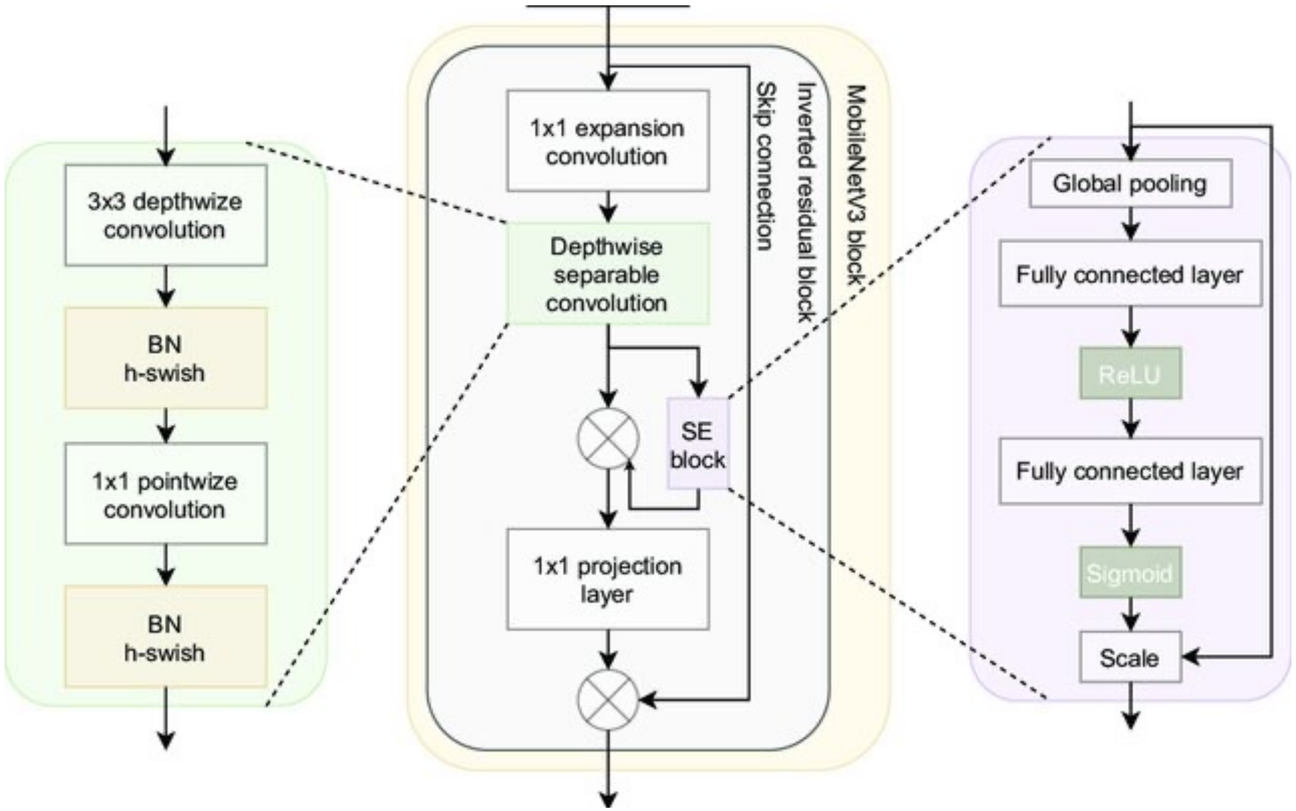
MobileNetV3 Architecture

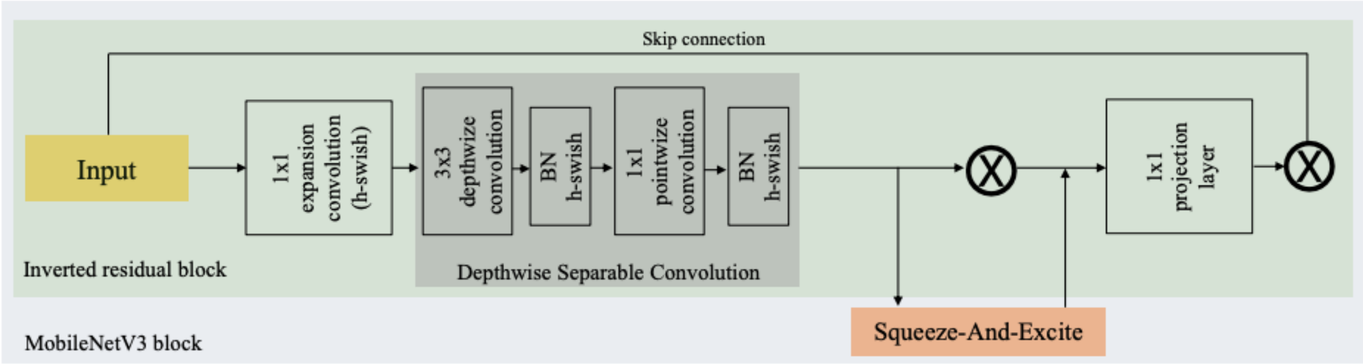




MobileNetV3 block (bottleneck block) ได้รับการออกแบบมาเพื่อเพิ่มประสิทธิภาพโดยรวมของแบบจำลองและลดการใช้ทรัพยากรหน่วยความจำ

อ้างอิง S. Qian, C. Ning and Y. Hu, "MobileNetV3 for Image Classification," *2021 IEEE 2nd International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE)*, Nanchang, China, 2021, pp. 490-497, DOI: 10.3390/e23111383 และ https://towardsdatascience.com/everything-you-need-to-know-about-mobilenetv3-and-its-comparison-with-previous-versions-a5d5e5a6eeaa

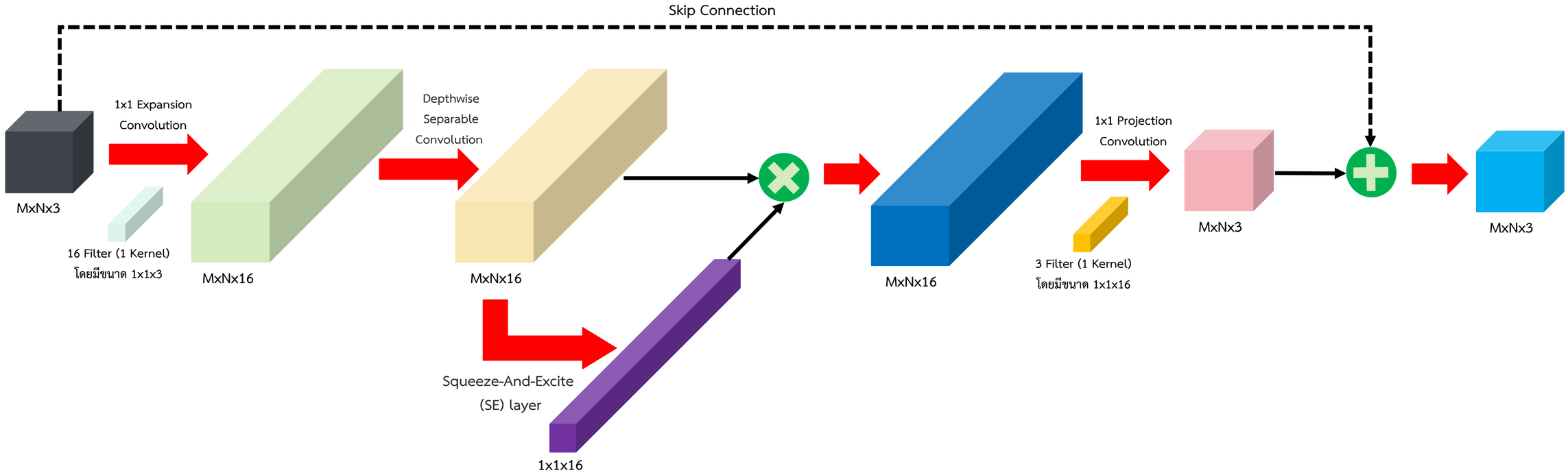




โครงสร้างภายใน MobileNetV3 block ประกอบด้วย 2 องค์ประกอบหลัก ได้แก่

- Inverted residual block
  - 1x1 Expansion Convolution
  - Depthwise Separable Convolution
  - 1x1 projection layer
  - Skip connection
- Squeeze-And-Excite (SE) block

อ้างอิง https://doi.org/10.3390/diagnostics13091579 และ https://doi.org/10.3390/e23111383



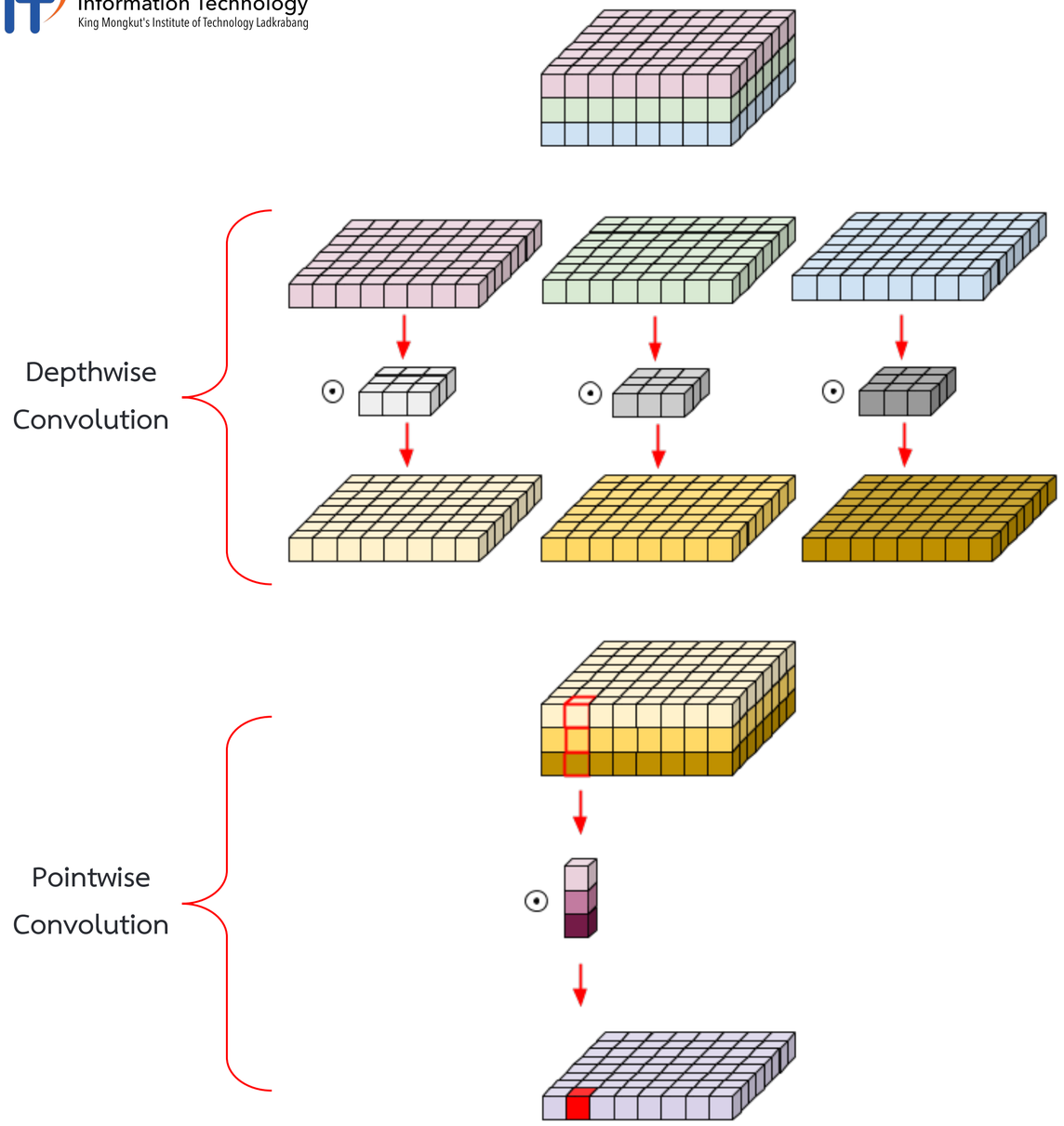
- **ตัวดำเนินการ 1x1 Expansion Convolution**

เป็นส่วนที่ขยายจำนวน channels ของข้อมูลอินพุตก่อนที่จะทำการ Depthwise Convolution โดยเริ่มจากการขยายจำนวนช่องทางของข้อมูลอินพุตโดยการใช้ 1x1 Convolution เพิ่มมิติของข้อมูลเพื่อให้สามารถจับลักษณะเชิงพื้นที่ได้มากขึ้นในขั้นตอน Depthwise Convolution อีกทั้ง ยังเป็นการช่วยให้โมเดลสามารถเรียนรู้ลักษณะที่ซับซ้อนมากขึ้นก่อนที่จะทำการลดมิติในขั้นตอนถัดไป

- **ตัวดำเนินการ 1x1 Projection Convolution**

เป็นส่วนที่ลดจำนวน channels ของข้อมูลเอาต์พุตหลังจากที่ทำการ Depthwise Convolution โดยเริ่มจากการลดจำนวนช่องทางของข้อมูลโดยการใช้ 1x1 Convolution ทำให้ข้อมูลที่ได้มีขนาดเล็กลงและง่ายต่อการประมวลผลในขั้นตอนถัดไป สิ่งนี้ทำให้ช่วยลดจำนวนพารามิเตอร์และการคำนวณที่ใช้ในโมเดล

อ้างอิง https://global.discourse-cdn.com/dlai/original/3X/0/1/017036506b0d47805b188a5ced7fc41c888bd2b7.jpeg

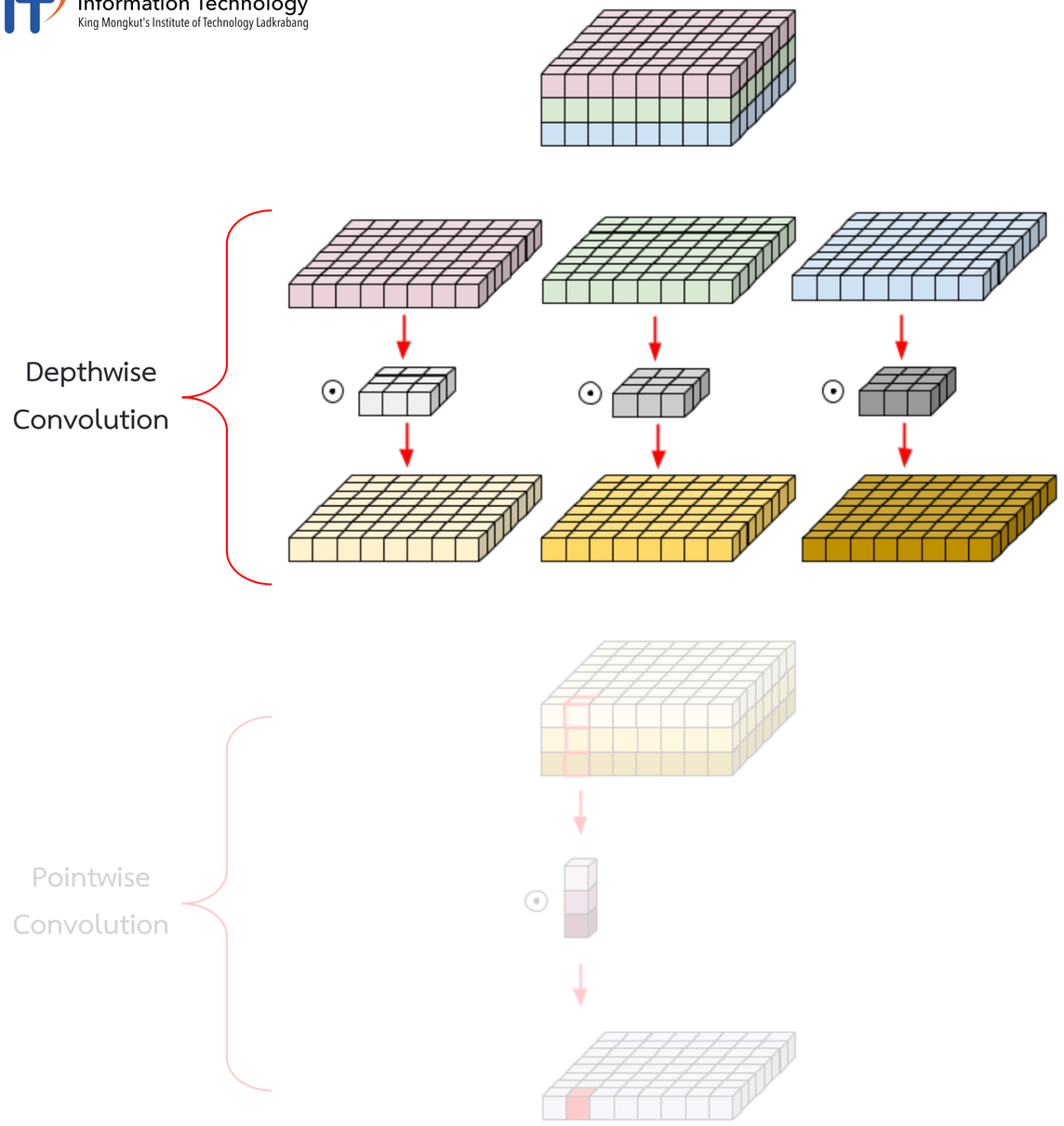


ตัวอย่างการทำงานของ Depthwise Sqparable Convolution

Depthwise Separable Convolution ได้รับการออกแบบมาทำงานแทนที Convolution Layer แบบดังเดิมเพื่อลดขนาดของแบบจำลองลง ซึ่งประกอบไปด้วย 2 ส่วนย่อย ได้แก่

- **3x3 Depthwise Convolution, BN และ H-swish**
- **1x1 Pointwise Convolution, BN และ H-swish**

อ้างอิง https://medium.com/@zurister/depth-wise-convolution-and-depth-wise-separable-convolution-37346565d4ec



ตัวอย่างการทำงานของ Depthwise Sqparable Convolution

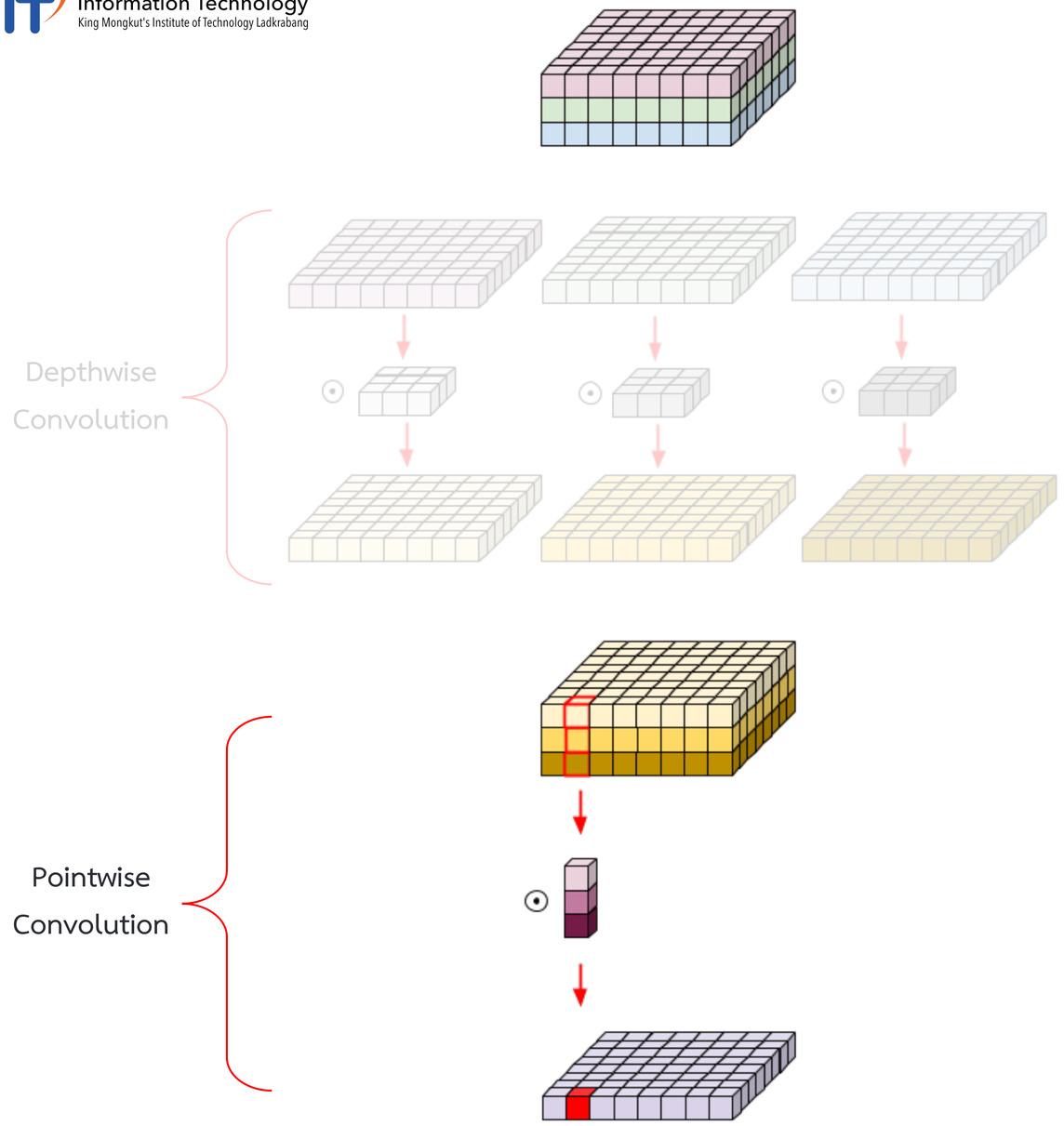
Depthwise Separable Convolution ได้รับการออกแบบมาทำงานแทนที Convolution Layer แบบดังเดิมเพื่อลดขนาดของแบบจำลองลง ซึ่งประกอบไปด้วย 2 ส่วนย่อย ได้แก่

- **3x3 Depthwise Convolution, BN และ H-swish**

ในการคอนโวลูชันแบบเดิม ฟิลเตอร์จะถูกนำไปใช้กับทุก channel ของข้อมูลอินพุตพร้อมกัน แต่ใน Depthwise Convolution ฟิลเตอร์คอนโวลูชันจะถูกนำไปใช้กับแต่ละ channel ของข้อมูลอินพุตแยกกัน ฟิลเตอร์แต่ละตัวจะทำงานกับช่องทางเดียวเท่านั้น โดย Depthwise Convolution ได้รับการออกแบบมาเพื่อลดจำนวนพารามิเตอร์และการคำนวณลง เนื่องจากการคอนโวลูชันแต่ละฟิลเตอร์จะทำงานกับช่องทางเดียว จำนวนการคำนวณและพารามิเตอร์ที่ใช้จะน้อยลงเมื่อเทียบกับการคอนโวลูชันแบบเดิม และส่งผลทำให้การฝึกสอนโมเดลจะเร็วขึ้นและใช้ทรัพยากรน้อยลง

- **1x1 Pointwise Convolution, BN และ H-swish**

อ้างอิง https://medium.com/@zurister/depth-wise-convolution-and-depth-wise-separable-convolution-37346565d4ec



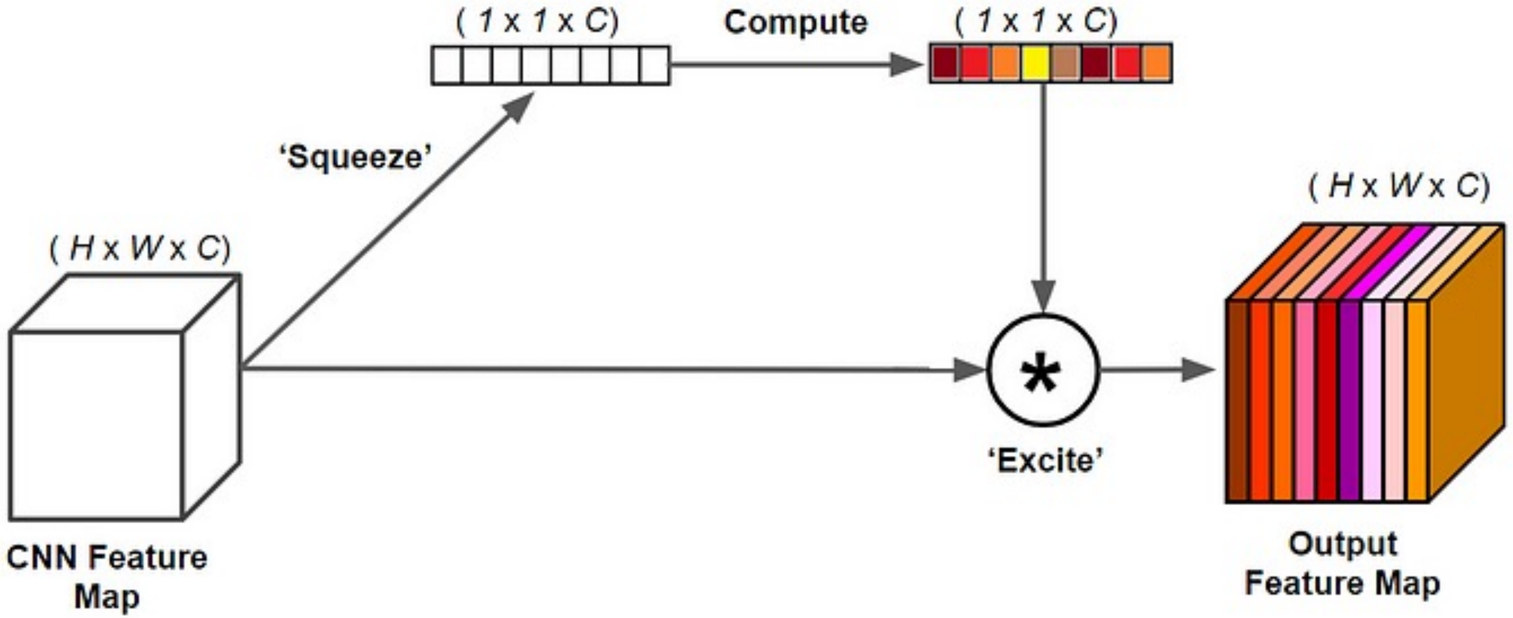
ตัวอย่างการทำงานของ Depthwise Sqparable Convolution

Depthwise Separable Convolution ได้รับการออกแบบมาทำงานแทนที Convolution Layer แบบดังเดิมเพื่อลดขนาดของแบบจำลองลง ซึ่งประกอบไปด้วย 2 ส่วนย่อย ได้แก่

- **3x3 Depthwise Convolution, BN และ H-swish**
- **1x1 Pointwise Convolution, BN และ H-swish**

ฟิลเตอร์ที่ใช้ในการคอนโวลูชันนี้มีขนาด 1x1 ซึ่งหมายความว่ามันจะทำการคอนโวลูชันกับค่าพิกเซลเดียวในแต่ละ channel ของข้อมูลอินพุต ส่งผลทำให้เกิดการรวมข้อมูลจากหลาย channel ณ พิกเซลนั้น เพื่อสร้าง channel ใหม่ในข้อมูลเอาต์พุต (Feature Map) ซึ่งช่วยในการเปลี่ยนแปลงจำนวนช่องทางของข้อมูลอินพุต (channel depth) ซึ่ง 1x1 Pointwise Convolution ได้รับการออกแบบมาเพื่อช่วยเปลี่ยนแปลงจำนวน channel ของข้อมูลอินพุต กล่าวคือ สามารถเพิ่มหรือลดจำนวน channel ของข้อมูลอินพุตได้ตามต้องการ ซึ่งเป็นประโยชน์ในการปรับขนาดของโมเดลให้มีประสิทธิภาพสูงสุด อีกทั้ง ยังถือว่าเป็นการผสมผสานข้อมูลจากหลาย channel เข้าด้วยกัน เพื่อให้ได้ข้อมูลเชิงลึกที่มากขึ้น

อ้างอิง https://medium.com/@zurister/depth-wise-convolution-and-depth-wise-separable-convolution-37346565d4ec



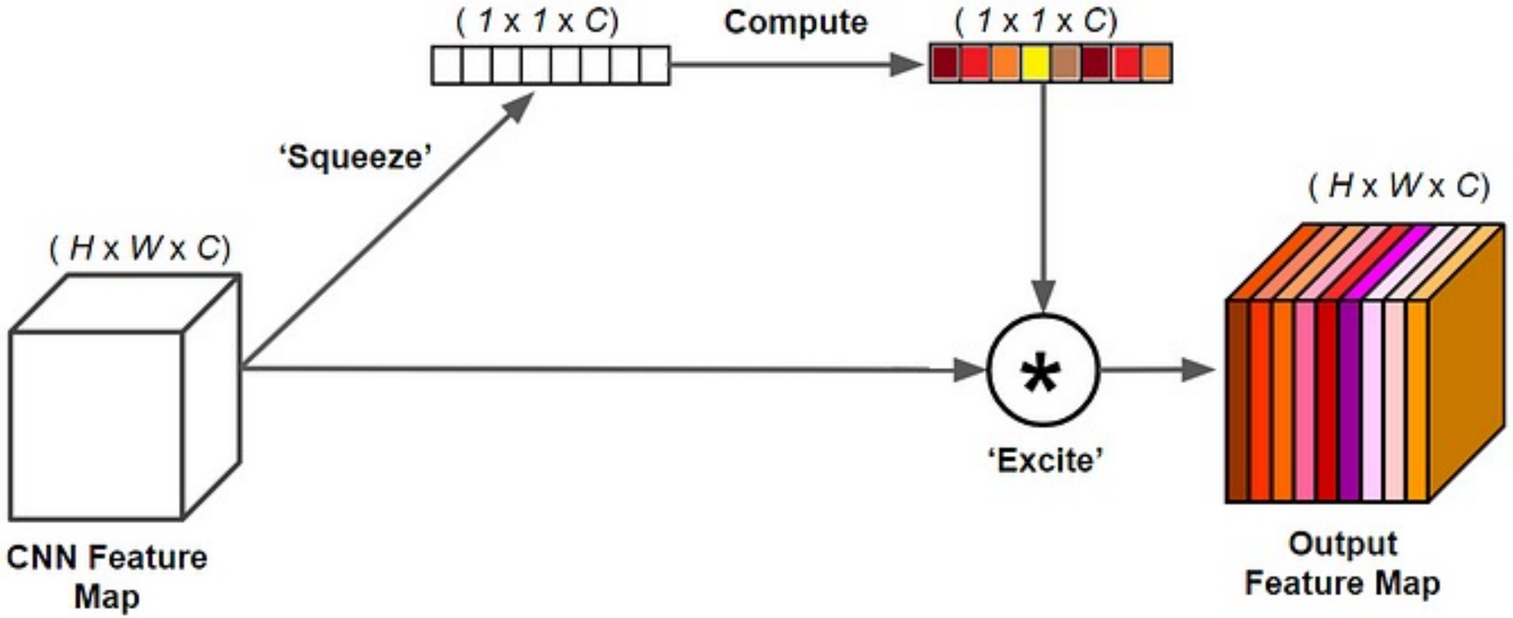
Squeeze-and-Excitation (SE) block

Squeeze-and-Excite (SE) Block เป็นส่วนประกอบที่ใช้ในสถาปัตยกรรม MobileNetV3 ซึ่งช่วยเพิ่มความสามารถในการรับรู้ของโมเดลโดยการสร้างความสัมพันธ์ระหว่างช่องทาง (channel) ในโครงข่ายประสาทเทียมแบบคอนโวลูชัน บล็อกนี้ถูกนำเสนอครั้งแรกในงานวิจัยชื่อ "Squeeze-and-Excitation Networks" โดย Jie Hu และคณะ ซึ่ง SE Block สามารถเพิ่มความสามารถในการรับรู้ของแบบจำลองได้โดยการสร้างความสัมพันธ์ระหว่าง Channel ส่งผลทำให้โมเดลสามารถโฟกัสที่ Feature map ที่สำคัญได้มากขึ้น จึงทำให้ประสิทธิภาพโดยรวมสูงขึ้น การทำงานของ SE Block ประกอบด้วย 2 ส่วน ได้แก่

- **Squeeze (Global Average Pooling)**

คือการ "บีบอัด" ข้อมูลโดยการใช้ Global Average Pooling ซึ่งจะลดมิติข้อมูลเชิงพื้นที่ (ความสูงและความกว้าง) ของ Feature map ลงเหลือค่าเดียวต่อ Channel ทำให้ได้ตัวบ่งชี้ Channel ที่จับข้อมูลเชิงพื้นที่แบบ Global ของแต่ละ Channel สำหรับ Feature map ที่มีมิติ HxWxC (ความสูง H, ความกว้าง W, และจำนวนช่องทาง C) ขั้นตอนนี้จะได้ผลลัพธ์เป็นเทนเซอร์ 1×1×C

อ้างอิง https://towardsdatascience.com/introduction-to-squeeze-excitation-networks-f22ce3a43348



- **Excitation (Fully Connected Layers)**

หลังผ่านกระบวนการ Squeeze ข้อมูลจะผ่านไปยังชั้น Fully Connected (FC) จำนวน 2 ชั้นเพื่อจับความสัมพันธ์ระหว่าง Channel โดยชั้น FC แรกจะลดมิติข้อมูลลงและชั้น FC ที่สองจะคืนมิติข้อมูลกลับไปเหมือนเดิม ซึ่งผลลัพธ์จาก FC แรกจะผ่านฟังก์ชันการกระตุ้น (นิยมกำหนดให้เป็น ReLU) และผลลัพธ์จาก FC ที่สองจะผ่านฟังก์ชันการกระตุ้น (นิยมกำหนดให้เป็น sigmoid) เพื่อสร้างชุดน้ำหนักที่ปรับแต่ละ Channel ได้ (per-channel modulation weights) ซึ่งน้ำหนักเหล่านี้จะกำหนดความสำคัญของแต่ละ Channel

- **Recalibration (Channel-wise Multiplication)**

คือ การปรับ Feature map โดยการคูณน้ำหนักที่ปรับแต่ละช่องทางกับ Feature map เดิม การคูณนี้ช่วยปรับค่าการตอบสนองของ Feature map ให้ความสำคัญกับ Channel ที่สำคัญและลดความสำคัญของ Channel ที่ไม่สำคัญ

อ้างอิง https://towardsdatascience.com/introduction-to-squeeze-excitation-networks-f22ce3a43348